TSIX Course

English Edition

25 module
Kernel, VFS, Networking, GUI & Desktop, Development

System design & architecture: Andriansah

Generated: 2026-08-13

Platform: TSIX (educational OS-like runtime on Node.js/TypeScript)

Contents

Module 00 **Overview & Mental Map** Foundations

RFC-TSIX-000 | Mental map of the TSIX architecture — for contributors (human & AI), hobbyists, educators, and professionals.

> This document is the **system's mental map**. It explains *what*, *where*, and *why* — not a per-file tutorial. Use it as an entry point before reading the code, and as a reference while exploring subsystems.
>
> This is **Module 00** of the TSIX curriculum. For the full roadmap, see `toc.md`.

Table of Contents

---

1. Philosophy & Big Picture

TSIX is a **simulated operating system built on Node.js + TypeScript**. It is not a VM that emulates a CPU — it builds **OS abstractions on top of the existing Node.js runtime**.

Three core principles:

1. **"Syscall = the only door"** — Worker threads never touch resources directly. Even `print` is a syscall. This keeps the ring boundaries truly meaningful.
2. **"Everything is a File"** — files and devices are both `IDevice`; `read/write` is polymorphic. Open a regular file → `FileSystemDevice`; open `/dev/tty1` → `TTYDevice`.
3. **"Direct Memory Execution"** — the framework (`/lib`) is pre-compiled into memory at boot, sent to workers via `workerData`, and executed without hitting the filesystem. `@tsix/*` feels instant.

Key: the kernel never runs apps. All of userland (including PID 1 / init) runs in Worker Threads. The Ring 1/2 vs Ring 4 boundary is a *thread + IPC* boundary, reinforced by the uid/gid/mode-based `PermissionManager`.

---

2. Ring Model & Privilege Boundaries

Rings are a **concept** (documentation), not a hardware or V8 isolation mechanism:

Ring	Contents	Main files
0	Host: Linux + Node/V8	— (reserved)
1	Kernel core: Scheduler, Syscall, Permission	`src/kernel/*`, `src/common/*`
2	Driver & FS: HAL devices, VFS backends, MountManager	`src/kernel/devices/*`, `src/vfs/*`
3	Library framework: UserLib, Application	`src/mirror/lib/*`
4	Applications: `/bin/*`, `init`, `tsh`, daemon	`src/mirror/bin/*`

The **real privilege boundaries** live in two layers:

- **WorkerEntry sandbox** — apps may only `require` the `@tsix/*` / `@common/*` framework; `process.exit` / `process.kill` are sabotaged. Privileged apps (names containing `server` / `daemon` / `dome` / `tbuild` / `vfs`) get access to an allow-list of host modules (`http`, `ws`, `fs`, `crypto`, `esbuild`, etc.).
- **PermissionManager (kernel)** — rwx checks: root (uid 0) bypass → owner → group → others. The SetUID bit is supported (e.g. `/bin/login` `0o4755`).

> ⚠️ **Contributor note**: privileged status is based on **name substrings** — a fragile heuristic. This is an architectural gap worth fixing (ideally capability-based).

---

3. Boot Sequence

```
main.ts
  Config.load()                    → baca sysconfig.json
  new Kernel()                     → Logger, PermissionManager, MountManager, GUIRegistry
  kernel.boot()
    initializeSubsystems()
      mount BKFS("/")              → SQLite system.db (root filesystem)
      processFstab()               → /tmp (RamFS), /mnt/* (HostVFS), /mnt/sbak (BKFS)
      new Scheduler()
      new PermissionManager()
      new PortManager()
      new SyscallDispatcher()      → scheduler.setSyscallHandler(...)
      rebuildVFSCache()            → pre-compile /lib ke memori
    new TTYManager(32) + TTYDevice tty1..32
    devices{}                      → stdin, fb0, stdout, stderr, null
    init network interfaces        → SimpleMQTNLDriver per config
    loadAuxDevices()               → plugin driver (random, mysql, mcp23017)
    SerialDeviceManager            → auto-detect ttyUSB*
    applyDeviceConfigs()           → "udev": mode/uid/gid dari sysconfig
    pastikan /dev ada di VFS
    load RSA identity              → fingerprint visual di TTY
    ensureDefaultAuth()            → seed /etc/passwd + /etc/shadow (root)
    wire keyboard + TTY callbacks  → Ctrl+C → SIGINT fg; Alt+F1-6 → switch
  kernel.runInit()
    resolve /bin/init.js
    createProcess("init", fds:[tty1×3])   → PID 1
    setForegroundProcess(1, 1)
  [init.ts di Worker]
    enforce setuid (passwd, sudo)
    generate/verify RSA identity
    exec /etc/rc.local.js + waitpid
    spawn login TTY2..6 (monitorProcess → respawn)
```

```
    spawn login TTY1 (foreground) → loop forever
  keepAlive (100ms di main.ts)
    jika PID 1 EXITED → process.exit(exitCode)  (1 = reboot)
```

rc.local order (daemons): `airtermd` (remote access) → `tpkgd` (package server) → `scpd` (SCP) → `otad` (ESP OTA) → `iot-listener` (MQTNL sensor) → `dome` (display server, must run after the listener) → Asteracea WM (1s delay, waits for DOME) → `crond`.

---

4. Kernel Core

4.1 Processes (PCB)

`interface PCB` in `Scheduler.ts`: `pid`, `ppid`, `name`, `state`, `pc`, `owner/uid/gid/ruid/groups`, `cwd`, `worker?`, `fdTable`, `env`, `exitCode?`, `ttyId?`, `uuid?`.

Lifecycle: `READY → RUNNING → BLOCKED → EXITED`

- **Spawn**: `createProcess()` → allocate pid → PCB → `spawnWorker()` (Worker Thread).
- **Exit**: `EXIT(code)` → FD cleanup → `worker.terminate()`. The `worker.on("exit")` event → reparent orphans to PID 1 → resolve waitQueue → `reap()` (zombie if there is no waiter).
- **waitpid**: if EXITED → reap and return exitCode; if not yet → register in `waitQueue`.
- **daemonize**: `DETACH` → resolve waiters, detach the foreground TTY, clear ttyId.
- **REEXEC**: terminate the old worker, spawn a new one with the **same PID/PCB** (`REEXECING` state prevents the cleanup hook).

4.2 Syscall Dispatch

```
App → UserLib.dispatch(code, args)
  → postMessage({ requestId: uuid, pid, code, args })
  → Kernel: scheduler → syscallHandler.handleRequest(req)
      → validateArgs(code, args)          // kontrak argumen
      → dispatch(pid, code, args)         // switch-case ~65 syscall
      → PermissionManager.check()         // satpam
      → MountManager.resolve()            // path → backend
      → VFS / device / scheduler
  → postMessage({ requestId, success, data|error })
  → UserLib: cocokkan requestId di responseMap → resolve/reject
```

Request-response correlation via `requestId` (UUID) + `responseMap` — pure asynchronous RPC, no ordering guaranteed. Many syscalls can be in-flight at once.

Push events (kernel → worker, without requestId): `{ type, data }`. Types: `signal`, `ipc_message` (SEND_MSG), `gui_request` (forwarding to gued), `resize`.

4.3 Signals

Signal	Effect
SIGKILL (9)	Hard kill: `worker.terminate()` immediately
SIGINT (2)	Graceful: push event, 100ms grace → terminate if not handled. Default exit 130
SIGTERM (15)	Graceful: 300ms grace. Default exit 143. Used by SHUTDOWN
SIGSTOP/SIGCONT	Soft: change `pcb.state` to BLOCKED/RUNNING + event
SIGSEGV	Sent to the PID that violates GUI window ownership
SIGHUP/USR1/USR2/WINCH	Generic push events

PID 1 is protected from kill — only `SHUTDOWN`/`REBOOT` (root) are valid. `SHUTDOWN` is staged: SIGTERM broadcast → 5s → SIGKILL survivors → flush network 1s → kill PID 1.

4.4 MountManager & PortManager

- **MountManager**: array of `MountPoint {vfsPath, vfs, type, source, readOnly, uid, gid}`. `mount()` sorts descending by path length; `resolve()` → the longest prefix wins → `{vfs, relativePath, mountPoint}`.
- **PortManager**: virtual ports 0–65535 for MQTNL; `allocatePort`, `allocateRandomPort(10000-20000)` for bind(0), `releasePortsByPid()` on process exit (safety net for leaked sockets).

---

5. VFS & Device Drivers (HAL)

5.1 VFS Layers

```
Aplikasi → syscall OPEN/READ/WRITE
  → MountManager.resolve(path) → backend IVFS
      "/"        → BKFS    (SQLite system.db, persisten)
      "/tmp"     → RamFS   (volatile, in-memory)
      "/mnt/*"   → HostVFS (bridge folder host, anti-escape)
  → path /dev/xxx → kernel.devices[xxx]  (HAL, bypass MountManager)
```

- **IVFS** = a single contract (`ls/mkdir/read/touch/stat/chmod/chown/.../readChunk/writeChunk/getSize`) for all backends — allows "swapping backends" without changing syscalls.
- **BKFS**: a file = a row in the `vnodes` table (parent_id tree). Chunked I/O runs **inside SQL** (`SUBSTR`/`CONCAT`) without pulling the full content — for large files/OTA.
- **HostVFS**: `toHostPath()` + anti-escape checks (Security Violation) — read-only view of a host folder.

5.2 Device Model (HAL)

`IDevice`: `read/write/ioctl` + optional `init/open/close` + metadata `uid/gid/mode`.

/dev/	Class	Function
`stdin`	KeyboardDevice	Keyboard input (cooked/raw)
`fb0/stdout/stderr`	TTYDevice	Standard output → active TTY
`tty1..tty32`	TTYDevice	Virtual console
`null`	NullDevice	Black hole
`smqtnl0/1`	SimpleMQTNLDriver	MQTNL network interface
`randomdevice`	RandomDevice	Random numbers
`mysql` *(experimental)*	MySQLDevice	External DB connection — integration POC, see note below
`mcp23017`	MCP23017Device	I2C GPIO
`ttyUSB*`	SerialDevice	Auto-detect
(virtual)	PipeDevice / SocketDevice	Runtime instances (PIPE/SOCKET syscalls), not paths

The "everything is a file" key: the FD table holds `FDEntry {device, context, flags}` → all `IDevice` objects. Regular files are wrapped in `FileSystemDevice`. Pipe refcount via ioctl (`INC_REF`/`DEC_REF`), EOF when `writeRefs==0`.

Plugin drivers: the `aux-devices/` folder is scanned at boot; `new DeviceClass()` + the `static autoRegister(kernel)` convention for hardware (MCP23017). `applyDeviceConfigs()` = "udev" from `sysconfig.json`.

> [!NOTE] **About `MySQLDevice` (`/dev/mysql`) — the first transport, not the only one** `MySQLDevice` is an external database integration through the device model — it is **not** a real hardware driver, but the **first transport** for DB access (an example of HAL extension). **`DbLib` is already implemented** (a UserLib sub-library, following the `lib.fs`/`lib.net` pattern) with **pluggable dual transport**:
>
> - **Device** (`/dev/mysql`): `DB_*` syscalls (67-69) → kernel → device → `mysql2`
> - **Service daemon** (`mysqld`): `DB_*` syscalls → kernel → Ring 4 daemon → `mysql2`

> The kernel routes **dynamically**: if `mysqld` is registered (syscall `DB_SERVICE_REGISTER` =70) → to the daemon; otherwise → to the device (fallback). The daemon replies via `DB_SERVICE_REPLY` =71. Apps only see `db.connect()/query()/disconnect()` — the medium is invisible. The sandbox is safe because `mysql2` is only touched by the kernel or a privileged daemon.

---

6. Worker Thread & Sandboxing

6.1 Bootstrap Worker

```
Kernel.spawnWorker()
  workerData = { pid, appName, args, appPath, appContent, env, vfsCache }
  execArgv:
    *.js  → JS-Direct (FAST)   : --enable-source-maps
    *.ts  → TS-Transpile       : -r esbuild-register -r tsconfig-paths/register

WorkerEntry.ts (bootloader)
  1. realExit = process.exit (sebelum sandbox)
  2. hijack Module._load → resolve @tsix/* & @common/* dari vfsCache (DME)
  3. pasang unhandledRejection/uncaughtException → kirim error ke parent
  4. new UserLibClass(pid) → global._tsixLib
  5. muat app:
       DME  : appContent di-transpile, __filename = BKFS path
       fisik: hostRequire(finalAppPath)
  6. cari export main/Main/default
  7. restrictHostAPI(appName)   ← kunci pintu
  8. new AppClass() → await execute(lib, args)
  9. return string → std.print; lalu shell.exit(0)
```

6.2 Direct Memory Execution

The kernel pre-compiles `/lib` → `vfsCache` → sent via `workerData` → the worker `_compile`s from memory. Key trick: the framework module is named `@tsix_Application.js` so relative imports `./x` can be rewritten to `@tsix/x` — a consistent alias cycle. Apps have **two filename identities**: physical (so `require` finds node_modules) vs BKFS (so stack traces are correct).

6.3 Sandbox Table

Layer	Restrictions
All apps	`process.exit` / `process.kill` → throw; non-framework `require` blocked
Non-privileged apps	`http/fs/crypto` etc. fully blocked
Privileged apps	allow-list: `http, ws, path, fs, url, esbuild, crypto, os, bcryptjs`
Kernel	`PermissionManager` + `validateArgs` + root-only `SETUID`

---

7. Networking MQTNL

MQTNL (MQTT Network Layer) = the TSIX networking protocol. Instead of IP / TCP/IP routing, it uses **MQTT pub/sub as the wire**, plus:

- **Address = node name** (string: `"tsix"`, `"esp32S3"`) — not an IP.
- **Port = application endpoint** (PortManager 0–65535).
- **Topic =** `mqtnl@1.0/<dstAddress>` (JSON) / `mqtnl@1.1/<dstAddress>` (Binary).
- **Packet**: 9-field header + payload; 32KB fragmentation; 30s reassembly TTL; RSA + ChaCha20-Poly1305 encryption.

```
App → socket() → bind(port) → driver.registerHandler(port)
  → sendto(addr, port, data) → driver.send → publish topic
```

```
  → MQTT broker → semua node subscribe 'mqtnl@1.x/#'
  → filter dstAddress → reassembly → decrypt → socket.push()
  → recvfrom() → buffer.shift()
```

Dual protocol: `MQTNLProtocolJSON` (v1.0, legacy, readable) vs `MQTNLProtocolBinary` (v1.1, compact, no encryption — for byte-exact OTA). The driver detects from the **first magic byte** per srcAddress. `src/common/protocols/` holds the `IMQTNLProtocol` contract — userland **never touches this directly**.

Syscalls: `SOCKET=30, BIND=31, SENDTO=32, RECVFROM=33, NETSTAT=34`. There is no `listen/accept` — UserLib emulates them: `listen()` = socket+bind, `accept()` = polling `recv()`.

---

8. TTY & Virtual Console

- **TTYManager** manages 1–32 virtual consoles; switching only 1–12, login on 1–6.
- **TTY** (`src/kernel/tty/TTY.ts`): `char[x][y]` buffer, cursor, ANSI subset parser, full re-render.
- **TTYDevice** (`/dev/ttyN`): ioctl clear(1), raw(10), inject input(0x2001), read output(0x2002), window size(4) — lets remote apps (pixelterm) control a TTY.
- **TTY allocation to processes**: via `ttyId` in the `EXEC` syscall — the kernel overrides FD 0/1/2 to `tty{n}`. (Different from PortManager, which is network-only.)
- **Keyboard**: cooked/raw mode, Ctrl+C → SIGINT to fg, Alt+F1-6 → TTY switch, resize → SIGWINCH broadcast.

---

9. Userland

9.1 Init (PID 1)

No `/etc/inittab` — the logic is hardcoded in `init.ts`: enforce setuid → RSA identity → exec `rc.local` → spawn login per TTY (via `monitorProcess` respawn, anti-crash-loop) → loop.

9.2 Login → Shell

`login.ts`: verify `/etc/passwd` + `/etc/shadow` (bcrypt) → `setgroups → setgid → setuid` (POSIX order) → exec `$SHELL` → `tsh.ts`.

9.3 Two Application Styles

1. **Class `IProgram`** (majority legacy): `export class main implements IProgram { async execute({fs,shell,std}: OSContext, args) {...} }` — `ls`, `cat`, `ps`, `kill`, `tsh`.
2. **`Program()` wrapper + proxy singletons** (new): `import { Program, std, fs, shell, net } from "@tsix/Application"` — `esp-send`, `dome`, `iot-dashboard`.

9.4 Error = "Window"

`std.error()` does 4 things: write syslog → broadcast to parent → print red on TTY → send `GUI_WINDOW_ERROR` to the WM (with `wid`, `pid`, `fileHint` from the stack trace).

9.5 DbLib — Database Sub-Library

`DbLib` (`@tsix/DbLib`, `lib.db`) is the fifth UserLib sub-library (after std/fs/shell/net). It wraps the `DB_*` syscalls (67-71) into the `db.connect()/query()/disconnect()` API. **Pluggable transport**: the kernel routes to `/dev/mysql` (device) or `mysqld` (Ring 4 service daemon) dynamically. Apps don't know the medium — exactly like the `lib.fs` pattern, which doesn't care about its VFS backend.

```
App → db.query(sql) → dispatch(DB_QUERY=68)
  → Kernel: mysqld terdaftar?
       YA  → sendEvent(daemon, "db_request") → mysql2 → dbServiceReply → resolve
       TIDAK → /dev/mysql device → mysql2 → resolve
  → App: rows
```

---

10. PixelSpace & TDE

> Full details in `PIXELSPACE_DEVELOPER_GUIDE.md`. Architectural summary:

```
Worker (app) → GUI_REQ (61) → Kernel (GUIRegistry auth pid↔wid)
  → DOME daemon (WS broadcast) → Browser (DOM)
Browser → event → DOME → Kernel (SEND_MSG) → Worker (callback)
```

- **Contract**: `GUITypes.ts` (IDOMNode, IGUIPayload, GUIAction, IBrowserEvent, IGUIEventIPC) — the "constitution", don't change it carelessly.
- **GUIRegistry (kernel)** = the single authority over window ownership: `CREATE_WINDOW` = register `wid↔pid`; accessing someone else's window → SIGSEGV; corrupted payload → SIGKILL; process death → window auto-destroy.
- **DOME** = display server (Ring 4 daemon, port 8080): WS relay + DOM primitive producer + **compositor** (titlebar, drag, resize, focus, replay). Monolithic due to drag/resize latency considerations.
- **Emerald** = widget toolkit (`@tsix/emerald`): `Screen`, `Window`, factory functions, connected widgets.
- **Cashew** = component framework (`@tsix/cashew`): OOP/Delphi-style `TForm` / `TButton` / `TEdit`, auto-bind lifecycle, TDialogs & TTimer — a layer above Emerald.
- **Asteracea** = window manager (fullscreen frameless app): taskbar, launcher, login, wallpaper; listens to `GUI_WINDOW_*` lifecycle events via `/etc/asteracea/wm-pid`.

---

11. Development Flow

Dev cycle (host ↔ VFS):

```
scripts/vfs-bootstrap.ts   host → system.db (transpile TS→JS, chmod, setuid)
scripts/sync-vfs.ts        sync satu file host ↔ VFS
scripts/vfs-pull.ts        system.db → src/root (pull balik)
SYNC_TO_HOST (syscall)     DB → host (app dalam VFS menulis /lib → src/.tsix_sdk)
userlib-update.ts          sinkronkan /lib → src/.tsix_sdk/lib (agar Node.js host bisa require)
```

Configuration: `src/sysconfig.json` (database path, workerEntryPath, bootEntry, network interfaces). `tsconfig.json` maps `@tsix/*` → `src/.tsix_sdk/lib/*`, `src/root/lib/*`, `src/mirror/lib/*`.

---

12. Design Insights & Gotchas

Insights (why it was designed this way):

1. **Syscalls as a mini ABI** — `SyscallCode` (1–66) + `validateArgs` + `requestId` correlation. One mechanism for everything: files, processes, IPC, GUI, networking.
2. **Pure asynchronous RPC** — request/response via UUID; different from Linux synchronous traps.
3. **Staged kill** — SIGINT/SIGTERM = event + grace period, then terminate. Replicates the Unix default (unhandled signal → exit).
4. **Faithful zombies & reparenting** — orphans → PID 1; zombies wait for `waitpid` + `reap`.
5. **UUID identity** — `SET_IDENTITY` allows `SEND_MSG` to a stable name even when the PID changes (well-known services).
6. **GUI auth in the kernel** — `payload.pid` is overridden by the kernel (don't trust userland).
7. **MQTNL = free no-IP** — the MQTT broker is the backbone; by design for IoT (ESP32) without a public IP.
8. **TTY = full PTY emulation** — master-side I/O via ioctl enables remote terminals.

Gotchas (be careful when contributing):

⚠️ **Stale compile artifacts**: `src/mirror/lib/*.js` and `src/userland/WorkerEntry.js` can differ from their `.ts` sources. The runtime uses the `.ts` recompiled by the kernel, so committed `.js` files are prone to drift (example: the privileged list in `WorkerEntry.js` doesn't include `dome`).

⚠️ **Docs vs code**: `rc.local.ts` (source) holds the full daemon list, but `rc.local.js` (the executed one) only has 3 daemons. The wiki `Networking-MQTNL.md` mentions the topic `tsix/net/*` but the code uses `mqtnl@1.0/`. **Code is the truth.**

⚠️ **Dead code**: `ExecutableRegistry` is defined but not wired into the Kernel — binary resolution goes through `MountManager.resolve()` + `vfs.read()`.

⚠️ **No `inittab`** — init hardcodes the TTY logins.

⚠️ **Dual NetworkLib** — `UserLib.ts` (inline, `lib.net`) vs `src/mirror/lib/NetworkLib.ts` (legacy, OSContext-based) both target the same syscalls.

⚠️ **Name-based privilege** — app name substring heuristic (fragile; ideally capability-based).

---

13. Code Reading Map

Start here (required)

File	Role
`src/main.ts`	Host entry point + keep-alive
`src/kernel/Kernel.ts`	Boot orchestrator for all subsystems
`src/common/IPCTypes.ts`	IPC contract (request/response/event)
`src/common/SyscallCode.ts`	Syscall ABI (1–66)
`src/common/GUITypes.ts`	PixelSpace contract

Kernel Core

File	Role
`src/kernel/Scheduler.ts`	PCB, processes, signals, reexec
`src/kernel/Syscalls.ts`	~65 syscall dispatcher + permission
`src/kernel/PermissionManager.ts`	rwx checks
`src/kernel/MountManager.ts`	Path → backend routing
`src/kernel/PortManager.ts`	Network ports
`src/kernel/GUIRegistry.ts`	Window authority

VFS & Devices

File	Role
`src/vfs/IVFS.ts`	Filesystem contract
`src/vfs/BKFS.ts`	Root FS (SQLite)
`src/vfs/VFS.ts` / `RamFS.ts` / `HostVFS.ts`	Other backends
`src/kernel/devices/IDevice.ts`	HAL contract
`src/kernel/devices/FileSystemDevice.ts`	File ↔ IVFS bridge
`src/kernel/devices/*.ts`	Drivers: TTY, Keyboard, Pipe, Socket, MQTNL
`src/kernel/devices/aux-devices/`	Plugin drivers

Worker & Userland

File	Role
`src/userland/WorkerEntry.ts`	Worker bootloader + sandbox + DME
`src/mirror/lib/UserLib.ts`	Worker-side "libc"
`src/mirror/lib/Application.ts`	Program() v2.1 framework
`src/mirror/lib/emerald.ts`	Widget toolkit
`src/mirror/bin/init.ts`	PID 1

Networking & Protocols

File	Role
`src/kernel/devices/SimpleMQTNLDriver.ts`	MQTNL driver
`src/common/protocols/IMQTNLProtocol.ts`	Protocol contract
`src/common/protocols/MQTNLProtocolJSON.ts`	v1.0
`src/common/protocols/MQTNLProtocolBinary.ts`	v1.1

GUI/TDE

File	Role
`src/mirror/bin/dome.ts`	DOME server
`src/mirror/bin/dome-client.html`	DOME browser
`src/mirror/bin/asteracea.ts`	WM
`src/mirror/lib/theme.ts`	Theme

Appendix: Key data flow summary

Read a file:

```
fs.readFile("/etc/passwd")
→ OPEN: resolve → MountManager → BKFS → FileSystemDevice → fd
→ READ: fdTable[fd].device.read() → vfs.read → SQL SELECT
→ CLOSE: ioctl DEC_REF → fdTable[fd] = null
```

Spawn an app:

```
shell.exec("/bin/ls")
→ EXEC: resolve → vfs.read(appContent) → permission EXECUTE
→ createProcess → spawnWorker → WorkerEntry → DME app → execute()
```

Send a message between apps:

```
shell.send(pidOrUuid, {type, ...})
→ SEND_MSG → kernel resolve pid (atau uuidMap) → sendEvent(pid, "ipc_message", data)
```

Open a window:

```
Screen → CREATE_WINDOW → kernel (GUIRegistry.auth) → DOME → WS → browser
```

TSIX Architecture Guide v1.0 — for contributors & developers. This document is alive — update it as the code changes. "Code is the truth."

Module 01 **Philosophy & Big Picture** Foundations

RFC-TSIX-EDU-002 | The first module of the TSIX curriculum. Builds a mental map: what TSIX is, why it was designed this way, and the four core principles that form the foundation of the entire system.

> Before writing code, understand the **philosophy** first. TSIX is not just an "OS emulator" — it is an **OS abstraction built on top of the existing Node.js runtime**. This module explains *why* it was designed this way, and the four principles that are consistent across all subsystems.

Learning Objectives

- [ ] Explain what TSIX is and how it differs from a VM/emulator
- [ ] Name the four core principles of the TSIX architecture
- [ ] Explain why the kernel, drivers, and FS are combined into a single thread
- [ ] Recognize the Ring 0–4 layers and their contents in broad outline
- [ ] Explain why the kernel never executes applications

Core Concepts

What is TSIX?

TSIX is a **simulated operating system based on Node.js + TypeScript**. It is not a VM that emulates a CPU. It builds an **OS abstraction on top of the existing Node.js runtime** — using `Worker Thread` as the process boundary, `postMessage` as IPC, and `SQLite` as the filesystem.

With this approach, TSIX adopts **UNIX architecture concepts** — process isolation, UID/GID permissions, POSIX filesystem, signal handling, device abstraction (HAL), syscall communication — expressed in TypeScript on top of Node.js.

Why not a microkernel?

One of the most fundamental architectural decisions: **the kernel, drivers, and filesystem are combined into a single main thread**, not split into separate threads like a pure microkernel (Minix, QNX, seL4).

Reason	Explanation
IPC overhead	A pure microkernel = 3–4x `postMessage` per operation (App → Kernel → FS → Kernel → App). A single thread = 1x (App → Kernel). Each `postMessage` performs a structured clone (serialize → deserialize).
Worker limitation	Node.js Worker Threads **cannot use shared memory**. Each transfer = new allocation + data copy. The larger the data, the larger the overhead.

Reason	Explanation
Embedded target	TSIX is for IoT with limited CPU and small RAM. Each additional Worker = ~4–8MB heap.
Isolation already exists	Applications (Ring 4) are already in a separate worker → 90% of the isolation benefit at 10% of the cost.

The result: the most frequent crashes are user applications — and they are already isolated. Kernel + drivers + FS rarely crash, and even if they do, the system dies (but that is rare).

Four Core Principles

1. "Syscall = the only door"

Worker threads **never touch resources directly**. Even `print` goes through a syscall. This keeps the ring boundary truly meaningful — there are no shortcuts.

```
App → UserLib.dispatch(code, args) → postMessage → Kernel → dispatch → kembali
```

2. "Everything is a File"

Files and devices are both `IDevice`; `read/write` is polymorphic.

- Opening a regular file → `FileSystemDevice`
- Opening `/dev/tty1` → `TTYDevice`
- Opening `/dev/smqtnl0` → `SimpleMQTNLDriver`

Even pipes and sockets are devices. One contract, many implementations.

3. "Direct Memory Execution" (DME)

The framework (`/lib`) is pre-compiled into memory at boot, sent to workers via `workerData`, and executed **without a filesystem hit**. `@tsix/*` feels instant — there is no recompilation per process.

4. "Unix fidelity first, pragmatic later"

TSIX mirrors Unix/Linux behavior and architecture as closely as practical — syscall semantics, UID/GID permissions, credentials (saved UID), file formats (`/etc/shadow` , `passwd`), filesystem hierarchy — as the design **north star**. Not because it "must match exactly", but because **observable** behavior must be consistent: non-root cannot read `/etc/shadow`, a non-root process cannot `setuid` arbitrarily, and so on.

Deviating from Unix is **allowed**, but only when the Node.js/V8 runtime truly cannot model it — never as a "it's easier this way" shortcut. Every deviation **must be documented** (a changelog entry and/or a code comment) with a clear technical reason, so it is not mistaken for a bug.

> [!NOTE] **Semantics > Mechanisms** — there is no need to emulate bare-metal (hardware interrupts, MMU, etc.). What matters is that behavior observed from userland matches Unix. Example: `setuid` is simulated with a saved UID (`pcb.suid`) in the kernel, not with a CPU register — but the effect is the same.

Source Code

File	Role
`src/main.ts`	Host entry point + keep-alive
`src/kernel/Kernel.ts`	Boot orchestrator for all subsystems
`src/kernel/Syscalls.ts`	Syscall dispatcher

File	Role
`src/kernel/Scheduler.ts`	Process management
`src/userland/WorkerEntry.ts`	Worker bootloader + sandbox
`src/common/IPCTypes.ts`	IPC contract

Exercises / Practice

1. Read `src/main.ts` — find where the 100ms keep-alive is. What happens if PID 1 exits with code 1?
2. Read `src/kernel/Kernel.ts` — list the order of `initializeSubsystems()`. Compare it with the boot diagram in Module 03.
3. Run TSIX (`npm start` / per `README.md`) and observe the boot log.

References

- `wiki/Arsitektur-Sistem.md` — layer diagram and microkernel analysis
- `wiki/course/00-overview.en.md` §1 — global mental map
- `src/main.ts`, `src/kernel/Kernel.ts`

Module 01 — complete. Continue to Module 02 — Ring Model & Privilege Boundaries.

Module 02 **Ring Model & Privilege Boundaries** Foundations

RFC-TSIX-EDU-002 | The second module of the TSIX curriculum. Understand the "Ring" concept as a privilege boundary, and where the *actual* security boundary sits in TSIX.

> Important: **A Ring in TSIX is a concept (documentation), not a hardware isolation mechanism.** TSIX runs on top of V8, not on bare metal. The real boundaries live in two layers: the **WorkerEntry sandbox** and the **PermissionManager (kernel)**.

Learning Objectives

- [ ] Explain the contents of each Ring 0–4 and its main files
- [ ] Distinguish a Ring as a concept vs. a real isolation mechanism
- [ ] Explain the two real privilege boundary layers: the WorkerEntry sandbox & the PermissionManager
- [ ] Explain what the setuid bit is and an example of its use
- [ ] Know the weakness of privilege based on app name

Core Concepts

Ring Map

Ring	Contents	Main files
0	Host: Linux + Node/V8	— (reserved)
1	Kernel core: Scheduler, Syscall, Permission	`src/kernel/*`, `src/common/*`
2	Drivers & FS: HAL devices, VFS backends, MountManager	`src/kernel/devices/*`, `src/vfs/*`
3	Library framework: UserLib, Application	`src/mirror/lib/*`
4	Apps: `/bin/*`, `init`, `tsh`, daemon	`src/mirror/bin/*`

 Ring 1 & 2 — kernel internals (dispatcher, scheduler, FS/Net/HAL stacks) *Source:* *`wiki/diagram/Arsitektur-Sistem-2.mmd`*

A Ring is a **map of responsibility** — not isolation enforced by V8. Two files from different rings may call each other; what keeps the system secure are the rules below.

Comparison with Linux

Feature	Linux Equivalent	TSIX Ring
Core OS Logic	Ring 0 (Kernel Mode)	**Ring 1**
Drivers & FS	Ring 0 (Kernel Mode)	**Ring 2**
Standard Library	Ring 3 (glibc)	**Ring 3**
User Apps	Ring 3 (User Mode)	**Ring 4**

> [!NOTE] **Ring 0 is the host domain.** Because TSIX is not bare metal, "Ring 0" means Linux/Windows + V8. TSIX starts numbering from Ring 1.

The Two Real Privilege Boundary Layers

Layer 1 — WorkerEntry Sandbox (worker side)

`src/userland/WorkerEntry.ts` is the **bootloader** of every worker. It locks the door before the app runs:

- An app may only `require` the `@tsix/*` / `@common/*` frameworks — everything else is blocked.
- `process.exit` / `process.kill` are sabotaged (thrown).
- **Privileged** apps (name contains `server`, `daemon`, `dome`, `tbuild`, `vfs`) get an **allow-list** of host modules: `http`, `ws`, `path`, `fs`, `url`, `esbuild`, `crypto`, `os`, `bcryptjs`.

Layer 2 — PermissionManager (kernel side)

`src/kernel/PermissionManager.ts` is the "security guard" in the kernel. It performs layered rwx checks:

```
check(pid, path, mode):
  root (uid 0) → bypass
  owner        → check owner bit
  group        → check group bit
  others       → check others bit
```

The **SetUID bit** (`0o4000`) is supported: a process runs with the file's ownership. Example: `/bin/login` has mode `0o4755` — whoever runs login, the process runs as root (to perform setuid/setgid to the target user).

Source Code

File	Role
`src/userland/WorkerEntry.ts`	Worker-side sandbox (restrictHostAPI)
`src/kernel/PermissionManager.ts`	rwx checks + setuid
`src/kernel/Syscalls.ts`	Call the permission check before an action
`src/mirror/bin/login.ts`	Example of setuid usage (`0o4755`)

Snippet (code level)

```
// src/kernel/PermissionManager.ts — core of the permission check (condensed)
check(pid: number, path: string, mode: number): boolean {
  const { uid, gid } = this.scheduler.getProcess(pid);
  const stat = this.getStat(path);
  if (uid === 0) return true;            // root bypass
  if (uid === stat.uid) return !!(stat.mode & mode);        // owner
  if (gid === stat.gid) return !!(stat.mode & (mode >> 3)); // group
  return !!(stat.mode & (mode >> 6));                       // others
}
```

> [!WARNING] **Known weakness.** Privileged status is based on **app name substring** — a fragile heuristic. Anyone who names their app `my-daemon` automatically becomes privileged. Ideally this should be replaced with *capability-based*.

Exercises / Practice

1. Read `src/userland/WorkerEntry.ts` — find the allow-list of host modules.
2. Read `src/kernel/PermissionManager.ts` — test the rwx check logic with several uid/gid/mode combinations.
3. Run a non-privileged app that tries to `require("fs")` — observe the error that appears.

References

- `wiki/ARCHITECTURE_RINGS.md` — official ring definitions
- `wiki/Keamanan-dan-Sandboxing.md` — sandbox details
- `wiki/course/00-overview.md` §2

Module 02 — done. Continue to Module 03 — Boot Sequence.

Module 03 **Boot Sequence** Boot & Kernel Runtime

RFC-TSIX-EDU-002 | Third module of the TSIX curriculum. Traces the boot sequence from `main.ts` → `kernel.boot()` → PID 1 (`init`) → login, up to the 100ms keep-alive on the host.

> TSIX boot mimics a UNIX/Linux boot: from host bootstrap → kernel subsystem init → init (PID 1) → rc.local → getty/login per TTY. What decides whether the system is "alive or dead" is the **100ms keep-alive** in `main.ts`.

Learning Objectives

- [ ] Explain the role of `main.ts` and the 100ms keep-alive
- [ ] List the order of `initializeSubsystems()` in the kernel
- [ ] Explain the content and role of `init` (PID 1)
- [ ] Explain the order of daemons in `rc.local`
- [ ] Explain what exit code 1 means when PID 1 dies (reboot)

Flow / How It Works

TSIX boot follows the classic UNIX/Linux pattern: host bootstrap → kernel subsystem init → `init` (PID 1) in a Worker → `rc.local` → getty/login per TTY → host keep-alive. The diagram below summarizes the entire sequence, from the first second the host starts until the system is "alive".

```
sequenceDiagram
    participant H as Host (Node.js)
    participant M as main.ts
    participant K as Kernel.ts
    participant V as VFS (BKFS/SQLite)
    participant W as Worker Thread (Ring 4)
    participant I as init (PID 1)
    participant R as rc.local
    participant L as login (TTY1-6)

    H->>M: node src/main.ts
    M->>M: Config.load() → sysconfig.json
    M->>K: new Kernel() (Logger, PermissionManager, MountManager, GUIRegistry)
    M->>K: await kernel.boot()

    activate K
    K->>K: initializeSubsystems()
    K->>V: mount BKFS "/" (system.db)
    K->>K: processFstab() → /tmp (RamFS), /mnt/shared, /mnt/sbak
    K->>K: new Scheduler() + new PermissionManager() + new PortManager()
    K->>K: new SyscallDispatcher() → scheduler.setSyscallHandler(...)
    K->>K: rebuildVFSCache() → pre-compile /lib (DME)
    K->>K: TTYManager(32) + /dev devices (stdin, fb0, stdout, stderr, null, tty1-32)
    K->>K: network interfaces (SimpleMQTNLDriver) + loadAuxDevices() + SerialDeviceManager
    K->>K: applyDeviceConfigs() (udev) + pastikan /dev di VFS
    K->>K: load RSA identity → fingerprint visual
    K->>K: ensureDefaultAuth() + ensureDefaultGroups()
    K-->>M: boot() selesai

    M->>K: kernel.runInit()
    K->>V: resolve /bin/init.js (cfg.scheduler.bootEntry)
    K->>W: createProcess("init", fds:[tty1×3]) → PID 1
    K->>K: setForegroundProcess(1, 1)
    deactivate K

    activate W
    activate I
    I->>I: enforce setuid (/bin/passwd.js, /bin/sudo.js)
    I->>I: RSA identity: generate/verify keypair + fingerprint
```

```
    I->>R: exec /etc/rc.local.js + waitpid
    activate R
    R->>R: airtermd → tpkgd → scpd → otad → iot-listener
    R->>R: tunggu /var/run/dome.ready → dome → asteracea
    R->>R: crond → mysqld
    R-->>I: exit 0
    deactivate R

    I->>L: spawnLogin TTY2-6 (monitorProcess → respawn)
    I->>L: spawnLogin TTY1 (foreground)
    I->>I: while(true) { wait 10s }  ← PID 1 tak pernah exit
    deactivate I
    deactivate W

    M->>M: keepAlive: setInterval(100ms)
    M->>M: scheduler.getProcess(1) → EXITED?
    M-->>M: exitCode === 1 → reboot | else → power off
```

> [!NOTE] This mermaid version is more complete than `wiki/boot_sequence.md` and follows the current code (`Kernel.ts`, `init.ts`, `rc.local.ts`).

Concise flow (ASCII)

```
main.ts
  Config.load()                   → baca sysconfig.json
  new Kernel()                    → Logger, PermissionManager, MountManager, GUIRegistry
  kernel.boot()
    initializeSubsystems()
      mount BKFS("/")             → SQLite system.db (root filesystem)
      processFstab()              → /tmp (RamFS), /mnt/shared (HostVFS), /mnt/sbak (BKFS)
      new Scheduler()
      new PermissionManager()
      new PortManager()
      new SyscallDispatcher()     → scheduler.setSyscallHandler(...)
      rebuildVFSCache()           → pre-compile /lib ke memori (DME)
    new TTYManager(32) + TTYDevice tty1..32
    devices{}                     → stdin, fb0, stdout, stderr, null
    init network interfaces       → SimpleMQTNLDriver per config
    loadAuxDevices()              → plugin driver (random, mysql, mcp23017)
    SerialDeviceManager           → auto-detect ttyUSB*
    applyDeviceConfigs()          → "udev": mode/uid/gid dari sysconfig
    pastikan /dev ada di VFS
    load RSA identity             → fingerprint visual di TTY
    ensureDefaultAuth()           → seed /etc/passwd + /etc/shadow (root)
    ensureDefaultGroups()         → seed users (100) + sudo (27)
    wire keyboard + TTY callbacks  → Ctrl+C → SIGINT fg; Alt+F1-6 → switch
  kernel.runInit()
    resolve /bin/init.js
    createProcess("init", fds:[tty1×3])   → PID 1
    setForegroundProcess(1, 1)
  [init.ts di Worker]
    enforce setuid (passwd, sudo)
    generate/verify RSA identity
    exec /etc/rc.local.js + waitpid
    spawn login TTY2..6 (monitorProcess → respawn)
    spawn login TTY1 (foreground) → loop forever
  keepAlive (100ms di main.ts)
    jika PID 1 EXITED → process.exit(exitCode)  (1 = reboot)
```

`initializeSubsystems()` order (from `Kernel.ts`)

```
this.bootLogStart("VFS: Mounting root filesystem (BKFS/SQLite)");
this.bkfs = new BKFS(cfg.kernel.database);
this.mountManager.mount("/", this.bkfs, "bkfs", cfg.kernel.database, false);
this.bootLogEnd(true);

await this.processFstab();                    // /tmp (RamFS), /mnt/* (HostVFS/BKFS)
```

```
this.scheduler = new Scheduler();             // Core: Process Scheduler
this.satpam = new PermissionManager();        // Security: Permission Manager
this.portManager = new PortManager();         // Network: Port Manager
this.syscall = new SyscallDispatcher(
  this.bkfs, this.mountManager, this.scheduler, this, this.satpam,
);
this.scheduler.setSyscallHandler(async (req) => {
  return await this.syscall!.handleRequest(req);
});
this.rebuildVFSCache();                       // VFS: pre-compile /lib (DME)
this.scheduler.setVFSCacheProvider(() => this.vfsCache);
```

rc.local order (daemons, from `rc.local.ts`)

1. **SetUID** `/bin/login.js` → `0o4755`, chown root:root.
2. `airtermd` (`/sbin/airtermd.js`) — remote access.
3. `tpkgd` (`/sbin/tpkgd.js`) — package server (TPKG).
4. `scpd` (`/sbin/scpd.js`) — SCP file transfer.
5. `otad` (`/sbin/otad.js`) — ESP firmware OTA.
6. `iot-listener` (`/sbin/iot-listener.js`) — MQTNL sensor.
7. Clean old marker `/var/run/dome.ready`, then **wait for DOME ready** (polling 200ms, timeout 10s).
8. `dome` (`/opt/dome/dome.js`) — display server (PixelSpace).
9. `asteracea` (`/opt/asteracea/asteracea.js`) — window manager (after DOME is ready).
10. `crond` (`/sbin/crond.js`) — cron scheduler.
11. `mysqld` (`/opt/mysqld/mysqld.js`) — database service.

> [!WARNING] **Documentation vs code.** The `rc.local.ts` source contains the 11 steps above. However, the compiled `rc.local.js` file (the one actually executed) only contains 3 daemons: `airtermd`, `tpkgd`, `scpd`. **Code is truth** — at runtime, only the first 3 daemons actually run from `rc.local`.

Source Code

File	Role
`src/main.ts`	Host entry point + keep-alive
`src/kernel/Kernel.ts`	`boot()` + `initializeSubsystems()` + `runInit()`
`src/mirror/bin/init.ts`	PID 1: setuid, identity, rc.local, spawn login
`src/mirror/etc/rc.local.ts`	Startup daemon list
`src/common/Config.ts`	Reads `sysconfig.json`

Snippet (code level)

Keep-alive in `main.ts` (the host must not die)

```
const keepAlive = setInterval(() => {
    const scheduler = kernel.getScheduler();
    const p1 = scheduler?.getProcess(1);
    if (!p1 || p1.state === "EXITED") {
        clearInterval(keepAlive);
        if (process.stdin.isTTY) {
            (process.stdin as any).setRawMode(false);
        }

        const exitCode = (kernel as any).wantedExitCode ?? (process.exitCode ?? 0);
        if (exitCode === 1) {
```

```
            console.log("\n[Kernel] System is rebooting...");
        } else {
            console.log("\n[Kernel] System halted. Powering off...");
        }
        process.exit(exitCode);
    }
}, 100); // 100ms biar satset responnya
```

Key point: **as long as PID 1 is alive, the host does not stop.** When PID 1 exits with **code 1** → reboot; otherwise → power off. Before exiting, the terminal raw mode is restored so the host terminal is not left "broken".

Boot log: `bootLogStart()` + `bootLogEnd()`

`Kernel.ts` prints boot progress with a `[ ]` bracket that updates in place (`\r` + ANSI `\x1b[K`). Usage pattern:

```
this.bootLogStart("VFS: Mounting root filesystem (BKFS/SQLite)");
this.bkfs = new BKFS(cfg.kernel.database);
this.mountManager.mount("/", this.bkfs, "bkfs", cfg.kernel.database, false);
this.bootLogEnd(true);                    // → [  OK  ] VFS: Mounting root...
```

`bootLogEnd(false, "pesan")` → `[ FAIL ]`. Both are no-ops if `cfg.kernel.verbose` is false.

runInit: spawning PID 1

```
const initPath = "/bin/" + cfg.scheduler.bootEntry;   // bootEntry = "init.js"
const res = this.mountManager.resolve(initPath);
const raw = res.vfs.read(res.relativePath);           // kontrak IVFS, bukan stat()

const initPcb = this.scheduler?.createProcess("init", {
    fds: [tty1, tty1, tty1],
    appName: "init",
    appContent: initContent,
    env: {
        PATH: cfg.scheduler.defaultPath,
        HOME: "/root",
        HOSTNAME: cfg.shell.defaultHostname,
        PROMPT_FORMAT: cfg.shell.promptFormat,
        LINES: (process.stdout.rows || cfg.shell.defaultRows).toString(),
        COLUMNS: (process.stdout.columns || cfg.shell.defaultColumns).toString(),
    },
    cwd: cfg.scheduler.defaultCwd,
    ttyId: 1,
});
if (initPcb) this.scheduler?.setForegroundProcess(initPcb.pid, 1);
```

init: enforce setuid

```
// Runtime mengeksekusi sidecar .js (bukan source .ts), jadi chmod harus di .js
await lib.fs.chmod("/bin/passwd.js", 2541);   // 2541 = 0o4755 (setuid root)
await lib.std.print(`${ok} [INIT] SetUID bit applied to: /bin/passwd.js\n`);
await lib.fs.chmod("/bin/sudo.js", 2541);
await lib.std.print(`${ok} [INIT] SetUID bit applied to: /bin/sudo.js\n`);
await lib.std.print(`${ok} [INIT] System binary permissions enforced.\n`);
```

> [!NOTE] `2541` decimal = `0o4755` octal (setuid + rwxr-xr-x). This SetUID lets `passwd` and `sudo` elevate privileges to root even when run by a regular user.

init: exec rc.local + waitpid

```
const result = await lib.shell.exec("/etc/rc.local.js", [], undefined, undefined, undefined);
if (result) {
    const exitCode = await lib.shell.waitpid(result.pid);
    if (exitCode === 0) {
```

```
        await lib.std.print(`${ok} [INIT] Startup scripts completed successfully.\n`);
    } else {
        await lib.std.print(`Init: Warning - rc.local exited with code ${exitCode}\n`);
    }
}
```

init: spawnLogin + monitorProcess (respawn to avoid crash-loop)

```
const spawnLogin = async (ttyId: number) => {
    try {
        if (ttyId > 1)
            await lib.std.print(`${ok} Initializing session on TTY${ttyId}...\n`);
        const result = await lib.shell.exec("/bin/login.js", [], undefined, undefined, ttyId);
        if (result && result.pid) {
            terminals.set(ttyId, result.pid);
            await lib.std.log(`Login service spawned on TTY${ttyId} (PID ${result.pid})`, "init");
            // Kita nungguin di background (thread terpisah di Worker)
            this.monitorProcess(lib, ttyId, result.pid, spawnLogin);
        }
    } catch (e: any) {
        await lib.std.print(`Init: Error spawning login on TTY${ttyId}: ${e.message}\n`);
    }
};
for (let i = 2; i <= 6; i++) await spawnLogin(i);   // TTY2-6 di background
await spawnLogin(1);                                 // TTY1 foreground
```

`monitorProcess` waits on `waitpid`; if login dies, it waits 1 second then `respawn`s — preventing crash-loop spam:

```
private async monitorProcess(
    lib: UserLib, ttyId: number, pid: number, respawn: (tty: number) => Promise<void>,
) {
    const exitCode = await lib.shell.waitpid(pid);
    await lib.std.print(
        `Init: Process on TTY${ttyId} (PID ${pid}) exited with code ${exitCode}. Respawning...\n`,
    );
    await new Promise(r => setTimeout(r, 1000)); // delay anti crash-loop
    await respawn(ttyId);
}
```

init: eternal loop (PID 1 must never exit)

```
// Init process must never exit
while (true) {
    await new Promise(r => setTimeout(r, 10000));
}
```

rc.local: waiting for DOME ready (marker, not a fixed sleep)

```
const DOME_READY_MARKER = "/var/run/dome.ready";
const DOME_WAIT_MS = 10000;
const DOME_POLL_MS = 200;
let domeReady = false;
const waitStart = Date.now();
while (Date.now() - waitStart < DOME_WAIT_MS) {
    try {
        if (await lib.fs.stat(DOME_READY_MARKER)) { domeReady = true; break; }
    } catch (_) { /* marker belum ada */ }
    await new Promise(r => setTimeout(r, DOME_POLL_MS));
}
```

Asteracea is only started after the marker appears — otherwise, `CREATE_WINDOW` is rejected by the kernel ("GUI_REQ: DOME engine is not running") and the screen stays blank.

ensureDefaultAuth / ensureDefaultGroups (identity seeding)

At the end of `boot()`, the kernel ensures the identity files exist before userland runs:

```
// ensureDefaultAuth() — seed bila belum ada
const rootPasswd = "root:x:0:0:root:/root:/bin/tsh.ts\n";        // /etc/passwd
const rootShadow = "root:$2b$10$...KupG:19750:0:99999:7:::\n";   // /etc/shadow (bcrypt "root")

// ensureDefaultGroups() — group wajib
missing.push("users:x:100:");   // GID 100
missing.push("sudo:x:27:");     // GID 27 (gaya Ubuntu)
```

Exercises / Practice

1. Run `npm start` — note the boot log order (`[ OK ]` / `[ FAIL ]`) and compare it with the mermaid diagram above.
2. Kill the `init` process from inside the system (`kill 1` as root) — what happens to the host? (100ms keep-alive → exit code → reboot/halt).
3. Read `src/mirror/bin/init.ts` — where are `enforce setuid` and RSA identity called? How many login TTYs are spawned?
4. Read `src/mirror/etc/rc.local.ts` — match the 11 daemon steps above with the code. Compare with `rc.local.js` (only 3 daemons) — why are they different?
5. Read `src/kernel/Kernel.ts` — find the order of `initializeSubsystems()` and `rebuildVFSCache()`; explain the role of DME (Direct Memory Execution).

References

- `wiki/boot_sequence.md` — mermaid sequence diagram
- `wiki/course/00-overview.en.md` §3 — Boot Sequence
- `src/main.ts` — host entry point + keep-alive
- `src/kernel/Kernel.ts` — boot, initializeSubsystems, runInit, ensureDefaultAuth
- `src/mirror/bin/init.ts` — PID 1 (setuid, identity, rc.local, spawn login)
- `src/mirror/etc/rc.local.ts` — startup daemons
- `src/common/Config.ts` — reads `sysconfig.json`

Module 03 — done. Continue to Module 04 — Processes & Scheduler.

Module 04 **Processes & Scheduler** Boot & Kernel Runtime

RFC-TSIX-EDU-002 | Fourth module of the TSIX curriculum. Understand the process lifecycle in TSIX: PCB, spawn → run → block → exit, zombie, orphan reparent, daemonize, and reexec.

> One worker thread = one process. TSIX multitasking is **Node.js concurrency**, not preemption — the kernel never forcibly stops a process. What the kernel manages is **the lifecycle and signals**.

Learning Objectives

- [ ] State the main `PCB` fields and distinguish enum states vs transient states
- [ ] Explain the lifecycle: READY → RUNNING → BLOCKED → EXITED (including zombie, orphan, reexec)
- [ ] Explain `waitpid` + `reap` (zombie) and reparenting orphans to PID 1
- [ ] Explain `daemonize` (DETACH) and `REEXEC`
- [ ] Explain the signal model (SIGKILL, SIGINT, SIGTERM, SIGSTOP/CONT) and how it is delivered
- [ ] Trace the real flow: spawn → waitpid → exit → reap

Core Concepts

PCB (Process Control Block)

Every TSIX process is a single **worker thread**. The kernel (main thread) never runs application code — it only manages process metadata through the **PCB** (`interface PCB` in `src/kernel/Scheduler.ts`). One PCB = one process.

PCB Fields

Field	Type	Meaning
`pid`	`number`	Unique process ID; allocated from `nextPid++`
`ppid?`	`number`	Parent PID — forms the process tree
`name`	`string`	Process name (e.g. `"Shell"`, `"init"`)
`state`	`ProcessState`	`READY` / `RUNNING` / `BLOCKED` / `EXITED`
`pc`	`number`	Program counter — address of the next instruction
`owner`	`string`	Name of the owning user (e.g. `"root"`)
`uid`	`number`	User ID (effective)
`gid`	`number`	Group ID (effective)
`ruid`	`number`	Real user ID — who actually launched it
`groups`	`number[]`	Supplementary groups
`cwd`	`string`	Current working directory
`worker?`	`Worker`	Worker thread — the process body (exists after `spawnWorker`)
`fdTable`	`(FDEntry | null)[]`	File descriptor table; 0=stdin, 1=stdout, 2=stderr
`env`	`Record<string, string>`	Environment variables
`exitCode?`	`number`	Explicit exit code from the `EXIT` syscall
`ttyId?`	`number`	Virtual console (TTY) where the process runs
`uuid?`	`string`	Application identity — persistent across PIDs (for `SEND_MSG` by identity)

```
export interface PCB {
    pid: number;                // ID Unik Proses
    ppid?: number;              // Parent PID — buat process tree
    name: string;               // Nama Proses (misal: "Shell")
    state: ProcessState;        // Status Saat Ini
    pc: number;                 // Program Counter (Alamat instruksi berikutnya)

    owner: string;              // User yang menjalankan (misal: "root")
    uid: number;                // User ID (Effective)
    gid: number;                // Group ID (Effective)
    ruid: number;               // Real User ID (Siapa yang aslinya nge-jalanin)
    groups: number[];           // Supplementary Groups
    cwd: string;                // Current Working Directory (Posisi sekarang)
    worker?: Worker;            // Raga asli proses di thread terpisah

    // FD TABLE sekarang berisi object FDEntry yang menunjuk ke Driver nyata.
    fdTable: (FDEntry | null)[];

    // Environment Variables
    env: Record<string, string>;
    exitCode?: number; // Explicit exit code set by Syscall
    ttyId?: number;    // ID Virtual Console (TTY) tempat proses ini berjalan
    uuid?: string;     // Unique Application Identity (Persistent across PID changes)
}
```

> [!NOTE] **State** in TSIX has only the four enum values of `ProcessState`: `READY`, `RUNNING`, `BLOCKED`, `EXITED`. There is no `NEW` state — the PCB is already fully formed when `createProcess()` returns it. `REEXECING` is **not** part of the enum; it is a transient state set via `(pcb as any).state = "REEXECING"` to prevent the cleanup hook during `reexec()`.

Each `fdTable` entry is an `FDEntry`:

```
export interface FDEntry {
    device: IDevice;   // Driver nyata (FileSystemDevice, TTYDevice, dll)
    context: any;      // Konteks driver (path, offset, ...)
    flags?: string;    // "r" | "w" | ...
}
```

Lifecycle

```
stateDiagram-v2
    [*] --> READY : createProcess()\npid = nextPid++ → PCB
    READY --> RUNNING : spawnWorker()\nWorker Thread dibuat
    RUNNING --> BLOCKED : SIGSTOP(19) / menunggu event / IPC
    BLOCKED --> RUNNING : SIGCONT(18) / event selesai
    RUNNING --> EXITED : worker.on("exit")
    EXITED --> ZOMBIE : tidak ada waiter waitpid
    ZOMBIE --> [*] : waitpid() → reap()
    RUNNING --> REEXECING : reexec()\nterminate worker lama
    REEXECING --> READY : PCB sama, worker baru (spawnWorker)
    note right of EXITED
        orphan: semua child (ppid === pid)
        di-reparent ke init (PID 1)
    end note
```

> [!NOTE] `course-server.ts` does not render mermaid yet (it displays as a code block); text version: `READY → RUNNING → BLOCKED → EXITED`, where `EXITED` without a waiter becomes a **zombie** (it stays in the table until `waitpid` + `reap`).

Event	Mechanism
Spawn	`createProcess()` → `nextPid++` → PCB (`READY`) → `spawnWorker()` when `appName` / `appPath` exists → state `RUNNING`

Event	Mechanism
Exit	Syscall `EXIT(code)` → set `pcb.exitCode` → `cleanupProcess()` (close FDs, release ports) → `kill(pid)` → `worker "exit"` event → `EXITED`
Orphan	`worker.on("exit")` handler → all children (`ppid === pid`, not yet EXITED) are set `ppid = 1`
Zombie	Process `EXITED` without a waiter → stays in the table until `waitpid` + `reap()`
waitpid	Already `EXITED` → `reap()` + return exitCode; not yet → registered in `waitQueue`
daemonize	`DETACH` → FDs 0/1/2 redirected to `/dev/null`, resolve waiters, detach foreground TTY, clear `ttyId`
REEXEC	Set `(pcb as any).state = "REEXECING"` → terminate old worker → **same** PCB → new `spawnWorker()`
REPARENT (syscall)	`REPARENT { pid, newPpid }` → `childPcb.ppid = newPpid` (manual; verify the parent exists or is PID 1)

Walkthrough: spawn → waitpid → exit → reap

Follow the real flow of a `sleep 2` process launched from the shell (`tsh`):

1. **Spawn.** `tsh` (e.g. PID 3) calls the `EXEC` syscall → the kernel calls `createProcess("sleep", { appPath: "/bin/sleep.js", ppid: 3, fds: [...], ttyId: 1 })`. The PCB is created in state `READY`, then `spawnWorker()` creates a Worker Thread and sets state `RUNNING`. The process gets `pid = 42`.
2. **waitpid.** `tsh` calls `WAITPID(42)` → `waitpid(42)`. The process is not finished → `setForegroundProcess(42, 1)` (so Ctrl+C goes to `sleep`), then the resolver is registered in `waitQueue[42]`. `waitpid` returns a `Promise`.
3. **Exit.** `sleep` finishes → it sends the `EXIT(0)` syscall → the kernel sets `pcb.exitCode = 0` → `cleanupProcess(42)` closes all FDs and releases ports → `kill(42)` (default SIGKILL) → `worker.terminate()`.
4. **Reap.** Node triggers the `worker "exit"` event → state `EXITED` → `finalCode = 0` → `onProcessExitCallback` (global cleanup) → reparent orphans (none) → `waitQueue[42]` has a waiter → `resolve(0)` (the shell receives exit code 0) → `reap(42)` removes the PCB from the table. No zombie. The foreground TTY is returned to `null`/the shell.
5. **Zombie (anti-pattern).** If the shell does **not** call `waitpid`, the exit handler logs `became a ZOMBIE` and the PCB stays in the table. The PCB is removed later when `waitpid(42)` is called (reap).

Signals

Signals are sent through `scheduler.kill(pid, signal)` — the only gateway. Two syscalls use it:

- `KILL(pid)` → always **SIGKILL (9)**.
- `SIGNAL({ pid, sig })` → any signal according to `sig`.

Both perform the same checks: the process exists, **PID 1 is protected**, and permission (root or process owner).

Signal	Value	Effect
SIGKILL	9	Hard kill: `worker.terminate()` immediately, no event
SIGINT	2	Graceful: push `SIGINT` event, 100ms grace → terminate if unhandled. Default exit 130
SIGTERM	15	Graceful: push event, 300ms grace → terminate. Default exit 143. Used by SHUTDOWN
SIGSTOP	19	Soft: `pcb.state = BLOCKED` + event
SIGCONT	18	Soft: `pcb.state = RUNNING` + event
SIGSEGV	—	Sent by the kernel when a process violates GUI window ownership
SIGHUP (1), SIGUSR1 (10), SIGUSR2 (12), etc.	—	Generic event push `sendEvent(pid, "signal", name)`

 Ctrl+C → SIGINT to the foreground process of the active TTY *Source:* `wiki/diagram/Kernel-dan-Scheduler-3.mmd`

> [!IMPORTANT] **PID 1 (init) is protected** — both `KILL` and `SIGNAL` refuse to target PID 1, even for root. The only way to terminate the system is the `SHUTDOWN` / `REBOOT` syscall.

Staged `SHUTDOWN` (root only, `args` = 0 shutdown / 1 reboot):

1. `broadcastEvent("signal", "SIGTERM")` to all processes (except itself and PID 1).
2. Wait a grace of up to 5 s (100 ms polling) — all processes `EXITED` → success.
3. Remaining processes that did not respond → `kill(pid, 9)` (SIGKILL).
4. Flush network 1 s (so the last packets like `!exit!` get sent).
5. `kill(1, 9, exitCode)` → PID 1 is terminated; `main.ts` `keepAlive` reads the exit code (1 = reboot).

Source Code

File	Role
`src/kernel/Scheduler.ts`	PCB, lifecycle, waitpid, detach, signals, reexec
`src/kernel/Syscalls.ts`	KILL, WAITPID, EXEC, EXIT, DETACH, REEXEC syscalls
`src/common/IPCTypes.ts`	Signal event contract

Snippets (code level)

All snippets below are copied from `src/kernel/Scheduler.ts` and `src/kernel/Syscalls.ts` (code is truth).

createProcess (spawn)

```
public createProcess(name: string, options: SpawnOptions = {}): PCB | null {
    const pcb: PCB = {
        pid: this.nextPid++,
        name: name,
        state: ProcessState.READY,
        pc: 0,
        owner: options.owner || "root",
        uid: options.uid ?? 0,
        gid: options.gid ?? 0,
        ruid: options.ruid ?? (options.uid ?? 0),
        groups: options.groups ?? [],
        cwd: options.cwd ?? "/",
        fdTable: [],
        env: options.env ?? {},
        ttyId: options.ttyId,
        ppid: options.ppid ?? 0,
    };

    // Initialize FDs (0=stdin, 1=stdout, 2=stderr)
    if (options.fds && options.fds.length >= 3) {
        pcb.fdTable = [
            { device: options.fds[0], context: "", flags: "r" },
            { device: options.fds[1], context: "", flags: "w" },
            { device: options.fds[2], context: "", flags: "w" }
        ];
    }

    this.processes.push(pcb);

    // Worker baru hanya dibuat bila ada appName / appPath
    if (options.appName || options.appPath) {
        this.spawnWorker(pcb, options);
    }
```

```
    return pcb;
}
```

> [!NOTE] `createProcess` only creates the PCB (state `READY`) + (if an app exists) a Worker Thread. `pid = nextPid++`; standard FDs (0/1/2) are filled immediately when `options.fds` ≥ 3.

State transitions (spawnWorker, SIGSTOP/CONT, exit)

```
// spawnWorker() — worker dibuat → RUNNING
pcb.worker = new Worker(workerPath, { workerData, execArgv });
pcb.state = ProcessState.RUNNING;

// kill(pid, 19) — SIGSTOP → BLOCKED
pcb.state = ProcessState.BLOCKED;

// kill(pid, 18) — SIGCONT → RUNNING
pcb.state = ProcessState.RUNNING;

// Handler worker.on("exit") → EXITED (kecuali sedang REEXECING)
pcb.state = ProcessState.EXITED;
```

waitpid + reap (zombie)

```
public async waitpid(pid: number): Promise<number> {
    const pcb = this.getProcess(pid);

    // Jika proses sudah EXITED, langsung balikin 0 (atau simpan exit code di PCB)
    if (!pcb || pcb.state === ProcessState.EXITED) {
        const code = pcb?.exitCode ?? 0;
        // Jika proses sudah EXITED, kita reap sekalian biar gak jadi zombie
        this.reap(pid);
        return code;
    }

    // Proses yang ditunggu (foreground) berhak menerima interrupt Ctrl+C
    this.setForegroundProcess(pid, pcb.ttyId);

    return new Promise((resolve) => {
        if (!this.waitQueue.has(pid)) {
            this.waitQueue.set(pid, []);
        }
        this.waitQueue.get(pid)!.push(resolve);
    });
}
```

```
public reap(pid: number): void {
    const index = this.processes.findIndex(p => p.pid === pid);
    if (index !== -1) {
        const pcb = this.processes[index];
        if (pcb.state === ProcessState.EXITED) {
            this.logger.debug(`Reaping Process ${pcb.name} (PID ${pid}). Memory freed.`);

            // Remove from uuidMap if registered
            if (pcb.uuid && this.uuidMap.get(pcb.uuid) === pid) {
                this.uuidMap.delete(pcb.uuid);
                this.logger.debug(`UUID ${pcb.uuid} released from PID ${pid}`);
            }

            this.processes.splice(index, 1); // PCB dihapus dari tabel
        }
    }
}
```

> [!IMPORTANT] `reap()` only removes PCBs in the `EXITED` state. As long as `waitpid` has not been called, the PCB stays in the table — that is the **zombie**.

Auto-reparent orphans + notify waiters (the `worker.on("exit")` handler)

```
pcb.worker.on("exit", (code) => {
    // If it was a reexec, we don't trigger the exit logic yet
    if (pcb.state === (ProcessState as any).REEXECING) return;

    pcb.state = ProcessState.EXITED;

    // Use explicit exitCode if set (from EXIT syscall), otherwise use worker exit code
    const finalCode = pcb.exitCode !== undefined ? pcb.exitCode : (code || 0);

    // Global Cleanup Hook (reset terminal, close FDs, etc)
    if (this.onProcessExitCallback) {
        this.onProcessExitCallback(pcb.pid);
    }

    // Auto-reparent orphans: semua child dari proses yang exit → init (PID 1)
    const allProcs = Array.from(this.processes.values());
    const orphans = allProcs.filter((p: PCB) => p.ppid === pcb.pid && p.state !== ProcessState.EXITED);
    for (const orphan of orphans) {
        orphan.ppid = 1;
        this.logger.info(`[REPARENT] Auto-reparent orphan PID ${orphan.pid} (${orphan.name}) to init (PID 1)`);
    }

    // Notify Wait Queue
    if (this.waitQueue.has(pcb.pid)) {
        const waiters = this.waitQueue.get(pcb.pid)!;
        waiters.forEach(resolve => resolve(finalCode));
        this.waitQueue.delete(pcb.pid);
        // Jika ada yang nunggu (waitpid), kita bisa reap sekarang
        this.reap(pcb.pid);
    } else {
        this.logger.warn(`Process [${pcb.pid}] became a ZOMBIE (Needs waitpid)`);
    }

    // Jika proses foreground yang mati, kembalikan ke NULL
    if (this.getForegroundProcess(pcb.ttyId) === pcb.pid) {
        this.setForegroundProcess(null, pcb.ttyId);
    }
});
```

reexec (same PID, new body)

```
public async reexec(pid: number, appPath: string, args: string[]): Promise<boolean> {
    const pcb = this.getProcess(pid);
    if (!pcb || !pcb.worker) return false;

    // 1. Mark as Re-execing to prevent 'exit' hook cleanup
    (pcb as any).state = "REEXECING"; // Custom transient state

    // 2. Terminate current worker (Non-blocking to caller)
    pcb.worker.terminate().catch(() => { });
    pcb.worker = undefined;

    // 3. Update PCB info
    pcb.name = path.basename(appPath);
    pcb.state = ProcessState.READY;
    pcb.exitCode = undefined;

    // 4. Spawn new worker
    this.spawnWorker(pcb, { appPath, args });

    return true;
}
```

> [!TIP] `REEXECING` is not a member of the `ProcessState` enum — it is a transient state set via `(pcb as any).state`. Its purpose: the `worker.on("exit")` handler skips the cleanup logic because of the `return` on the first line.

detach (daemonize)

```
public async detach(pid: number): Promise<boolean> {
    const pcb = this.getProcess(pid);
    if (!pcb) return false;

    // 1. Resolve Wait Queue (si Shell jadi gak nungguin lagi)
    if (this.waitQueue.has(pid)) {
        const waiters = this.waitQueue.get(pid)!;
        waiters.forEach(resolve => resolve(0)); // Kasih status sukses (0)
        this.waitQueue.delete(pid);
    }

    // 2. Lepas dari Foreground TTY (biar Ctrl+C gak masuk sini lagi)
    if (pcb.ttyId && this.ttyForegroundPids.get(pcb.ttyId) === pid) {
        this.ttyForegroundPids.delete(pcb.ttyId);
    }

    // 3. Clear TTY Association (Crucial for ps display)
    pcb.ttyId = undefined;

    return true;
}
```

> [!NOTE] At the syscall level, `DETACH` also redirects FDs 0/1/2 to `/dev/null` before calling `scheduler.detach()` — so the daemon does not leak logs to the original TTY.

Signal delivery (scheduler.kill)

```
public async kill(pid: number, signal: number = 9, exitCode: number = 0): Promise<boolean> {
    const pcb = this.getProcess(pid);
    if (!pcb || !pcb.worker) return false;

    if (signal === 9) { // SIGKILL (Hard Kill)
        if (pcb.worker) {
            await pcb.worker.terminate().catch(() => { });
        }
        return true;
    }

    if (signal === 2) { // SIGINT (Ctrl+C)
        // Send event first to allow graceful handling
        const eventSent = this.sendEvent(pid, "signal", "SIGINT");
        // If no listener handled it, terminate after a brief delay
        // This preserves default Unix behavior: unhandled SIGINT = process exit
        setTimeout(async () => {
            const stillRunning = this.getProcess(pid);
            if (stillRunning && stillRunning.state !== ProcessState.EXITED && stillRunning.worker) {
                this.logger.debug(`PID ${pid} did not handle SIGINT, terminating...`);
                stillRunning.worker.terminate().catch(() => { });
            }
        }, 100); // 100ms grace period for handler to respond
        return eventSent;
    }

    if (signal === 19) { // SIGSTOP (Pause)
        pcb.state = ProcessState.BLOCKED;
        this.logger.info(`Process [${pid}] ${pcb.name} PAUSED (SIGSTOP)`);
        return this.sendEvent(pid, "signal", "SIGSTOP");
    }

    if (signal === 18) { // SIGCONT (Resume)
```

```
        pcb.state = ProcessState.RUNNING;
        this.logger.info(`Process [${pid}] ${pcb.name} RESUMED (SIGCONT)`);
        return this.sendEvent(pid, "signal", "SIGCONT");
    }

    if (signal === 15) { // SIGTERM (15)
        const eventSent = this.sendEvent(pid, "signal", "SIGTERM");
        setTimeout(async () => {
            const stillRunning = this.getProcess(pid);
            if (stillRunning && stillRunning.state !== ProcessState.EXITED && stillRunning.worker) {
                this.logger.debug(`PID ${pid} did not handle SIGTERM, terminating...`);
                stillRunning.worker.terminate().catch(() => { });
            }
        }, 300);
        return eventSent;
    }

    // Generic Signal Handling (HUP, USR1, etc)
    const sigNames: Record<number, string> = { 1: "SIGHUP", 10: "SIGUSR1", 12: "SIGUSR2" };
    const sigName = sigNames[signal] || `SIG${signal}`;
    return this.sendEvent(pid, "signal", sigName);
}
```

Syscall gateway: KILL / SIGNAL / WAITPID

```
case SyscallCode.KILL: {
    const targetPid = args as number;
    const targetPcb = this.scheduler.getProcess(targetPid);
    if (!targetPcb)
        throw new Error(`kill: No such process (PID ${targetPid})`);
    // PID 1 (init) cannot be killed — not even by root
    if (targetPid === 1) {
        throw new Error(`kill: PID 1 (init) is protected — cannot be killed directly. Use 'shutdown' or 'reboot' instead.`);
    }
    // Permission check: root can kill anyone, non-root can only kill own processes
    if (!this.isRoot(pcb) && targetPcb.uid !== pcb.uid) {
        throw new Error(`kill: Permission denied — you do not own process ${targetPid} (owned by UID ${targetPcb.uid})`);
    }
    return await this.scheduler.kill(targetPid, 9); // SIGKILL
}

case SyscallCode.SIGNAL: {
    const { pid: targetPid, sig } = args as { pid: number; sig: number };
    // cek sama: proses ada → PID 1 diproteksi → permission →
    return await this.scheduler.kill(targetPid, sig);
}

case SyscallCode.WAITPID: {
    const targetPid = args as number;
    return await this.scheduler.waitpid(targetPid);
}
```

> [!WARNING] The `KILL` syscall always maps to **SIGKILL (9)** — no other signal. For other signals (SIGTERM, SIGINT, SIGSTOP, ...) use the `SIGNAL { pid, sig }` syscall. Both refuse PID 1 and check process ownership.

Exercises / Practice

1. Run `ps` in the shell — identify the PID, PPID, state, and TTY of each process.
2. Run `sleep 30 &` then `ps` — find the process running in the background (state and ttyId empty).
3. Run an app, note its PID, then `kill <pid>` (SIGKILL) and `kill -15 <pid>` (SIGTERM) — observe the difference in behavior in the kernel logs.
4. Launch a foreground process (without `&`), press **Ctrl+C** — observe `SIGINT` being sent to the foreground process via `sendInterruptSignal()`.

5. Trace this module's walkthrough (spawn → waitpid → exit → reap) directly in the `Scheduler.ts` code: `createProcess()`, `waitpid()`, and the `worker.on("exit")` handler.
6. Read `src/kernel/Syscalls.ts` — compare the `KILL` case (always SIGKILL) vs `SIGNAL` (free sig) and the PID 1 protection.

References

- `wiki/Kernel-dan-Scheduler.md` — process & scheduler details
- `wiki/course/00-overview.en.md` §4.1, §4.3
- `src/kernel/Scheduler.ts` — PCB, lifecycle, waitpid, detach, signals, reexec
- `src/kernel/Syscalls.ts` — KILL, SIGNAL, WAITPID, EXIT, DETACH, REEXEC, SHUTDOWN cases
- `src/kernel/Scheduler.test.ts` — behavior confirmation (A2.19–A2.21 zombie/waitpid, A2.26 detach, reexec)

Module 04 — done. Continue to Module 05 — Syscall & IPC.

Module 05 **Syscalls & IPC** Boot & Kernel Runtime

RFC-TSIX-EDU-002 | Fifth module of the TSIX curriculum. Understand the TSIX mini ABI: how applications call the kernel via syscall, request-response correlation, and push events from the worker.

> TSIX syscalls are **purely asynchronous RPC**. No order is guaranteed — every request carries a `requestId` (UUID) and is answered with the same `requestId`. This differs from synchronous traps in Linux.

Learning Objectives

- [ ] Explain the complete flow of one syscall (from app to return)
- [ ] Explain the role of `requestId` + `responseMap`
- [ ] Distinguish request-response vs push events
- [ ] Name several important syscall codes (1–73)
- [ ] Explain the role of `validateArgs`

Core Concepts

TSIX uses asynchronous *request-response* IPC — not synchronous traps like Linux. The application and the kernel run in separate *threads* (Worker vs main thread). The only bridge is `postMessage` in both directions.

Concept	Explanation
`SyscallRequest`	Packet app → kernel: `{ requestId, pid, code, args }`
`SyscallResponse`	Packet kernel → app: `{ requestId, success, data?, error? }`
`requestId` (UUID)	Correlates request ↔ response. Created in `UserLib.dispatch`, reused by the kernel when replying.
`responseMap`	Map on the app side: `requestId` → callback resolve/reject. Replies are matched through this key.
Push event	Kernel → app notification *without* requestId: `{ type, data }` (signal, ipc_message, gui_request, db_request, resize).
`validateArgs`	Argument contract in the kernel: checks which syscalls must have `args`, plus type checks per syscall.

> [!IMPORTANT] Because RPC is purely asynchronous, **reply order is not guaranteed**. Two consecutive syscalls (A then B) can be answered with B first. Correlation always goes through `requestId`, never through order.

Flow / How It Works

Full syscall flow: `PRINT`

Simplest example: the app calls `std.print("hello")`. Here is the complete path (code is truth):

```
// src/mirror/lib/UserLib.ts — StdLib
public async print(text: string) {
  return await this.dispatch(SyscallCode.PRINT, text);
}
```

Step by step:

1. `std.print("hello")` calls `UserLib.dispatch(PRINT, "hello")` on the worker side.
2. `dispatch()` creates `requestId = uuidv4()`, stores the callback in `responseMap`, then `parentPort.postMessage({ requestId, pid, code: PRINT, args: "hello" })`.

3. The kernel receives the message in `Scheduler.spawnWorker()` → handler `pcb.worker.on("message", ...)`.
4. The handler calls `syscallHandler(request)` — wired in `Kernel.initializeSubsystems()` via `scheduler.setSyscallHandler(...)`.
5. `SyscallDispatcher.handleRequest(req)` → `validateArgs(PRINT, args)` → `dispatch(pid, PRINT, args)`.
6. The `PRINT` case takes FD 1 from `pcb.fdTable`, then `entry.device.write("hello")` → output to screen.
7. The result (`0`) returns to the Scheduler handler; the Scheduler replies `postMessage({ requestId, success: true, data: 0 })`.
8. In the worker, `parentPort.on("message")` finds `requestId` in `responseMap`, calls the callback → Promise `resolve(0)` → `await print` finishes.

```
sequenceDiagram
    participant App as Aplikasi (Worker)
    participant Lib as UserLib (dispatch)
    participant Sch as Scheduler (worker.on message)
    participant Disp as SyscallDispatcher
    participant Dev as Device (fdTable[1])

    App->>Lib: std.print("hello")
    Lib->>Lib: requestId = uuidv4(); responseMap.set(requestId, cb)
    Lib->>Sch: postMessage({ requestId, pid, code: PRINT, args })
    Sch->>Disp: await syscallHandler(request)
    Disp->>Disp: validateArgs(PRINT, args)
    Disp->>Dev: dispatch(pid, PRINT, args) → fdTable[1].device.write()
    Dev-->>Disp: return 0
    Disp-->>Sch: return 0
    Sch-->>Lib: postMessage({ requestId, success: true, data: 0 })
    Lib->>Lib: cocokkan requestId di responseMap → resolve(0)
    Lib-->>App: await print selesai
```

Request-response correlation via `requestId` (UUID) + `responseMap` — purely asynchronous RPC. Many syscalls can be in-flight at once.

Push events (kernel → worker, without requestId)

Unlike request-response, push events have no `requestId`. The kernel sends `{ type, data }` whenever needed:

```
{ type, data }
```

Types actually used in the code (`Scheduler.ts`, `Syscalls.ts`, `Kernel.ts`):

type	Usage in code	Purpose
`signal`	`sendEvent(pid, "signal", "SIGINT")` etc.	Signals to processes (SIGINT, SIGTERM, SIGSTOP, SIGCONT, SIGWINCH, SIGSEGV...)
`ipc_message`	`SEND_MSG` between apps; forward `NET_SNIFF` packets	Horizontal IPC / sniffer
`gui_request`	Window cleanup & forward to GUI daemon	GUI daemon (PixelSpace)
`db_request`	`forwardDbRequest` → DB service daemon	Alternative database transport
`resize`	`broadcastEvent("resize", { lines, columns })` when host terminal changes	All active processes

> [!NOTE] On the worker side, `UserLib.parentPort.on("message")` distinguishes three things: (1) syscall reply — has a `requestId` matching in `responseMap`; (2) event — has `type` and `type !== "signal"`; (3) signal — `type === "signal"`, with default action `SIGINT → exit(130)` and `SIGTERM → exit(143)` when there is no listener.

Main Syscall List (mini ABI)

Kode	Nama	Kode	Nama
1	`PRINT`	29	`PIPE`
2	`MKDIR`	30–34	`SOCKET/BIND/SENDTO/RECVFROM/NETSTAT`
3	`LS`	35	`UPTIME`
4	`EXIT`	36	`DETACH`
5–8	`OPEN/READ/WRITE/CLOSE`	37	`UNAME`
9	`SCREEN_INFO`	38	`SETGROUPS`
10–12	`PS/KILL/EXEC`	40–41	`GET_SYSPATH/GET_USAGE`
13–14	`CHDIR/GETCWD`	50	`SHUTDOWN`
15–16	`CHMOD/CHOWN`	53–55	`SYNC_TO_HOST/REEXEC/SYNC_FROM_HOST`
17–19	`WHOAMI/GETENV/SETENV`	56–58	`MOUNT/UMOUNT/GET_MOUNTS`
20–23	`STAT/UNLINK/RMDIR/IOCTL`	60	`SET_IDENTITY`
24–28	`SEND_MSG/WAITPID/SIGNAL/SETUID/SETGID`	61–66	`GUI_REQ/GET_PPID/READ_CHUNK/WRITE_CHUNK/GET_SIZE/REPARENT`
—	—	67–71	`DB_CONNECT/DB_QUERY/DB_DISCONNECT/DB_SERVICE_REGISTER/DB_SERVICE_REPLY`
—	—	72–73	`NET_SNIFFER_REGISTER/NET_SNIFFER_UNREGISTER`

> `SyscallCode` is an enum in `src/common/SyscallCode.ts` — the ABI contract. Do not change existing numbers (breaking change).

Some key codes (number → name → purpose):

Number	Name	Purpose
1	`PRINT`	Prints text to the screen (via FD 1)
14	`GETCWD`	Gets the process working directory
24	`SEND_MSG`	Horizontal IPC (app-to-app)
25	`WAITPID`	Waits for a process to finish by PID
26	`SIGNAL`	Sends a signal to a process
61	`GUI_REQ`	PixelSpace Display Protocol (RFC-TSIX-002)
67	`DB_CONNECT`	Opens an external database connection

Source Code

File	Contents	Relevance
`src/common/SyscallCode.ts`	ABI enum 1–73	Syscall numbers
`src/common/IPCTypes.ts`	Packet shapes `SyscallRequest`, `SyscallResponse`, `IPCEvent`	Packet contract

File	Contents	Relevance
`src/kernel/Syscalls.ts`	`SyscallDispatcher`: `handleRequest`, `validateArgs`, `dispatch`	Dispatcher
`src/kernel/Scheduler.ts`	`spawnWorker` + `worker.on("message")` (reply) + `sendEvent`	Bridge & reply
`src/kernel/Kernel.ts`	Wiring `setSyscallHandler(...)`	Wiring
`src/mirror/lib/UserLib.ts`	`dispatch()`, `responseMap`, `parentPort.on("message")`, `onEvent`	Worker side

Snippets (code level)

> All snippets below are copied from the source. "Code is truth."

1. UserLib dispatch (worker side) — `src/mirror/lib/UserLib.ts`

```
  private async dispatch(code: SyscallCode, args: any): Promise<any> {
    return new Promise((resolve, reject) => {
      const requestId = uuidv4();
      const request: SyscallRequest = { requestId, pid: this.pid, code, args };

      this.responseMap.set(requestId, (response: SyscallResponse) => {
        if (response.success) {
          resolve(response.data);
        } else {
          reject(new Error(response.error || "Syscall Failed"));
        }
      });

      if (parentPort) {
        parentPort.postMessage(request);
      } else {
        reject(new Error("No parentPort found! Are you running in a Worker?"));
      }
    });
  }
```

> `responseMap` has type `Map<string, (res: SyscallResponse) => void>` — its values are **callbacks**, not `{ resolve, reject }` objects. The Promise is resolved/rejected inside the callback when the reply arrives.

2. Receiver side (worker) — `parentPort.on("message")`

```
parentPort.on("message", (msg: any) => {
  // 1. Balasan Syscall
  if (msg.requestId && this.responseMap.has(msg.requestId)) {
    const resolve = this.responseMap.get(msg.requestId)!;
    resolve(msg as SyscallResponse);
    this.responseMap.delete(msg.requestId);
  }
  // 2. Event (Push Notification) — selain signal
  else if (
    msg.type &&
    msg.type !== "signal" &&
    this.eventListeners.has(msg.type)
  ) {
    const callbacks = this.eventListeners.get(msg.type)!;
    callbacks.forEach((cb) => cb(msg.data));
  }
  // 3. Signal (SIGINT/SIGTERM) dengan default action
  else if (msg.type === "signal") {
    const sig = msg.data;
    // ... default: exit(130) untuk SIGINT, exit(143) untuk SIGTERM
```

```
  }
});
```

3. Kernel handleRequest — `src/kernel/Syscalls.ts`

```
public async handleRequest(req: SyscallRequest): Promise<any> {
    const silentCodes = [
      SyscallCode.READ,
      SyscallCode.PS,
      SyscallCode.GETCWD,
      SyscallCode.WHOAMI,
    ];
    if (!silentCodes.includes(req.code)) {
      this.logger.debug(
        `[IPC] PID ${req.pid} Request: ${SyscallCode[req.code]} (${req.requestId})`,
      );
    }

    try {
      this.validateArgs(req.code, req.args);
      return await this.dispatch(req.pid, req.code, req.args);
    } catch (e: any) {
      this.logger.error(
        `Syscall Error [${SyscallCode[req.code]}]: ${e.message}`,
      );
      throw e;
    }
  }
```

> ⚠️ **Important correction**: `handleRequest` **does not reply by itself**. It only validates, executes, then returns the result (or throws an error). The `postMessage` reply is sent by the **Scheduler** in the `worker.on("message")` handler (see snippet 4).

4. Reply sent by Scheduler — `src/kernel/Scheduler.ts` (`spawnWorker`)

```
pcb.worker.on("message", async (request: SyscallRequest) => {
    if (this.syscallHandler) {
        try {
            const result = await this.syscallHandler(request);
            if (pcb.worker) {
                pcb.worker.postMessage({
                    requestId: request.requestId,
                    success: true,
                    data: result
                });
            }
        } catch (error: any) {
            if (pcb.worker) {
                pcb.worker.postMessage({
                    requestId: request.requestId,
                    success: false,
                    error: error.message
                });
            }
        }
    }
});
```

5. Real syscall handler example — `src/kernel/Syscalls.ts`

```
      case SyscallCode.PRINT: {
        const entry = pcb.fdTable[1];
        if (!entry) return -1;
        entry.device.write(args as string);
        return 0;
```

```
    }

    case SyscallCode.GETCWD:
      return pcb.cwd;
```

> `PRINT` does not write directly to the screen — it writes through **FD 1** (`stdout`) in `fdTable`. This embodies "everything is a file": output is just a device attached to fdTable. `GETCWD`, on the other hand, simply returns `pcb.cwd` (process state in the PCB).

6. Push events — `sendEvent` in `src/kernel/Scheduler.ts`

```
public sendEvent(pid: number, type: string, data: any): boolean {
    const pcb = this.getProcess(pid);
    if (pcb && pcb.worker && pcb.state !== ProcessState.EXITED) {
        pcb.worker.postMessage({ type, data });
        return true;
    }
    return false;
}
```

Example usage (forwarding sniffer packets in `Syscalls.ts`):

```
this.scheduler.sendEvent(pid, "ipc_message", { data: sniff });
```

Exercises / Practice

1. Read `src/common/SyscallCode.ts` — note all syscall numbers (1–73). Pay attention to the number gaps (39, 42–49, 51–52, 59) intentionally reserved.
2. Read `src/common/IPCTypes.ts` — recognize the shapes of `SyscallRequest`, `SyscallResponse`, `IPCEvent`.
3. Add a *log* in `handleRequest` for the `PRINT` syscall — then run `echo hello` in the shell. Observe the flow (same requestId in request & response).
4. Explain why `requestId` is needed — what happens without correlation?
5. Send two syscalls at once (e.g. `READ` large file + `PRINT`) and observe the reply order — prove that order is not guaranteed.
6. From the worker side, check the contents of `responseMap` right after `dispatch()` is called (before the reply arrives) — prove the values are callbacks.

References

- `src/common/SyscallCode.ts` — ABI enum (source of truth for syscall numbers)
- `src/common/IPCTypes.ts` — IPC packet shapes
- `src/kernel/Syscalls.ts` — `handleRequest`, `validateArgs`, `dispatch`
- `src/kernel/Scheduler.ts` — `worker.on("message")` (reply) + `sendEvent` (push)
- `src/kernel/Kernel.ts` — wiring `setSyscallHandler`
- `src/mirror/lib/UserLib.ts` — `dispatch()`, `responseMap`, `onEvent`
- `wiki/identity_guid_ipc_walkthrough.md` — IPC & identity walkthrough
- `wiki/course/00-overview.en.md` §4.2

Module 05 — complete. Continue to Module 06 — Permission & Security.

Module 06 **Permissions & Security** Boot & Kernel Runtime

RFC-TSIX-EDU-002 | The sixth module of the TSIX curriculum. Understand the rwx permission model, the setuid bit, root bypass, and known weaknesses in the TSIX permission system.

> PermissionManager is the kernel's "security guard". Every file access goes through layered checks: root bypass → owner → group → others. Plus the setuid bit for privileged binaries like `login` and `sudo`.

Learning Objectives

- [ ] Explain the permission check order (root → owner → group → others)
- [ ] Explain the permission bits `r=4, w=2, x=1`
- [ ] Read the permission matrix and example modes (`600`, `644`, `755`, `4755`) with their decimal values
- [ ] Explain the setuid bit (`0o4000` = 2048) and its handling during `EXEC`
- [ ] Explain why `parseMode("4755")` is 2541 (decimal)
- [ ] Explain the `CHMOD` rule (owner or root) vs `CHOWN` (root only)
- [ ] Explain the app-name-based privilege weakness and its mitigation

Core Concepts

Layered rwx checks

```
check(pcb, node, requested):
  1. root (uid 0)        → true (God Mode)
  2. owner (uid == node.uid) → bit owner   ((mode >> 6) & 0x7)
  3. group (gid match)       → bit group   ((mode >> 3) & 0x7)
  4. others                  → bit others  (mode & 0x7)
```

 PermissionManager check flow (root → owner → group → others) *Source: `wiki/diagram/Keamanan-dan-Sandboxing-3.mmd`*

rwx permission matrix

One mode = **3 octal digits**, one for each class: owner, group, and others. `check()` shifts the bits according to the class:

Class	Calculation in `check()`	Bits	Example `644` (decimal 420)
owner (user)	`(mode >> 6) & 0x7`	r=4, w=2, x=1	digit `6` → `rw-`
group	`(mode >> 3) & 0x7`	r=4, w=2, x=1	digit `4` → `r--`
others	`mode & 0x7`	r=4, w=2, x=1	digit `4` → `r--`

Encode permission

Bit	Value	Meaning
r (read)	4	Read content
w (write)	2	Write / modify
x (execute)	1	Execute

Bit combination per octal digit (used in the mode table below):

Digit	Binary	rwx	Meaning
`0`	000	`---`	no access
`1`	001	`--x`	execute
`2`	010	`-w-`	write
`3`	011	`-wx`	write + execute
`4`	100	`r--`	read
`5`	101	`r-x`	read + execute
`6`	110	`rw-`	read + write
`7`	111	`rwx`	read + write + execute

Common modes & decimal values

Modes are stored as **decimal in SQLite** (not an octal string). `parseMode("4755")` = `parseInt("4755", 8)` = **2541** (decimal) — this is the value `init.ts` uses when it calls `chmod("/bin/passwd.js", 2541)`.

Mode	Octal	Decimal	owner	group	others	Example usage
`600`	`0o600`	384	`rw-`	`---`	`---`	private files: `/etc/shadow`, SSH keys
`644`	`0o644`	420	`rw-`	`r--`	`r--`	plain text files (default mode of `touch` in the `OPEN` syscall)
`755`	`0o755`	493	`rwx`	`r-x`	`r-x`	directories & binaries (default mode of `mkdir`)
`4755`	`0o4755`	2541	`rws`	`r-x`	`r-x`	setuid binaries: `/bin/passwd.js`, `/bin/sudo.js`

> The `s` in the owner position means the **setuid bit is active** (`0o4000` = 2048), not a regular `x`. The decimal values are confirmed by test `A5.10` (`644` → 420, `755` → 493) and `A5.13` (`4755 & 0o4000`).

The SetUID bit (`0o4000`)

SetUID = execute as the **file owner**, not the caller. Example:

- `/bin/passwd.js` → `chmod 2541` (0o4755): a normal user runs passwd, and the process runs as root so it can modify `/etc/shadow`.
- `/bin/sudo.js` → same.

Handling during `EXEC` (in `Syscalls.ts`): the `0o4000` bit = **2048** (decimal). When executing a binary, the kernel checks `node.mode & 2048`. If active, `targetUid = node.uid` and `targetGid = node.gid`; the new process is born with the file owner's uid/gid, while `ruid` (real UID) stays with the caller. See the snippet below.

> Note: the runtime executes the `.js` sidecar, so SetUID must be set on `/bin/*.js`, not the source `.ts` files. This is what `init.ts` does: `lib.fs.chmod("/bin/passwd.js", 2541)` and `lib.fs.chmod("/bin/sudo.js", 2541)`. `rc.local.ts` also sets `chmod("/bin/login.js", 0o4755)`.

The POSIX order during login: `setgroups → setgid → setuid` (this order is required so the group is correct). Visible in `login.ts`: `setgroups(supplementaryGids) → setgid(gid) → setuid(uid)`.

Snippet (code level)

PermissionManager.check()

```
public check(pcb: PCB, node: any, requested: Permission): boolean {
    if (pcb.uid === 0) return true;            // 1. root bypass
```

```
    const mode = node.mode;                    // decimal di SQLite

    if (pcb.uid === node.uid) {                // 2. owner
        const userMode = (mode >> 6) & 0x7;
        if ((userMode & requested) === requested) return true;
    }

    if (pcb.gid === node.gid || (pcb.groups && pcb.groups.includes(node.gid))) {
        const groupMode = (mode >> 3) & 0x7;   // 3. group
        if ((groupMode & requested) === requested) return true;
    }

    const otherMode = mode & 0x7;              // 4. others
    if ((otherMode & requested) === requested) return true;

    return false;
}

public static parseMode(octal: string | number): number {
    return parseInt(octal.toString(), 8);      // "4755" → 2541
}
```

EXEC — permission enforcement & setuid (Syscalls.ts)

```
// 1. Cek x bit dulu — tanpa izin eksekusi, langsung ditolak
if (!this.satpam.check(pcb, node, Permission.EXECUTE)) {
  throw new Error(`Permission Denied: Cannot execute ${absoluteExecPath}`);
}

let targetUid = pcb.uid;
let targetGid = pcb.gid;
let targetOwner = pcb.owner;

// 2. SETUID BIT SUPPORT (0o4000 = 2048)
if (node && node.mode & 2048) {
  targetUid = node.uid;        // eksekusi sebagai pemilik file
  targetGid = node.gid;
  if (targetUid === 0) targetOwner = "root";
  this.logger.info(
    `SetUID Execution detected for ${absoluteExecPath}: Running as UID ${targetUid}`,
  );
}

// 3. Proses baru lahir dengan uid/gid target; ruid (real UID) dipertahankan
const newPcb = this.scheduler.createProcess(binaryName, {
  // ...
  uid: targetUid,
  gid: targetGid,
  ruid: pcb.ruid, // Preserve Real UID
  owner: targetOwner,
  // ...
});
```

> This path is the security key for `/bin/login.js`, `/bin/passwd.js`, and `/bin/sudo.js`: a normal user cannot setuid directly, but executing a setuid-root binary makes the process run as `uid 0`.

OPEN — read vs write (Syscalls.ts)

```
const requiredPerm =
  flags.includes("w") || flags.includes("+")
    ? Permission.WRITE
    : Permission.READ;

const node = vfs.stat(relativePath);

if (node) {
```

```
  // File sudah ada → cek izin file itu sendiri
  if (!this.satpam.check(pcb, node, requiredPerm)) {
    throw new Error(
      `Permission Denied: Cannot open ${absoluteOpenPath} for ${Permission[requiredPerm]}`,
    );
  }
} else if (requiredPerm === Permission.WRITE) {
  // File belum ada → cek WRITE ke direktori parent, lalu buat mode 420 (644)
  // ...
  vfs.touch(relativePath, "", pcb.uid, pcb.gid, 420); // 644 equivalent
}
```

CHMOD — change mode (Syscalls.ts)

```
case SyscallCode.CHMOD: {
  const { path: argPath, mode } = args as { path: string; mode: number };
  const absolutePath = PathResolver.resolve(pcb.cwd, argPath);

  if (absolutePath.startsWith("/dev/")) {
    if (!this.isRoot(pcb)) return false;   // chmod /dev/* → root only
    const devName = absolutePath.replace("/dev/", "");
    const device = this.kernel.devices[devName];
    if (!device) return false;
    device.mode = mode;
    return true;
  }

  const { vfs, relativePath } = this.mountManager.resolve(absolutePath);
  const node = vfs.stat(relativePath);
  if (!node) return false;
  if (pcb.uid !== 0 && pcb.uid !== node.uid) return false; // root ATAU owner
  return vfs.chmod(relativePath, mode);
}
```

> `CHMOD` rule: a regular file may be changed by **its owner or root**. Verified by test `A5.11`.

CHOWN — change owner/group (Syscalls.ts)

```
case SyscallCode.CHOWN: {
  const {
    path: argPath,
    uid: targetUid,
    gid: targetGid,
  } = args as { path: string; uid: number; gid: number };
  const absolutePath = PathResolver.resolve(pcb.cwd, argPath);

  if (absolutePath.startsWith("/dev/")) {
    if (!this.isRoot(pcb)) return false;   // chown /dev/* → root only
    // ...
  }

  if (!this.isRoot(pcb)) return false;     // chown file → ROOT SAJA
  const { vfs, relativePath } = this.mountManager.resolve(absolutePath);
  return vfs.chown(relativePath, targetUid, targetGid);
}
```

> `CHOWN` rule: **only root** may change a file's owner (test `A5.15`). Unlike `CHMOD`, which the owner may also do.

Known Weaknesses

> [!WARNING] **App-name-based privilege (fragile).** In `WorkerEntry.ts`, `restrictHostAPI(appName)` marks an app as "privileged" if the **substring** of its name (after `toLowerCase()`) contains one of:
>
> ```
> const isPrivileged = appName.toLowerCase().includes("server") ||
>     appName.toLowerCase().includes("daemon") ||
>     appName.toLowerCase().includes("dome") ||
>     appName.toLowerCase().includes("tbuild") ||
>     appName.toLowerCase().includes("vfs") ||
>     appName.toLowerCase().includes("mysqld");
> ```
>
> A privileged app gets an **allow-list of host modules**:
>
> ```
> const allowedModules = ["http", "ws", "path", "fs", "url", "esbuild", "crypto", "os", "bcryptjs", "mysql2", "mysql2/promise"];
> ```

Exploitation scenario (example):

1. An attacker uploads an app named `evil-daemon` (the name contains `daemon`).
2. When executed, `isPrivileged = true` → `global.require = privilegedRequire`.
3. The app can now `require("fs")`, `require("crypto")`, `require("http")`, etc. — **direct host** Node.js modules, not TSIX syscalls.
4. With `fs`, the attacker can read files outside the TSIX VFS sandbox (e.g., the host Linux `/etc/shadow`) → **sandbox escape**.

> [!IMPORTANT] **Mitigation.**
>
> - **Long-term (architectural):** replace the substring heuristic with **capability-based** checks — an explicit list of `appName → capabilities` pairs (or a per-app manifest).
> - **Now (defense-in-depth):** even if an app is "privileged" inside the sandbox, the kernel still has `PermissionManager` + `validateArgs` + root-only `SETUID`. The app name only opens the host module door — **not** full kernel access; every syscall is still checked per-invocation on the kernel side.

Exercises / Practice

1. Run `ls -l /bin/passwd.js` — observe the setuid bit (`s`) in the owner mode (mode `rwsr-xr-x`).
2. Run `stat /bin/passwd.js` — check the decimal mode value (should be `2541` = `0o4755`).
3. Run `chmod 755 /bin/passwd.js` then `ls -l` again — the `s` bit disappears; `init` will set it again (`init.ts`).
4. As a non-root user, try to read `/etc/shadow` — observe the permission error.
5. As a non-root user, run `chown root /tmp/foo` — observe the error (only root may chown); then `chmod 600 /tmp/foo` — must succeed because the owner may chmod.
6. Read `src/kernel/PermissionManager.ts` and `src/kernel/Syscalls.ts` — find all call sites of `satpam.check()`.
7. Read `src/userland/WorkerEntry.ts` → the `restrictHostAPI` function — test: run an app named `evil-daemon` that calls `require("fs")`, and compare it with a normal app named `hitung`.

References

- `wiki/Keamanan-dan-Sandboxing.md` — complete security model
- `wiki/course/00-overview.en.md` §4.3 (ring model & privilege boundaries)
- `src/kernel/PermissionManager.ts` — implementation of `check()` & `parseMode()`
- `src/kernel/PermissionManager.test.ts` — tests A5.1–A5.30 (rwx, chmod, chown, umask, setuid, sudo)
- `src/kernel/Syscalls.ts` — OPEN/EXEC/CHMOD/CHOWN permission enforcement + setuid in EXEC
- `src/userland/WorkerEntry.ts` — the `restrictHostAPI` sandbox (name-based privilege)
- `src/mirror/bin/init.ts`, `src/mirror/bin/rc.local.ts` — setuid bit setup
- `src/mirror/bin/login.ts`, `src/mirror/bin/sudo.ts` — the setgroups/setgid/setuid flow

Module 06 — complete. Continue to Module 07 — Mount & Path Resolution.

Module 07 **Mount & Path Resolution** Boot & Kernel Runtime

RFC-TSIX-EDU-002 | Seventh module of the TSIX curriculum. Understand how VFS paths are routed to the correct filesystem backend — by the "longest prefix wins" principle.

> MountManager is the "path router". When an app opens `/tmp/foo`, the kernel must know that it belongs to RamFS, not BKFS. The rule is simple: **the longest matching prefix wins.**

Learning Objectives

- [ ] Explain the `MountPoint` structure
- [ ] Explain the "longest prefix wins" principle
- [ ] Explain the contents of `fstab.json` (default mounts)
- [ ] Distinguish resolution via MountManager vs device `/dev/*`
- [ ] Explain the role of `PathResolver`
- [ ] Walk through `PathResolver.resolve()` for relative, absolute, `.` and `..` paths
- [ ] Predict the result of `resolve()` by reading the mount sort order

---

Core Concepts

Two layers of resolution

Path resolution in TSIX runs in two layers:

1. **`PathResolver.resolve()`** — normalizes a path string. It only processes text (`//`, `.`, `..`), without knowing about filesystems.
2. **`MountManager.resolve()`** — routes the normalized path to the correct `IVFS` backend, using the "longest prefix wins" rule.

> [!IMPORTANT] `PathResolver` works at the **string** level. `MountManager` works at the **filesystem backend** level. They are two different functions.

MountPoint

```
export interface MountPoint {
  vfsPath: string;     // titik kait di VFS, misal "/tmp"
  vfs: IVFS;           // backend filesystem
  type: string;        // "bkfs" | "ramfs" | "host"
  source: string;      // "system.db" | "shared" | "systembak.db" | ...
  readOnly: boolean;
  uid?: number;
  gid?: number;
}
```

> `type` uses the real values from `fstab.json`: `"bkfs"`, `"ramfs"`, and `"host"` (HostVFS).

PathResolver.resolve() — walkthrough

`PathResolver.resolve(cwd, targetPath)` takes two arguments: the working directory (`cwd`) and the target path. The result is always a normalized absolute path (without `//`, `.`, or `..`).

#	cwd	targetPath	Result	Explanation
1	`/`	`/etc/passwd`	`/etc/passwd`	Absolute path — `cwd` is ignored
2	`/`	`etc/passwd`	`/etc/passwd`	Relative path resolved at root
3	`/bin`	`../etc/passwd`	`/etc/passwd`	`..` goes up one level out of `/bin`
4	`/home/user`	`docs/../a.txt`	`/home/user/a.txt`	`..` cancels the `docs` segment
5	`/`	`/tmp/./foo//bar`	`/tmp/foo/bar`	`//` and `.` are cleaned up
6	`/`	`/../x`	`/x`	`..` above root cannot go higher

> [!NOTE] Examples 4 and 5 show that normalization happens **before** mount lookup. So `resolve("/tmp/./foo")` and `resolve("/tmp/foo")` are considered the same by `MountManager`.

Default mounts

Root `/` is **not** in `fstab.json`. It is mounted directly in `initializeSubsystems()`:

```
this.mountManager.mount("/", this.bkfs, "bkfs", cfg.kernel.database, false);
```

`cfg.kernel.database` is `"system.db"` (see `src/sysconfig.json`). The rest are loaded from `src/mirror/etc/fstab.json` by `processFstab()`:

Path	type	Backend	Source	readOnly	uid/gid	mode	Nature
`/`	`bkfs`	BKFS (SQLite)	`system.db`	false	—	—	persistent, root fs
`/tmp`	`ramfs`	RamFS	in-memory (hostPath `"RAM"` is ignored)	false	0/100	`0o1777`	volatile, sticky
`/mnt/shared`	`host`	HostVFS	host folder `shared`	false	1000/100	`0o775`	host bridge, anti-escape
`/mnt/sbak`	`bkfs`	BKFS	`systembak.db`	false	1000/100	`0o775`	backup DB

> [!NOTE] `fstab.json` only contains `/tmp`, `/mnt/shared`, and `/mnt/sbak`. The `/tmp` entry uses `active: true`; the rest are active by default. Mode `1023 = 0o1777` (sticky, everyone can write — typical for `/tmp`), `509 = 0o775`.

Resolution: longest prefix wins

After all mounts are in place, the list is sorted **descending by path length**. The resulting sort order for the default mounts:

```
/mnt/shared  (11)   ← diperiksa dulu
/mnt/sbak    ( 9)
/tmp         ( 4)
/            ( 1)   ← fallback terakhir
```

Example output of `MountManager.resolve()`:

Requested path	Selected MountPoint	relativePath	vfs
`/tmp/a.txt`	`/tmp`	`/a.txt`	RamFS
`/tmp`	`/tmp` (exact)	`/`	RamFS
`/etc/passwd`	`/`	`/etc/passwd`	BKFS
`/mnt/shared/readme.md`	`/mnt/shared`	`/readme.md`	HostVFS

Requested path	Selected MountPoint	relativePath	vfs
`/mnt/shared`	`/mnt/shared` (exact)	`/`	HostVFS
`/mnt/sbak/backup.tar`	`/mnt/sbak`	`/backup.tar`	BKFS
`/mnt/sharedx/foo`	`/`	`/mnt/sharedx/foo`	BKFS

> [!WARNING] The last row is an important gotcha. `/mnt/sharedx/foo` does **not** match the prefix `/mnt/shared/` (because after `shared` there is `x`, not `/`). So it falls back to `/` → BKFS. This is why the prefix uses an extra slash (`m.vfsPath + "/"`), not a plain `startsWith(m.vfsPath)`.

 Path resolution: syscall → MountManager → VFS backend *Source:* *wiki/diagram/Home-2.mmd*

The `/dev/*` path does not pass through MountManager

`dev/xxx` paths are handled by **HAL** (`kernel.devices[xxx]`), bypassing MountManager. This is consistent with the "everything is a file" principle — devices are not vnode filesystems.

Flow / How It Works

Mount flow at boot

```
initializeSubsystems()
  └─ new BKFS("system.db")                      → root filesystem
  └─ mount("/", bkfs, "bkfs", "system.db", false)
  └─ processFstab()                             → baca /etc/fstab.json
       └─ untuk tiap entry:
            active === false → skip
            pastikan dir mount point ada (mode ?? 0o755)
            pilih driver: bkfs → BKFS, ramfs → RamFS, else → HostVFS
            mountManager.mount(vfsPath, driver, type, hostPath, ...)
  └─ mount() normalisasi path + sort descending by panjang
```

Resolution flow on a file syscall

```
Aplikasi → syscall file (mis. OPEN, path mentah)
  → MountManager.resolve(path)
      1. normalized = PathResolver.resolve("/", path)
      2. loop mounts (terpanjang dulu):
           exact match  → relativePath = "/"
           prefix match → relativePath = sisa path
      3. hasil: { vfs, relativePath, mountPoint }
  → vfs.<op>(relativePath)            // backend mengeksekusi
Path /dev/* → kernel.devices[xxx]     // HAL, bukan MountManager
```

Source Code

File	Role
`src/kernel/MountManager.ts`	mount/unmount/resolve/listMounts
`src/common/PathResolver.ts`	Path normalization (`//`, `.`, `..`)
`src/kernel/Kernel.ts`	`initializeSubsystems()` + `processFstab()` at boot
`src/mirror/etc/fstab.json`	Default mount list
`src/vfs/HostVFS.ts`, `RamFS.ts`, `BKFS.ts`	Backends

Snippets (code level)

PathResolver.resolve()

```
public static resolve(cwd: string, targetPath: string): string {
    // 1. Jika path diawali '/', berarti absolute
    let absolutePath = targetPath.startsWith("/")
        ? targetPath
        : (cwd === "/" ? "/" + targetPath : cwd + "/" + targetPath);

    // 2. Normalisasi (Bersihkan //, ./ dan ..)
    const parts = absolutePath.split("/").filter(p => p.length > 0 && p !== ".");
    const stack: string[] = [];

    for (const part of parts) {
        if (part === "..") {
            stack.pop();
        } else {
            stack.push(part);
        }
    }

    return "/" + stack.join("/");
}
```

MountManager.resolve()

```
public resolve(path: string): {
    vfs: IVFS;
    relativePath: string;
    mountPoint: string;
} {
    // Normalize: robustly handle //, ./ and .. using PathResolver
    const normalized = PathResolver.resolve("/", path);

    for (const m of this.mounts) {
      // Case 1: Exact match
      if (normalized === m.vfsPath) {
        return { vfs: m.vfs, relativePath: "/", mountPoint: m.vfsPath };
      }

      // Case 2: Sub-path match
      // Special handling for root mount point "/"
      const prefix = m.vfsPath === "/" ? "/" : m.vfsPath + "/";
      if (normalized.startsWith(prefix)) {
        let relative = normalized.substring(
          m.vfsPath === "/" ? 0 : m.vfsPath.length,
        );
        if (!relative.startsWith("/")) relative = "/" + relative;
        return { vfs: m.vfs, relativePath: relative, mountPoint: m.vfsPath };
      }
    }

    throw new Error(`Path not found in any mounted filesystem: ${path}`);
}
```

MountManager.mount()

```
public mount(
    vfsPath: string,
    vfs: IVFS,
    type: string = "bkfs",
    source: string = "system.db",
    readOnly: boolean = false,
    uid?: number,
    gid?: number,
) {
    // Normalisasi path menggunakan PathResolver agar konsisten
```

```
    const cleanPath = PathResolver.resolve("/", vfsPath);

    // Cek apakah sudah ada
    const existing = this.mounts.find((m) => m.vfsPath === cleanPath);
    if (existing) {
      this.logger.warn(`Mount point ${cleanPath} already exists. Replacing...`);
      existing.vfs = vfs;
      existing.type = type;
      existing.source = source;
      existing.readOnly = readOnly;
      existing.uid = uid;
      existing.gid = gid;
    } else {
      this.mounts.push({
        vfsPath: cleanPath,
        vfs,
        type,
        source,
        readOnly,
        uid,
        gid,
      });
      // Sort by path length descending so longest match wins
      this.mounts.sort((a, b) => b.vfsPath.length - a.vfsPath.length);
    }

    this.logger.info(
      `Mounted ${type} filesystem at ${cleanPath} from ${source} (${readOnly ? "ro" : "rw"})`,
    );
}
```

> [!NOTE] `mount()` only sorts when a **new** entry is added. If the path already exists, `mount()` replaces its backend without touching the sort order.

Root mount in initializeSubsystems()

```
this.bkfs = new BKFS(cfg.kernel.database);
this.mountManager.mount("/", this.bkfs, "bkfs", cfg.kernel.database, false);
```

processFstab() — fstab processing loop

```
private async processFstab() {
    if (!this.bkfs) return;

    const fstabPath = "/etc/fstab.json";
    if (!this.bkfs.exists(fstabPath)) return;

    try {
      const content = this.bkfs.read(fstabPath);
      if (!content) return;

      const entries = JSON.parse(content) as any[];
      for (const entry of entries) {
        const { vfsPath, hostPath, type, readOnly, uid, gid, active, mode } =
          entry;

        // Skip if explicitly marked inactive (default: active = true)
        if (active === false) {
          this.bootLogStart(`FSTAB: Skipping ${vfsPath} (inactive)`);
          this.bootLogEnd(true);
          continue;
        }

        // Ensure mount point exists with correct ownership & permissions
        const dirMode = mode ?? 0o755;
        if (!this.bkfs.exists(vfsPath)) {
          this.bkfs.mkdir(vfsPath, uid ?? 0, gid ?? 0, dirMode);
```

```
        } else {
          if (uid !== undefined || gid !== undefined) {
            // Re-apply ownership if dir already existed (e.g. created by ensureDefaultAuth)
            this.bkfs.chown(vfsPath, uid ?? 0, gid ?? 0);
          }
          if (mode !== undefined) {
            this.bkfs.chmod(vfsPath, dirMode);
          }
        }

        let driver: IVFS;
        if (type === "bkfs") {
          driver = new BKFS(
            path.resolve(process.cwd(), hostPath),
            readOnly || false,
            uid,
            gid,
            dirMode,
          );
        } else if (type === "ramfs") {
          // RamFS tidak butuh hostPath — murni di RAM
          const label = vfsPath.replace(/\//g, "_").replace(/^_/, "");
          driver = new RamFS(label, uid, gid, dirMode);
        } else {
          driver = new HostVFS(hostPath, readOnly || false, uid, gid, dirMode);
        }

        this.bootLogStart(`FSTAB: Mounting ${vfsPath} (${type})`);
        this.mountManager.mount(
          vfsPath,
          driver,
          type,
          hostPath,
          readOnly || false,
          uid,
          gid,
        );
        this.bootLogEnd(true);
      }
    } catch (e: any) {
      this.bootLog(`FSTAB: Error processing fstab: ${e.message}`, false);
    }
  }
```

Exercises / Practice

1. Run `mount` (or `cat /proc/mounts` if available) — observe the list of mount points and their order.
2. Write a file in `/tmp/x` then reboot — does the file still exist? (RamFS = volatile)
3. Read `src/kernel/Kernel.ts` — find `initializeSubsystems()` (root mount) and `processFstab()` (fstab).
4. Compute the result of `MountManager.resolve()` for `/mnt/shared/../sbak/x` before running it. Compare with real behavior.
5. Modify `resolve()` to log the resolved path, then open a file from the shell.

References

- `wiki/Virtual-File-System.md` — VFS layer
- `wiki/course/00-overview.en.md` §4.4
- `src/kernel/MountManager.ts`, `src/common/PathResolver.ts`
- `src/kernel/Kernel.ts` — `initializeSubsystems()` & `processFstab()`
- `src/mirror/etc/fstab.json` — default mount list

Module 07 — done. Part II complete. Continue to Module 08 — VFS.

Module 08 **VFS** Storage & I/O

RFC-TSIX-EDU-002 | The eighth module of the TSIX curriculum. Understand the Virtual File System layer: one `IVFS` contract, three backends (BKFS/RamFS/HostVFS), and chunked I/O executed inside SQL.

> Design key: **IVFS is one contract, many backends.** A syscall does not care whether the file lives in SQLite, RAM, or a host folder — it only calls `vfs.read()`. This enables "backend swapping" without changing the kernel.

Learning Objectives

- [ ] List the core methods of the `IVFS` contract
- [ ] Explain the differences between BKFS, RamFS, and HostVFS
- [ ] Explain why chunked I/O is executed inside SQL (SUBSTR/CONCAT)
- [ ] Explain the structure of the `vnodes` table in BKFS
- [ ] Explain the role of `getSize`, which does not fetch content
- [ ] Explain the flow of reading a single file: syscall `OPEN` → `VFS.read` → SQL

Core Concepts

The IVFS Contract

```
export interface IVFS {
    ls(path: string): any[];
    mkdir(path: string, uid?: number, gid?: number, mode?: number): boolean;
    read(path: string): string | null;
    touch(path: string, content?: string, uid?: number, gid?: number, mode?: number): boolean;
    stat(path: string): any;
    chmod(path: string, mode: number): boolean;
    chown(path: string, uid: number, gid: number): boolean;
    unlink(path: string): boolean;
    rmdir(path: string): boolean;
    exists(path: string, type?: VNodeType): boolean;
    getUsage(): Promise<{ size: number, files: number, dirs: number, diskSize?: number }>;
    append(path: string, content: string): boolean;

    // --- Chunked I/O (Progress-aware, untuk file besar) ---
    /** Membaca sebagian konten file (offset-based, return null jika di luar range) */
    readChunk(path: string, offset: number, length: number): string | null;
    /** Menulis (replace) sebagian konten file di offset tertentu */
    writeChunk(path: string, chunk: string, offset: number): boolean;
    /** Mendapatkan ukuran file dalam byte, atau -1 jika tidak ditemukan */
    getSize(path: string): number;
}
```

Three Backends — Comparison

Backend	Source	Persistence	Speed	Use case
BKFS	SQLite `system.db`, `vnodes` table	✅ persistent (disk)	moderate; chunked I/O optimized inside SQL	root filesystem `/`, backup `/mnt/sbak`
RamFS	memory (`VNode` tree)	❌ volatile (lost on restart)	⚡ very fast	`/tmp` (temporary data)
HostVFS	real folder on the host (Node `fs`)	✅ persistent (host files)	moderate (host syscall)	`/mnt/shared` (bridge + anti-escape)

Which backend is used is decided by `MountManager` at boot time (see `src/mirror/etc/fstab.json`):

Mount point	Type	Source	Description
`/`	`bkfs`	`system.db`	persistent root filesystem
`/tmp`	`ramfs`	RAM	volatile, mode `1023` = `0o1777` (sticky)
`/mnt/shared`	`host`	folder `shared`	bridge to host, uid `1000`
`/mnt/sbak`	`bkfs`	`systembak.db`	root backup in a separate SQLite file

> [!NOTE] **"Swap backend" without changing the kernel** The kernel only holds an `IVFS` reference. Because every backend implements the same contract, swapping storage (e.g. `/tmp` from RamFS to BKFS) only requires changing an entry in `fstab.json` — the syscall does not change at all.

VFS routing: app → syscall → MountManager → BKFS/RamFS/HostVFS *Source: `wiki/diagram/Virtual-File-System-1.mmd`*

BKFS: `vnodes` structure

Files in BKFS are stored as **rows** in the `vnodes` table — a tree structure where `parent_id` points to the parent row (`NULL` for the root `/`):

```
CREATE TABLE IF NOT EXISTS vnodes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    parent_id INTEGER,          -- FK ke id parent (NULL untuk root "/")
    name TEXT NOT NULL,
    type TEXT NOT NULL,         -- 'FILE' | 'DIRECTORY'
    content TEXT,               -- isi file (NULL untuk direktori)
    size INTEGER DEFAULT 0,
    uid INTEGER DEFAULT 0,      -- User ID (0 = root)
    gid INTEGER DEFAULT 0,      -- Group ID (0 = root)
    mode INTEGER DEFAULT 420,   -- Permission desimal (octal 644 = 420, 755 = 493)
    created_at INTEGER,
    modified_at INTEGER,
    FOREIGN KEY (parent_id) REFERENCES vnodes(id),
    UNIQUE(parent_id, name)     -- satu nama per folder induk
);
```

Important notes:

- **`uid`/`gid`/`mode` are stored as decimal**, not octal strings. Example: `mode 420` = `0o644`, `493` = `0o755`. Root `/` is inserted at init with mode `493`; `/tmp` in `fstab.json` uses `1023` = `0o1777` (sticky bit).
- **Path navigation** is done row by row: each segment is looked up with `SELECT id FROM vnodes WHERE name = ? AND parent_id = ? AND type = 'DIRECTORY'`, then `parent_id` shifts to the resulting id. This happens in `getNodeId()` / `getNodeIdAndSize()`.
- **`UNIQUE(parent_id, name)`** prevents duplicate names in the same folder. This schema also migrates old DBs: the `uid/gid/mode/size/modified_at` columns are added via `ALTER TABLE` if they do not exist yet.

Chunked I/O in SQL — and why it matters

The main trick: **never pull the full content into JS memory**. For large files, reading everything at once is expensive: memory is used up and the UI feels "hung" while waiting. Chunked I/O breaks it into small pieces (offset + length) — used for progress-aware read/write, streaming, and OTA firmware.

All slicing is done **inside SQLite**:

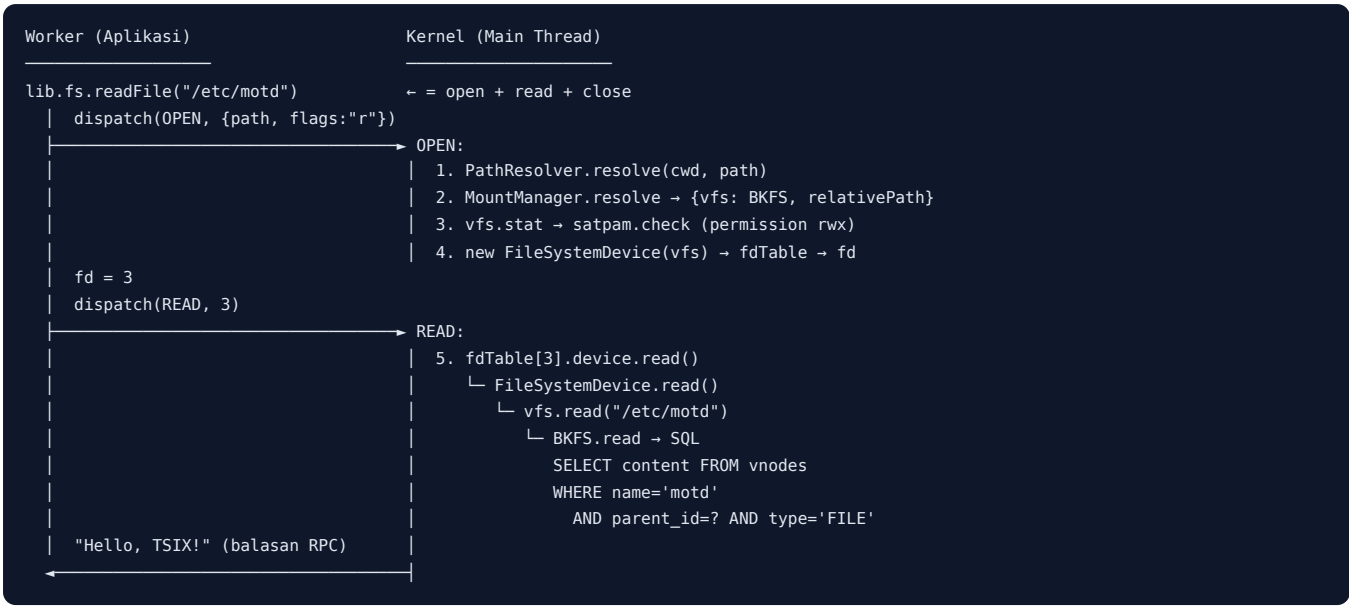
- `readChunk` → `SELECT SUBSTR(content, ? + 1, ?)` — only the requested bytes are read.
- `writeChunk` sequential append (`offset >= currentSize`) → simple `CONCAT` (`IFNULL(content,'') || ?`) — **very fast**, does not rewrite the content.
- `writeChunk` random write in the middle → `SUBSTR(content, 1, ?) || ? || SUBSTR(content, ? + 1)` — overwrites a specific part.

- `getSize` → reads only the `size` column (via `getNodeIdAndSize`), **without fetching content** — cheap for checking progress before `readChunk`.

> [!IMPORTANT] **Why `SUBSTR(content, ? + 1, ?)`?** SQLite `SUBSTR` uses a 1-based index. To read from byte `offset` (0-based in the API), the first parameter must be `offset + 1`. Example: `readChunk(path, 2, 4)` → `SUBSTR(content, 3, 4)`.

Flow / How it works

All file access goes through one path: **syscall → MountManager → IVFS → backend**. Here is the full flow of reading `/etc/motd` from an application (Worker) down to SQLite:



Step by step:

#	Layer	What happens
1	`UserLib` (Worker)	`lib.fs.readFile('/etc/motd')` → `open` → send the `OPEN` syscall via `postMessage` (RPC).
2	Syscall `OPEN`	`PathResolver.resolve(pcb.cwd, path)` normalizes the path (`//`, `.`, `..`) → absolute.
3	`MountManager.resolve()`	Find the **longest** prefix mount point (`/` → BKFS) → `{ vfs, relativePath, mountPoint }`.
4	PermissionManager	`vfs.stat()` fetches metadata (uid/gid/mode) → `satpam.check()` verifies read permission.
5	FD Table	The file is wrapped in `FileSystemDevice`, `setPath(relativePath, flags)`, enters `pcb.fdTable` → `fd` is returned.
6	Syscall `READ`	`pcb.fdTable[fd].device.read()` calls the driver.
7	`FileSystemDevice.read()`	Polymorphic delegation: `vfs.read(this.currentPath)` — the kernel does not care about the backend.
8	`BKFS.read()`	Tree navigation per segment in SQL → `SELECT content ...` → the string is sent back via RPC.

> [!TIP] **Relative vs absolute paths** The syscall receives a path relative to `pcb.cwd`; `PathResolver.resolve()` normalizes it before `MountManager`. So `/etc/../etc/motd` still opens the same file.

Large file variant — the `READ_CHUNK` syscall:

Instead of `READ` (which pulls the whole content into memory), the application uses `lib.fs.readChunk(path, offset, length)`:

```
lib.fs.readChunk("/firmware.bin", 65536, 4096)
  → dispatch(READ_CHUNK, {path, offset, length})
  → MountManager.resolve → vfs.stat → satpam.check(READ)
  → vfs.readChunk(relativePath, offset, length)
      └─ BKFS: SELECT SUBSTR(content, ? + 1, ?) ...   -- hanya byte yang diminta
```

The result: JS memory never holds the whole file — slicing happens inside SQLite.

Source Code

File	Role
`src/vfs/IVFS.ts`	Filesystem contract
`src/vfs/BKFS.ts`	SQLite backend + chunked I/O
`src/vfs/RamFS.ts`	In-memory backend
`src/vfs/HostVFS.ts`	Host backend (anti-escape)

Snippets (code level)

BKFS.read — read full content

```
public read(path: string): string | null {
  const parts = path
    .split("/")
    .filter((p) => p.length > 0 && p !== "." && p !== "..");
  const fileName = parts.pop();
  if (!fileName) return null;

  let parentId = this.getRootId();
  for (const part of parts) {
    let node = this.db
      .prepare(
        "SELECT id FROM vnodes WHERE name = ? AND parent_id = ? AND type = 'DIRECTORY'",
      )
      .get(part, parentId) as { id: number } | undefined;
    if (!node) return null;
    parentId = node.id;
  }

  const file = this.db
    .prepare(
      "SELECT content FROM vnodes WHERE name = ? AND parent_id = ? AND type = 'FILE'",
    )
    .get(fileName, parentId) as { content: string } | undefined;

  return file ? file.content : null;
}
```

Key point: the path is split per segment, then each segment is looked up with `name + parent_id` (tree). The final query targets `type = 'FILE'` — directories are not read.

BKFS.touch — create / overwrite file (upsert)

```
public touch(
  path: string,
  content: string = "",
  uid: number = 0,
  gid: number = 0,
  mode: number = 420,
): boolean {
  if (this.readOnly) throw new Error("Read-only filesystem");
  const parts = path
```

```
    .split("/")
    .filter((p) => p.length > 0 && p !== "." && p !== "..");
  const fileName = parts.pop();
  if (!fileName) return false;

  let parentId = this.getRootId();
  // Navigasi ke folder tujuan
  for (const part of parts) {
    let node = this.db
      .prepare(
        "SELECT id FROM vnodes WHERE name = ? AND parent_id = ? AND type = 'DIRECTORY'",
      )
      .get(part, parentId) as { id: number } | undefined;
    if (!node) return false;
    parentId = node.id;
  }

  // Cek apakah file sudah ada
  const existing = this.db
    .prepare(
      "SELECT id FROM vnodes WHERE name = ? AND parent_id = ? AND type = 'FILE'",
    )
    .get(fileName, parentId);

  if (existing) {
    this.db
      .prepare(
        "UPDATE vnodes SET content = ?, size = ?, modified_at = ? WHERE id = ?",
      )
      .run(content, content.length, Date.now(), (existing as any).id);
    return true;
  }

  const now = Date.now();
  this.db
    .prepare(
      "INSERT INTO vnodes (parent_id, name, type, content, size, uid, gid, mode, created_at, modified_at) VALUES (?, ?, 'FILE', ?, ?,
?, ?, ?, ?, ?)",
    )
    .run(
      parentId,
      fileName,
      content,
      content.length,
      uid,
      gid,
      mode,
      now,
      now,
    );

  this.logger.debug(
    `File created/updated in BKFS: ${fileName} (Mode: ${mode.toString(8)})`,
  );
  return true;
}
```

Key point: `touch` is an **upsert** — an existing file is `UPDATE`d (content + `size`), a missing one is `INSERT`ed. The default mode `420` = `0o644`. The `OPEN` syscall with the `w` flag uses this to create/truncate a file.

readChunk (BKFS) — content chunk

```
public readChunk(path: string, offset: number, length: number): string | null {
    const nodeId = this.getNodeId(path);
    if (nodeId < 0) return null;

    const row = this.db
      .prepare(
        "SELECT SUBSTR(content, ? + 1, ?) as chunk FROM vnodes WHERE id = ? AND type = 'FILE'",
      )
      .get(offset, length, nodeId) as { chunk: string | null } | undefined;
```

```
    return row?.chunk ?? null;
}
```

writeChunk (BKFS) — two paths

```
if (offset >= currentSize) {
    // Sequential append — simple CONCAT (paling cepat)
    UPDATE vnodes SET content = IFNULL(content,'') || ?, size = ?, modified_at = ? WHERE id = ?;
} else {
    // Random write di tengah — SUBSTR + CONCAT
    UPDATE vnodes SET
      content = SUBSTR(content, 1, ?) || ? || SUBSTR(content, ? + 1),
      size = ?, modified_at = ? WHERE id = ?;
}
```

Exercises / Practice

1. Open `system.db` with the SQLite CLI: `.schema vnodes` — observe the table structure.
2. Read `src/vfs/HostVFS.ts` — find `toHostPath()` and check the anti-escape.
3. Write a large file (e.g. 1MB) then call `readChunk` with random offsets — compare the speed vs a full `read`.
4. Read `src/vfs/IVFS.ts` — recognize the whole contract a new backend must satisfy.
5. Place a breakpoint in the `OPEN` and `READ` cases of `src/kernel/Syscalls.ts`, then read `/etc/motd` from the shell — trace down to `BKFS.read` and observe its SQL query.

References

- `wiki/Virtual-File-System.md` — the full VFS layer
- `wiki/course/00-overview.en.md` §5.1
- `src/vfs/IVFS.ts`, `src/vfs/BKFS.ts`, `src/vfs/RamFS.ts`, `src/vfs/HostVFS.ts`
- `src/kernel/MountManager.ts` — backend selection from path (longest prefix)
- `src/kernel/Syscalls.ts` — syscall `OPEN`, `READ`, `READ_CHUNK`, `WRITE_CHUNK`, `GET_SIZE`
- `src/kernel/devices/FileSystemDevice.ts` — FD → IVFS bridge
- `src/mirror/etc/fstab.json` — mount configuration at boot

Module 08 — complete. Continue to Module 09 — FD Table & File Syscalls.

Module 09 **FD Table & File Syscalls** Storage & I/O

RFC-TSIX-EDU-002 | Ninth module of the TSIX curriculum. Understand "everything is a file" at the FD table level: how `open` → `FDEntry` → device, and how a regular file is wrapped by `FileSystemDevice`.

> In TSIX, **a file descriptor is not an index into a file array** — it is an index into an `FDEntry { device, context, flags }` that points to any `IDevice` object. A regular file, TTY, pipe, socket — all of them are `IDevice`.

Learning Objectives

- [ ] Explain the structure of `FDEntry { device, context, flags }` and `fdTable`
- [ ] Explain the `OPEN` flow: resolve path → check permission → pick device → `fdTable.push`
- [ ] Explain how `READ`/`WRITE` dispatch to `entry.device`
- [ ] Explain "everything is a file": regular files vs `/dev/*`
- [ ] Explain pipe refcount via ioctl (`INC_REF`/`DEC_REF`)
- [ ] Explain FD cleanup when a process exits

Core Concepts

FDEntry

```
export interface FDEntry {
    device: IDevice;   // objek device nyata
    context: any;      // state per-FD
    flags?: string;    // "r" | "w" | "a" | "+"
}
```

`fdTable` lives in the PCB — each process has an array of `(FDEntry | null)[]`. FD 0/1/2 (stdin/stdout/stderr) are filled at spawn from the TTY device.

Everything is a File

Opened item	Device in the FD table
Regular file (`/etc/passwd`)	`FileSystemDevice` (wraps IVFS)
`/dev/tty1`	`TTYDevice`
`/dev/null`	`NullDevice`
Pipe (PIPE syscall)	`PipeDevice`
Socket (SOCKET syscall)	`SocketDevice`

There is no *type* difference between a file and a device in the FD table — both are merely objects that implement `IDevice` (`read()`, `write()`, `ioctl()`). Opening a regular file → the kernel creates a new `FileSystemDevice` that wraps the `IVFS` backend returned by `mountManager.resolve()`. Opening `/dev/*` → the kernel takes the device already registered in `kernel.devices` (TTY, null, stdin, etc.). This is the heart of "everything is a file": `read`/`write` are always polymorphic to `entry.device`.

Standard FDs (0/1/2)

When a process is created (`Scheduler.createProcess`) or on `EXEC`, the kernel fills the first three FDs from the stdio devices. The default at `EXEC` is the global device `kernel.devices.stdin`/`stdout`/`stderr`; if the process runs on a TTY (`pcb.ttyId`), all three are pointed at the same `tty{N}`.

FD	Name	flags	Device (common)
`0`	stdin	`"r"`	`TTYDevice` (`ttyN`) when the process has a TTY; fallback `KeyboardDevice` (`kernel.devices.stdin`)
`1`	stdout	`"w"`	`TTYDevice` (`kernel.devices.stdout` = `tty1`)
`2`	stderr	`"w"`	`TTYDevice` (`kernel.devices.stderr` = `tty1`)

> [!NOTE] The `flags` of FD 0 is `"r"` (read), while FD 1 and 2 are `"w"` (write). This is what lets `WRITE` refuse to write to an FD that was not opened for writing ("Bad File Descriptor").

Pipe refcount

A pipe has a refcount on both the read and write sides. EOF occurs when `writeRefs == 0`. The kernel manages the refcount through the device `ioctl` — command numbers:

ioctl cmd	Meaning	Called from
`10`	`INC_READ_REF`	`OPEN` when flags contain `"r"`
`20`	`INC_WRITE_REF`	`OPEN` when flags contain `"w"` / `"a"`
`11`	`DEC_READ_REF`	`CLOSE` when flags contain `"r"`
`21`	`DEC_WRITE_REF`	`CLOSE` when flags contain `"w"` / `"a"`

Every `open` increments, every `close` decrements. `FileSystemDevice` does not use refcount — its `ioctl()` returns `-1` (no-op). Refcount only matters for real devices such as pipes, TTYs, and serial.

Flow / How It Works

OPEN: from path to FD

Steps in the `SyscallCode.OPEN` case (`src/kernel/Syscalls.ts`):

1. **Resolve path** — `PathResolver.resolve(pcb.cwd, args)` converts the path (relative/absolute) to an absolute one.
2. **Resolve VFS** — `mountManager.resolve(absolutePath)` → `{ vfs, relativePath }`. The backend is chosen from the longest mount prefix (`/` → BKFS, `/tmp` → RamFS, `/mnt/*` → HostVFS).
3. **Check permission** — `vfs.stat(relativePath)` to get the node. `requiredPerm` = `WRITE` if flags contain `"w"` / `"+"`, otherwise `READ`.
   - Node exists → `satpam.check(pcb, node, requiredPerm)`; failure → `Permission Denied`.
   - Node does not exist → flags `"r"` → `File not found`; flags `"w"` / `"+"` → check the parent directory's permission, then `vfs.touch` creates the file (mode `0644`), and truncates when flags are `"w"` without `"a"`.
4. **Pick device**:
   - Path `/dev/*` → look up `kernel.devices[devName]` (aliases: `screen` / `console` / `stdout` / `fb0` and `keyboard` / `stdin`; `tty` → `tty{pcb.ttyId}`). The refcount is incremented via `ioctl` `INC_*_REF`, then `device.open()` is called lazily.
   - Regular path → `new FileSystemDevice(vfs)` + `setPath(relativePath, flags)`.
5. **Insert into the table** — `const fd = pcb.fdTable.length; pcb.fdTable.push({ device, context, flags }); return fd;`. `fd` is a new index in the array, not a free number. For a regular file, `context` is set to `relativePath`; for `/dev/*`, `context` is set to the absolute path.

READ / WRITE: dispatch to `entry.device`

`READ` and `WRITE` do not care about the device type. They take `entry = pcb.fdTable[fd]`, then call the polymorphic method on `entry.device`:

- `READ(fd)` → `entry.device.read()` — a TTY reads its buffer, a file reads from the VFS.
- `WRITE(fd, content)` → check `entry.flags` (must contain `"w"` / `"a"` / `"+"`, otherwise `Bad File Descriptor`), then `entry.device.write(content)`.

There is a special IPC route: `WRITE` with `{ pid, content }` writes to the target process's `fdTable[0]` (TTY → injection via `ioctl(0x2001)`); `READ` with `{ pid }` reads from the target process's `fdTable[1]` (TTY → `ioctl(0x2002)`).

Walkthrough: opening `/etc/passwd`

Trace a shell process that executes `cat /etc/passwd`:

```
cat /etc/passwd
     |
     | OPEN("/etc/passwd", "r")
     v
+-----------------------------------------------------------+
| case OPEN (kernel)                                        |
|  1. PathResolver.resolve(cwd, path)                       |
|       -> "/etc/passwd"  (absolut)                        |
|  2. mountManager.resolve("/etc/passwd")                   |
|       -> { vfs: BKFS, relativePath: "etc/passwd" }        |
|  3. vfs.stat("etc/passwd") -> node ada                    |
|     satpam.check(pcb, node, READ) -> lolos                |
|  4. const fsDriver = new FileSystemDevice(BKFS)           |
|     fsDriver.setPath("etc/passwd", "r")                   |
|  5. const fd = pcb.fdTable.length        // misal = 3     |
|     pcb.fdTable.push({ device: fsDriver,                  |
|                       context: "etc/passwd",              |
|                       flags: "r" })                       |
|  return fd;                            // fd = 3          |
+-----------------------------------------------------------+
     |
     | fd = 3, lalu READ(3)
     v
+-----------------------------------------------------------+
| case READ                                                 |
|  const entry = pcb.fdTable[3];                            |
|  return entry.device.read();                              |
|      -> FileSystemDevice.read()                           |
|         -> vfs.read("etc/passwd")   // BKFS: SQL SELECT   |
+-----------------------------------------------------------+
     |
     | isi file /etc/passwd
     v
     | CLOSE(3)
     v
+-----------------------------------------------------------+
| case CLOSE                                                |
|  device.ioctl(11 / 21)  // DEC_REF; FileSystemDevice no-op |
|  pcb.fdTable[3] = null;                                   |
+-----------------------------------------------------------+
```

> [!NOTE] `FileSystemDevice` returns `null` when `currentPath` is empty and `false` when writing without a path — a "fail safe" before touching the VFS.

Cleanup on exit

The kernel installs a global exit hook in the `SyscallHandler` constructor (`setOnProcessExit`). When a process EXITED:

1. **Close all FDs** — iterate `pcb.fdTable`; every entry that is not `null` is `CLOSE`-dispatched (triggering `DEC_*_REF` on devices that support it). Errors during mass cleanup are ignored.
2. **Port safety net** — `portManager.releasePortsByPid(pid)` releases all MQTNL ports owned by the process (in anticipation of sockets that escaped `CLOSE`).
3. **GUI cleanup** — `guiRegistry.destroyAllForPid(pid)` destroys all windows owned by the process and sends `GUI_REQ DESTROY_WINDOW` to the GUI daemon (`gued`).

Source Code

File	Role
`src/kernel/Scheduler.ts`	Defines `FDEntry` + `fdTable` in the PCB
`src/kernel/devices/FileSystemDevice.ts`	File ↔ IVFS bridge
`src/kernel/devices/PipeDevice.ts`	Pipe + refcount
`src/kernel/Syscalls.ts`	OPEN/READ/WRITE/CLOSE/PIPE + cleanup

Snippets (code level)

OPEN — resolve, permission, pick device, fdTable.push

```
case SyscallCode.OPEN: {
  // ... PathResolver.resolve + mountManager.resolve (lihat "Alur / Cara Kerja") ...

  // 1. Permission Check
  const requiredPerm =
    flags.includes("w") || flags.includes("+")
      ? Permission.WRITE
      : Permission.READ;

  // Existence Check
  const node = vfs.stat(relativePath);

  if (node) {
    // Check File Permission
    if (!this.satpam.check(pcb, node, requiredPerm)) {
      throw new Error(
        `Permission Denied: Cannot open ${absoluteOpenPath} for ${Permission[requiredPerm]}`,
      );
    }
  } else {
    // ... "File not found", cek parent-dir, touch buat file, truncate ...
  }

  // ... branch /dev/* -> kernel.devices[devName] + INC_REF + lazy open ...

  // File biasa -> bungkus FileSystemDevice
  const fsDriver = new FileSystemDevice(vfs);
  fsDriver.setPath(relativePath, flags);

  // TRUNCATE: If opened with 'w' (and not 'a'), we must clear content first
  if (
    flags &&
    flags.includes("w") &&
    !flags.includes("a") &&
    !flags.includes("+")
  ) {
    vfs.touch(relativePath, "", pcb.uid, pcb.gid, 420);
  }

  const fd = pcb.fdTable.length;
  pcb.fdTable.push({ device: fsDriver, context: relativePath, flags });
  return fd;
}
```

> [!IMPORTANT] Note two details: (1) `fd` is `pcb.fdTable.length` — a new index at the end of the array, not a free number; (2) for a regular file `context` is set to `relativePath`, while for `/dev/*` it is set to the absolute path. `context` is freely interpreted by the device.

READ / WRITE — dispatch to `entry.device`

```
// WRITE
case SyscallCode.WRITE: {
  // ... route khusus pid -> targetPcb.fdTable[0] (TTY: ioctl(0x2001)) ...

  const { fd, content } = args;
  const entry = pcb.fdTable[fd];
  if (!entry) return false;

  // Check if FD was opened with Write permission
  const flags = entry.flags || "r";
  if (
    !flags.includes("w") &&
    !flags.includes("a") &&
    !flags.includes("+")
  ) {
    throw new Error("Bad File Descriptor: Not open for writing");
  }

  return entry.device.write(content);
}

// READ
case SyscallCode.READ: {
  // ... route khusus pid -> targetPcb.fdTable[1] (TTY: ioctl(0x2002)) ...

  const fd = args as number;
  const entry = pcb.fdTable[fd];
  if (!entry) throw new Error(`FD NOT FOUND: ${fd}`);
  return entry.device.read();
}
```

FileSystemDevice — file ↔ VFS bridge

```
export class FileSystemDevice implements IDevice {
    name = "FileSystem";
    private vfs: IVFS;
    private currentPath: string = "";
    private flags: string = "r";

    constructor(vfs: IVFS) {
        this.vfs = vfs;
    }

    public setPath(path: string, flags: string = "r") {
        this.currentPath = path;
        this.flags = flags;
    }

    read() {
        if (!this.currentPath) return null;
        return this.vfs.read(this.currentPath);
    }

    write(data: any) {
        if (!this.currentPath) return false;

        const isAppend = this.flags.includes("a");
        const isWrite = this.flags.includes("w");
        const isUpdate = this.flags.includes("+");

        if (isAppend || isWrite || isUpdate) {
            return this.vfs.append(this.currentPath, data);
        }

        // Fallback: Default Overwrite (Backward Compatibility)
        return this.vfs.touch(this.currentPath, data);
    }

    ioctl(cmd: number, arg: any) { return -1; }
}
```

FD cleanup on process exit

```
this.scheduler.setOnProcessExit(async (pid) => {
  const pcb = this.scheduler.getProcess(pid);
  if (pcb) {
    // Tutup semua FD — trigger DEC_REF via dispatch CLOSE
    for (let fd = 0; fd < pcb.fdTable.length; fd++) {
      if (pcb.fdTable[fd]) {
        try {
          await this.dispatch(pid, SyscallCode.CLOSE, fd);
        } catch (e) {
          // Ignore errors during mass cleanup
        }
      }
    }

    // === Force release semua port milik PID ini ===
    try {
      this.kernel.getPortManager().releasePortsByPid(pid);
    } catch (_) {}

    // --- GUI Cleanup ---
    const guiRegistry = this.kernel.guiRegistry;
    if (guiRegistry) {
      const ownedWindows = guiRegistry.destroyAllForPid(pid);
      if (ownedWindows.length > 0) {
        const guedPid = guiRegistry.getDaemonPid();
        if (guedPid !== null) {
          for (const wid of ownedWindows) {
            this.scheduler.sendEvent(guedPid, "gui_request", {
              syscall: "GUI_REQ",
              pid,
              wid,
              action: GUIAction.DESTROY_WINDOW,
            } as IGUIPayload);
          }
        }
      }
    }
  }
});
```

Exercises / Practice

1. Read `src/kernel/devices/PipeDevice.ts` — explain when a pipe returns EOF (connect it to `writeRefs` and the `DEC_WRITE_REF` ioctl).
2. From the shell, `echo hello > /tmp/x` then `cat /tmp/x` — trace the OPEN/WRITE/READ/CLOSE syscalls in the log. Note the fd returned by `OPEN` and the device behind it.
3. Read `src/kernel/Syscalls.ts` — find the `OPEN` implementation and how it picks the device (`FileSystemDevice` vs `/dev/*`).
4. Read `createProcess` in `src/kernel/Scheduler.ts` — explain how fds 0/1/2 (stdin/stdout/stderr) are filled from `options.fds`.

References

- `wiki/Virtual-File-System.md` — §device model
- `wiki/course/00-overview.en.md` — §5 VFS & Device Drivers
- `src/kernel/devices/FileSystemDevice.ts` — file ↔ IVFS bridge
- `src/kernel/Syscalls.ts` — OPEN/READ/WRITE/CLOSE/EXIT cleanup
- `src/kernel/Scheduler.ts` — `FDEntry` + `fdTable` in the PCB

Module 09 — done. Continue to Module 10 — Device Drivers (HAL).

Module 10 **Device Drivers (HAL)** Storage & I/O

RFC-TSIX-EDU-002 | Tenth module of the TSIX curriculum. Understand the `IDevice` contract, aux-devices plugins, udev-style configuration, and the virtual `/dev` device table.

> `/dev` in TSIX is the **device registry in the kernel** (`kernel.devices[xxx]`) — not a filesystem vnode. Every device is an `IDevice` object that can be opened like a file.

Learning Objectives

- [ ] Explain the `IDevice` contract (read/write/ioctl + optional init/open/close/present)
- [ ] Explain why `/dev` is virtual — not a filesystem vnode
- [ ] Explain the device registration order at boot
- [ ] Explain the `aux-devices` plugin: `export default` + `static autoRegister` conventions
- [ ] Explain `applyDeviceConfigs()` (udev-style from `sysconfig.json`)
- [ ] Explain `present()` for hotplug (udev-like)
- [ ] Explain the `DbLib` dual transport: `/dev/mysql` vs the `mysqld` daemon
- [ ] Write your own driver (10-step checklist)

Core Concepts

The IDevice Contract

Every device is an object that implements `IDevice`. The kernel only needs to call the three required methods — `read`, `write`, `ioctl` — without knowing what is inside. This is the HAL abstraction: one contract for all hardware.

```
export interface IDevice {
    name: string;
    read(offset?: number, length?: number): any;
    write(data: any, offset?: number): boolean;
    ioctl(cmd: number, arg: any): any;

    init?(ctx: KContext): void; // Opsional: Untuk nerima 'suntikan' dari Kernel
    open?(): boolean; // Opsional: Lazy open device hardware
    close?(): boolean; // Opsional: Close device hardware (with refcounting)

    /**
     * Opsional: Apakah hardware benar-benar tersedia SEKARANG?
     * Dipakai `ls /dev` untuk menyembunyikan device yang tidak present
     * (udev-like hotplug). Jika tidak diimplementasikan → selalu present.
     */
    present?(): boolean;

    uid?: number;
    gid?: number;
    mode?: number;
    disabled?: boolean;
}
```

`KContext` is a utility injection from the kernel into the driver (delivered through `init`):

```
export interface KContext {
    syslog: (message: string) => void;
}
```

Member	Required	Meaning
`name`	✅	Driver name; becomes the path name `/dev/<name>` (lowercase)
`read(offset?, length?)`	✅	Read data from the device
`write(data, offset?)`	✅	Write data; `boolean` success/failure
`ioctl(cmd, arg)`	✅	Control the device (clear, raw mode, refcount, etc.)
`init(ctx)`	⬜	Called once at boot; `ctx.syslog()` for logging
`open()`	⬜	Lazy open when the device is opened (e.g. SerialDevice)
`close()`	⬜	Close the device with refcounting
`present()`	⬜	Hotplug: `ls /dev` hides devices that are not present
`uid/gid/mode`	⬜	Permission metadata (used by the `OPEN` permission check)
`disabled`	⬜	`true` → skipped by the plugin loader at boot

`/dev` Virtual — Not a Vnode

`/dev` does **not** store device nodes in the VFS database. The `OPEN` code comment states it explicitly: *"we don't want to create device nodes in VFS database"*. What exists is only the `/dev` directory (created by `bkfs.mkdir("/dev", 0, 0, 493)` at boot) and the `kernel.devices` table.

Operation	Handling in syscall
`ls /dev`	Special: enumerate `Object.keys(kernel.devices)`, filter by `present()`, report metadata
`open("/dev/xxx")`	`devName = path.replace("/dev/", "")` → `kernel.devices[devName]` → `FDEntry {device, ...}`
`stat("/dev/xxx")`	Special: fetch device metadata (`uid/gid/mode`)
`chmod` / `chown /dev/*`	Root only; set directly on the device object
`/dev/` without a driver	Falls back to the VFS node (if any) with a normal permission check

Aliases recognized by `OPEN`:

- `tty` → the calling process's TTY (`tty${pcb.ttyId || 1}`).
- `screen`, `console`, `stdout`, `fb0` → the caller's active TTY (fallback `fb0`).
- `keyboard`, `stdin` → `kernel.devices.stdin`.

The `OPEN` permission check uses device metadata with a **default** `0o600` (root only) when `mode` is not set:

```
const devPerm = {
    name: devName,
    uid: device.uid ?? 0,
    gid: device.gid ?? 0,
    mode: device.mode ?? 0o600, // Default: root only (rw-------)
};
```

The standard I/O devices (`stdout` / `stdin` / `fb0` / `console` / `screen` / `keyboard`) **bypass** this check. When opened, the kernel calls `ioctl(10)` (INC_READ_REF) / `ioctl(20)` (INC_WRITE_REF) for refcounting, then `device.open()` if available (lazy open). A failed `open()` → error `Failed to open device /dev/xxx`.

The `/dev` Device Table

/dev/	Class	Function
`stdin`	KeyboardDevice	Keyboard input (cooked/raw)
`fb0/stdout/stderr`	TTYDevice	Standard output → active TTY
`tty1..tty32`	TTYDevice	Virtual consoles

/dev/	Class	Function
`null`	NullDevice	Black hole
`smqtnl0/1`	SimpleMQTNLDriver	MQTNL network interface
`randomdevice`	RandomDevice	Random numbers (0–999 per read)
`mcp23017`	MCP23017Device	I2C GPIO (`autoRegister`)
`joystick`	JoystickDevice	USB HID gamepad; `present()` = hotplug
`ttyUSB*`	SerialDevice	Auto-detect serial
`mysql` *(experimental)*	MySQLDevice	External DB connection — `disabled: true` by default
(virtual)	PipeDevice / SocketDevice	Runtime instances, not paths

> [!NOTE] `MySQLDevice` has `disabled: true` in the source code — so `/dev/mysql` is **not registered by default**. Enable it only when a device transport is needed. See the dual transport note below.

Device Registration at Boot

`Kernel.boot()` registers devices in a fixed order:

```
// 1. TTYManager(32) → tty1..tty32 (TTYDevice)
// 2. Map device inti:
this.devices = {
    stdin: new KeyboardDevice(),
    fb0: ttysDevs.tty1,    // Alias fb0 ke TTY1
    stdout: ttysDevs.tty1, // Alias stdout ke TTY1
    stderr: ttysDevs.tty1, // Alias stderr ke TTY1
    null: new NullDevice(),
    ...ttysDevs,
};
// 3. Interface jaringan (cfg.network.interfaces) → SimpleMQTNLDriver
// 4. loadAuxDevices()     → plugin dari folder aux-devices
// 5. SerialDeviceManager  → auto-detect ttyUSB*
// 6. applyDeviceConfigs() → udev-style dari sysconfig.json
// 7. init() semua driver  → suntikan KContext (syslog)
// 8. pastikan /dev ada di VFS (mkdir 493)
```

The order matters: `applyDeviceConfigs()` must run **after** all devices are registered, and `init()` after the configuration is applied.

aux-devices Plugins & the autoRegister Convention

The `src/kernel/devices/aux-devices/` folder is scanned by `loadAuxDevices()` at boot. Two conventions:

1. **`export default` is required** — the loader reads `module.default || module`. Without a default export, the class is not recognized.
2. **`static autoRegister(kernel)` is optional** — for platform-specific hardware configuration (e.g. MCP23017 + I2C bus). Called after the instance is registered.

applyDeviceConfigs (udev-style)

The `devices` block in `sysconfig.json` → "udev" rules: set `mode` / `uid` / `gid` per device name — exactly like Linux udev rules.

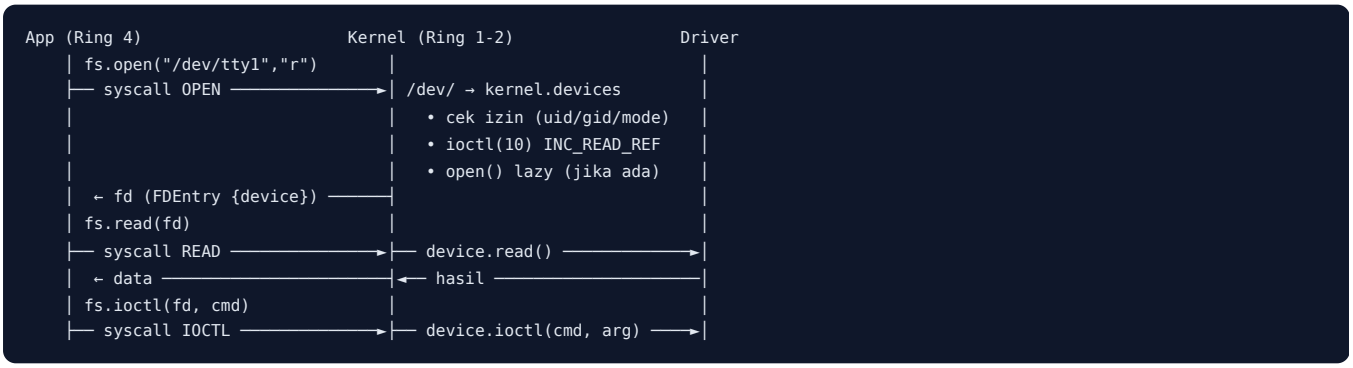
```
{
  "devices": {
    "randomdevice": {
      "mode": 438
    }
```

```
  }
}
```

`438` (decimal) = `0o666` (`rw-rw-rw-`), applied to `/dev/randomdevice`. See the `applyDeviceConfigs()` snippet.

Flow / How It Works

Device access flow (read/write)



Boot registration flow

1. `TTYManager(32)` creates 32 consoles → `TTYDevice` tty1..32.
2. The core map `this.devices` : `stdin` / `fb0` / `stdout` / `stderr` / `null` + tty1..32.
3. Network interfaces from `cfg.network.interfaces` → `SimpleMQTNLDriver` .
4. `loadAuxDevices()` : scan `aux-devices/` → `export default` → register → `autoRegister` .
5. `SerialDeviceManager` detects `ttyUSB*` .
6. `applyDeviceConfigs()` : apply `mode` / `uid` / `gid` from `sysconfig.json` .
7. Loop `init()` over all drivers → inject `KContext.syslog` .
8. The `/dev` directory is guaranteed to exist in VFS.

The `ls /dev` flow (udev-like hotplug)

```
ls /dev
  → syscall LS dengan path /dev
  → enumerasi kernel.devices
  → filter: device dengan present() hanya tampil jika present() true
  → laporan {name, type:"DEVICE", size:0, uid, gid, mode}
```

Example: `JoystickDevice.present()` returns `this.connected` — when the gamepad is unplugged, `/dev/joystick` disappears from `ls /dev` ; when plugged back in, it reappears.

Special Note: `MySQLDevice` & `DbLib` (Dual Transport)

> [!IMPORTANT] **`/dev/mysql` is not a hardware driver.** `MySQLDevice` is the **first transport** for external database integration through the device model — an example of extending HAL in an unusual direction.
>
> **`DbLib` is already implemented** (a UserLib sub-library, the `lib.fs` / `lib.net` pattern) with **pluggable dual transport**:
>
> - **Device** (`/dev/mysql`): syscall `DB_*` (67–69) → kernel → device → `mysql2`
> - **Service daemon** (`mysqld`): syscall `DB_*` → kernel → Ring 4 daemon → `mysql2`
>
> The kernel routes **dynamically**: if `mysqld` is registered (syscall `DB_SERVICE_REGISTER` =70) → to the daemon; if not → to the device (fallback). The daemon replies via `DB_SERVICE_REPLY` =71. Applications only see

> `db.connect()/query()/disconnect()` — the medium is invisible. The sandbox is safe because only the kernel or a privileged daemon touches `mysql2`.

DB syscall codes

Code	Name	Function
67	`DB_CONNECT`	Open a connection (`{host, user, password, database}`)
68	`DB_QUERY`	Execute SQL (SELECT → rows; INSERT/UPDATE/DELETE → ResultSetHeader)
69	`DB_DISCONNECT`	Close the connection
70	`DB_SERVICE_REGISTER`	The `mysqld` daemon registers as a service transport
71	`DB_SERVICE_REPLY`	The daemon sends the result `{requestId, result}` to the kernel

Dynamic routing flow

```
App (Ring 4)   lib.db.query(sql)            ← DbLib (@tsix/DbLib)
  ▼
UserLib.dispatch(DB_QUERY = 68)
  ▼
Kernel (Syscalls.ts)
  │  if (this.dbServicePid !== null)
  ├── YA   → forwardDbRequest(pid, "query", sql)
  │            → sendEvent(mysqld, "db_request") → daemon → mysql2
  │            → DB_SERVICE_REPLY=71 (requestId, result) → resolve pending
  └── TIDAK → /dev/mysql (MySQLDevice) → device.query(sql, pid) → mysql2
  ▼
resolve → App (rows)
```

The routing logic is identical in every `DB_CONNECT` / `DB_QUERY` / `DB_DISCONNECT` case: check `dbServicePid` first, then fall back to the device. When the daemon exits, `Syscalls.ts` resets `dbServicePid = null` so the route automatically returns to the device.

Implementation details

- **`MySQLDevice`** (`aux-devices/MySQLDevice.ts`) has `disabled: true` — **not registered by default**.
- **Multi-instance**: connections are stored per-PID (`Map<pid, Connection>`); each app has its own connection, and two apps can access different servers/databases in parallel. The raw ioctl path without a pid uses an anonymous slot (`ANON_PID = 0`): `ioctl(0x2001)` connect / `ioctl(0x2002)` disconnect.
- **`release(pid)`** : called by the kernel when a process dies → the connection is closed forcibly (anti-leak).
- **The `mysqld` daemon** (`src/mirror/etc/mysqld/mysqld.ts`): privileged Ring 4 — the host module allow-list includes `mysql2` and `mysql2/promise`.
- **Apps never touch `mysql2`** — only `lib.db.*` or `import { db } from "@tsix/Application"`.

Snippet (code level)

All snippets below are **exact copies from the source** (code is the truth).

The IDevice contract — `src/kernel/devices/IDevice.ts`

```
export interface IDevice {
    name: string;
    read(offset?: number, length?: number): any;
    write(data: any, offset?: number): boolean;
    ioctl(cmd: number, arg: any): any;

    init?(ctx: KContext): void; // Opsional: Untuk nerima 'suntikan' dari Kernel
    open?(): boolean; // Opsional: Lazy open device hardware
    close?(): boolean; // Opsional: Close device hardware (with refcounting)
```

```
    /**
     * Opsional: Apakah hardware benar-benar tersedia SEKARANG?
     * Dipakai `ls /dev` untuk menyembunyikan device yang tidak present
     * (udev-like hotplug). Jika tidak diimplementasikan → selalu present.
     */
    present?(): boolean;

    uid?: number;
    gid?: number;
    mode?: number;
    disabled?: boolean;
}
```

A small real driver — `NullDevice` (`/dev/null`)

```
// src/kernel/devices/NullDevice.ts
import { IDevice } from "./IDevice";

/**
 * NULL DEVICE (/dev/null)
 *
 * Lubang hitam kernel.
 * Apapun yang ditulis ke sini akan dibuang.
 * Apapun yang dibaca dari sini akan langsung EOF (null/empty).
 */
export class NullDevice implements IDevice {
    name = "NullDevice";

    read() {
        return "";
    }

    write(_data: any): boolean {
        return true;
    }

    ioctl(_cmd: number, _arg: any): any {
        return true;
    }
}
```

> Note: `IDevice.ts` also contains a concise `NullDevice` (name `"Null"`, `ioctl` → `-1`) as a learning example. The `/dev/null` driver used at boot is the `NullDevice.ts` above.

Driver with init + default export — `RandomDevice` (`/dev/randomdevice`)

```
// src/kernel/devices/aux-devices/RandomDevice.ts
import { IDevice, KContext } from "../IDevice";

/**
 * RANDOM DEVICE (/dev/random)
 *
 * Menghasilkan angka acak sebagai string.
 * Implementasi sederhana untuk demonstrasi plugin system.
 */
export class RandomDevice implements IDevice {
    name = "RandomDevice";
    private kctx: KContext | null = null;

    init(ctx: KContext) {
        this.kctx = ctx;
        this.kctx.syslog("Driver initialized and ready.");
    }

    read() {
        // Balikin angka acak 0-999 sebagai string
```

```
        const val = Math.floor(Math.random() * 1000);
        if (this.kctx) {
            this.kctx.syslog(`Random number generated: ${val}`);
        }
        return val.toString() + "\n";
    }

    write(_data: any): boolean {
        // Menulis ke random device biasanya diabaikan atau buat seeding
        if (this.kctx) {
            this.kctx.syslog("Write attempt to RandomDevice ignored.");
        }
        return true;
    }

    ioctl(_cmd: number, _arg: any): any {
        return true;
    }
}

// Plugin Export: Harus export default class yang implement IDevice
export default RandomDevice;
```

loadAuxDevices() — plugin loader (`src/kernel/Kernel.ts`)

```
private loadAuxDevices() {
    if (!this.devices) return;

    const auxPath = path.resolve(__dirname, "devices/aux-devices");
    if (!fs.existsSync(auxPath)) {
        this.logger.debug(`Auxiliary devices directory not found: ${auxPath}`);
        return;
    }

    const files = fs.readdirSync(auxPath);
    files.forEach((file) => {
        if (file.endsWith(".ts") || file.endsWith(".js")) {
            try {
                const fullPath = path.join(auxPath, file);
                const module = require(fullPath);
                const DeviceClass = module.default || module;

                if (typeof DeviceClass === "function") {
                    // 1. Try auto-loading as device instance (original behavior)
                    const instance = new DeviceClass() as IDevice;

                    // Check if device is explicitly disabled
                    if (instance.disabled === true) {
                        this.logger.debug(
                            `[Dynamic HAL] Kernel Plugin ${file} is disabled, skipping.`,
                        );
                        return;
                    }

                    const devName = (
                        instance.name || file.replace(".ts", "").replace(".js", "")
                    ).toLowerCase();
                    this.devices![devName] = instance;
                    this.logger.info(
                        `[Dynamic HAL] Kernel Plugin Loaded: /dev/${devName}`,
                    );

                    // 2. Check for static autoRegister method (new convention)
                    if (typeof (DeviceClass as any).autoRegister === "function") {
                        try {
                            (DeviceClass as any).autoRegister(this);
                            this.logger.debug(
                                `[Dynamic HAL] Auto-register called for ${file}`,
                            );
                        } catch (e: any) {
                            this.logger.debug(
                                `[Dynamic HAL] Auto-register skipped for ${file}: ${e.message}`,
```

```
                        );
                    }
                }
            }
        } catch (e: any) {
            this.logger.error(`Failed to load aux device ${file}: ${e.message}`);
        }
    }
});
}
```

> Note the `const DeviceClass = module.default || module;` line — this is why `export default` is **required** so the class can be instantiated.

The autoRegister convention — `MCP23017Device`

```
// src/kernel/devices/aux-devices/MCP23017Device.ts (bagian)

// Platform-specific hardware configuration
const HARDWARE_CONFIGS = [
    { bus: 2, address: 0x20, name: "mcp23017" }  // Orange Pi 3B default
];

export class MCP23017Device implements IDevice {
    name: string;
    uid: number = 0;
    gid: number = 0;
    mode: number = 0o660;
    disabled: boolean = false;

    static autoRegister(kernel: any): void {
        for (const config of HARDWARE_CONFIGS) {
            try {
                const device = new MCP23017Device(config.bus, config.address, config.name);
                kernel.devices[config.name] = device;
            } catch (e: any) {
                // Silently skip if hardware not present or i2c-bus not available
            }
        }
    }
    // ... (pinMode/digitalWrite/digitalRead)
}

export default MCP23017Device;
```

applyDeviceConfigs() — udev-style (`src/kernel/Kernel.ts`)

```
private applyDeviceConfigs() {
    const cfg = Config.get();
    if (!cfg.devices) return;

    for (const devName in cfg.devices) {
        const device = this.devices[devName];
        if (device) {
            const devCfg = cfg.devices[devName];
            if (devCfg.mode !== undefined) device.mode = devCfg.mode;
            if (devCfg.uid !== undefined) device.uid = devCfg.uid;
            if (devCfg.gid !== undefined) device.gid = devCfg.gid;
            this.logger.info(
                `[udev] Configuration applied to / dev / ${devName}: mode = ${device.mode?.toString(8)}, uid = ${device.uid}, gid =
${device.gid}`,
            );
        }
    }
}
```

`OPEN` routing for `/dev/*` — `src/kernel/Syscalls.ts` (core excerpt)

```
if (absoluteOpenPath.startsWith("/dev/")) {
    const devName = absoluteOpenPath.replace("/dev/", "");
    let device: IDevice | null = null;
    if (this.kernel.devices && this.kernel.devices[devName]) {
        device = this.kernel.devices[devName];
    }
    // ... (alias: tty / screen,console,stdout,fb0 / keyboard,stdin — diringkas)
    if (device) {
        const devPerm = {
            name: devName,
            uid: device.uid ?? 0,
            gid: device.gid ?? 0,
            mode: device.mode ?? 0o600, // Default: root only (rw-------)
        };
        if (!this.satpam.check(pcb, devPerm, requiredPerm)) {
            throw new Error(
                `Permission Denied: You cannot access device /dev/${devName}`,
            );
        }
        const f = flags || "r";
        if (f.includes("r") && device.ioctl) await device.ioctl(10, null); // INC_READ_REF
        if ((f.includes("w") || f.includes("a")) && device.ioctl)
            await device.ioctl(20, null); // INC_WRITE_REF
        if (device.open) {
            const opened = device.open();
            if (!opened) {
                throw new Error(`Failed to open device /dev/${devName}`);
            }
        }
        const fd = pcb.fdTable.length;
        pcb.fdTable.push({ device, context: absoluteOpenPath, flags });
        return fd;
    }
}
```

Guide: Writing Your Own Driver

> [!TIP] This answers the "Next steps" in the Module 10 ToC: *a tutorial for writing your own driver*. The smallest examples already exist: `NullDevice` and `RandomDevice`.

File locations

Driver type	Location	How to register
Core device (stdin, fb0, tty, null)	`src/kernel/devices/*.ts`	Directly in `Kernel.boot()` (the `this.devices` map)
Hardware / experimental	`src/kernel/devices/aux-devices/<Name>Device.ts`	Auto via `loadAuxDevices()` + `export default`
Network interface	—	Via `cfg.network.interfaces` in `sysconfig.json`

The 10-step checklist

1. **Create the file**: `src/kernel/devices/aux-devices/EchoDevice.ts`.
2. **Import the contract**: `import { IDevice, KContext } from "../IDevice";`
3. **Write the class**: `export class EchoDevice implements IDevice { name = "echo"; ... }` — `name` determines the path `/dev/echo` (lowercase).
4. **Implement the required methods**: `read()`, `write()`, `ioctl()`. The kernel only calls these three.
5. **Optional lifecycle**: `init(ctx)` for syslog, `open()`/`close()` for lazy open, `present()` for hotplug.
6. **Optional metadata**: `uid/gid/mode` (default permission `0o600`), `disabled` to turn it off at boot.

7. **`export default EchoDevice;`** — REQUIRED. Without it `loadAuxDevices()` does not recognize the plugin (`module.default || module`).
8. **Optional `static autoRegister(kernel)`** — for platform-specific hardware configuration (bus, address, name).
9. **Set udev-style permissions**: `sysconfig.json` → `"devices": { "echo": { "mode": 438 } }`.
10. **Test**: boot → `ls /dev/echo` → read/write from the shell → check the syslog for `[Dynamic HAL] Kernel Plugin Loaded: /dev/echo`. Add a unit test in `src/kernel/devices/aux-devices/C10-AuxDevices.test.ts` (the C10.08–C10.12 pattern).

> [!IMPORTANT] Default export convention: the loader uses `const DeviceClass = module.default || module;` then `if (typeof DeviceClass === "function")`. The driver class **must** be `export default` so it can be instantiated. `MCP23017Device` satisfies both: `export default` + `static autoRegister`.

Exercises / Practice

1. Read `wiki/DEVELOPER_GUIDE_DEVICES.md` — the driver-writing guide.
2. Read `src/kernel/devices/aux-devices/RandomDevice.ts` — the minimal device pattern (`export default` + `init`).
3. Read `src/kernel/devices/aux-devices/MCP23017Device.ts` — study `autoRegister` + `disabled`.
4. Read `src/kernel/devices/aux-devices/joystick.ts` — study `present()` (udev-like hotplug).
5. Run `ls /dev` after boot — notice `/dev/randomdevice` and its metadata.
6. Change `mode` in `sysconfig.json` (the `devices.randomdevice` block) → reboot → observe the `ls /dev` results.
7. (Challenge) Write the `/dev/echo` driver: `read()` returns the last text written, `write()` stores it. Follow the 10-step checklist above, then test it from the shell.

References

- `wiki/DEVELOPER_GUIDE_DEVICES.md` — driver-writing guide
- `wiki/mcp23017-registration.md` — plugin registration example
- `wiki/course/00-overview.en.md` §5.2 — device model (HAL) + DbLib notes
- `src/kernel/devices/IDevice.ts` — the `IDevice` + `KContext` contract
- `src/kernel/devices/NullDevice.ts`, `src/kernel/devices/aux-devices/RandomDevice.ts`, `src/kernel/devices/aux-devices/joystick.ts` — driver examples
- `src/kernel/devices/aux-devices/MCP23017Device.ts`, `src/kernel/devices/aux-devices/MySQLDevice.ts` — autoRegister & DB transport
- `src/kernel/Kernel.ts` — boot, `loadAuxDevices()`, `applyDeviceConfigs()`
- `src/kernel/Syscalls.ts` — `OPEN` / `LS` / `STAT` / `CHMOD` / `CHOWN` routing for `/dev`, DB_* 67–71
- `src/common/SyscallCode.ts` — `DB_*` syscall codes
- `src/mirror/lib/DbLib.ts` — the `db.connect/query/disconnect` API
- `src/mirror/etc/mysqld/mysqld.ts` — service daemon (alternative transport)
- `src/kernel/devices/aux-devices/C10-AuxDevices.test.ts` — driver tests (C10.08–C10.12)
- `src/sysconfig.json` — the `devices` block (udev-style)

Module 10 — complete. Part III done. Continue to Module 11 — Worker Thread & Sandbox.

Module 11 **Worker Thread & Sandbox** Process Isolation

RFC-TSIX-EDU-002 | Eleventh module of the TSIX curriculum. Understand how one worker thread becomes one TSIX process, and how the sandbox locks the doors to the host API.

> `WorkerEntry.ts` is the **bootloader** that runs first inside every worker. It initializes UserLib, loads the app, then **locks the doors** (`restrictHostAPI`) before the app runs. This sandbox is one of TSIX's two real privilege boundary layers.

Learning Objectives

- [ ] Explain TSIX's two real privilege boundary layers (thread + PermissionManager, and the WorkerEntry sandbox)
- [ ] Explain WorkerEntry's order of operations (bootloader)
- [ ] Explain the `Module._load` hijack and resolution of `@tsix/*` / `@common/*` from `vfsCache`
- [ ] Explain Direct Memory Execution (`_compile` with a dummy filename)
- [ ] Explain what is sabotaged in `process` (`exit` / `kill`)
- [ ] Explain the `isPrivileged` rule and the host module allow-list
- [ ] Explain why the `@tsix/*` framework is always accessible
- [ ] Explain unhandledRejection/uncaughtException handling
- [ ] Explain the weakness of the name-substring heuristic and defense in depth

---

Core Concepts

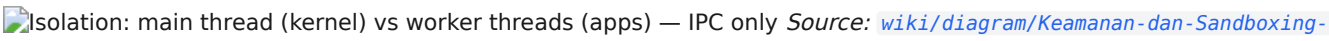
TSIX has **two real privilege boundary layers**. They work together, but each has different properties.

 Four layers of TSIX security *Source:* *wiki/diagram/Keamanan-dan-Sandboxing-1.mmd*

1. Thread + IPC + PermissionManager boundary (kernel)

The kernel runs on the **main thread**. Each app runs on its **own worker thread**. An app never touches kernel memory; the only bridge is `postMessage` (syscall request → response). On the kernel side, `PermissionManager` checks rwx (root bypass → owner → group → others), `validateArgs` validates the syscall argument contract, and the `SETUID` bit allows root-only privilege elevation (e.g. `/bin/login`).

This layer **cannot be tricked** by app code — the app has no reference to kernel objects.

 Isolation: main thread (kernel) vs worker threads (apps) — IPC only *Source:* *wiki/diagram/Keamanan-dan-Sandboxing-2.mmd*

2. WorkerEntry sandbox (worker-local)

`WorkerEntry.ts` is the **bootloader** that runs first inside the worker. It sabotages dangerous host APIs and restricts `require` , so the app is **forced** to use syscalls through UserLib. Core mechanisms:

Mechanism	Purpose
Hijack `Module._load`	`@tsix/*` & `@common/*` are resolved from `vfsCache` (memory), not the filesystem
Direct Memory Execution (DME)	`_compile` content from memory with a **dummy filename**; no disk hit
`restrictHostAPI(appName)`	Locks the doors: replaces `global.require` , sabotages `process.exit` / `process.kill`
Privileged check	**App name substring** (fragile): `server` , `daemon` , `dome` , `tbuild` , `vfs` , `mysqld`
Host module allow-list	Only privileged apps may `require` certain host modules
Fatal handler	`unhandledRejection` / `uncaughtException` → send `GUI_WINDOW_ERROR` to parent → `realExit(1)`

require behavior: non-privileged vs privileged

`require` request	Non-privileged app	Privileged app
Framework `@tsix/*`, `@common/*`	✅ always allowed	✅ always allowed
Path `/lib/...` or `/common/...`	✅ always allowed	✅ always allowed
Host allow-list modules (`http`, `ws`, `path`, `fs`, `url`, `esbuild`, `crypto`, `os`, `bcryptjs`, `mysql2`, `mysql2/promise`)	🚫 throw	✅ allowed
Other host modules (`net`, `child_process`, ...)	🚫 throw	🚫 throw (specific message)
`process.exit` / `process.kill`	🚫 throw	🚫 throw

> [!IMPORTANT] The `@tsix/*` and `@common/*` frameworks are **always** accessible, even in a locked sandbox. Reason: the framework is the only legitimate bridge to syscalls — without it, an app cannot do anything useful.

Known weaknesses

> [!WARNING] The privileged check is an **app name substring** heuristic — fragile. Anyone can name their app `evil-daemon` and gain the host module allow-list. This is **not** a real security boundary; it is only a convenience barrier. The real defense stays in the kernel: `PermissionManager` + `validateArgs` + `SETUID`. Long-term plan: replace this heuristic with *capability-based* (the app declares the permissions it needs).

Flow / How It Works

Short walkthrough

```
app dipanggil (tsh / rc.local / spawnProcess)
        │  syscall EXEC
        ▼
Kernel.spawnWorker()
        │  new Worker(WorkerEntry.ts, { workerData, execArgv })
        ▼
WorkerEntry.ts (bootloader di dalam worker)
  1. simpan realExit = process.exit    (sebelum disabotase)
  2. hijack Module._load                → @tsix/* & @common/* dari vfsCache
  3. pasang unhandledRejection / uncaughtException
  4. UserLib ← hijackRequire("@tsix/UserLib") dari memory cache
     lib = new UserLibClass(pid)  →  global._tsixLib
  5. DME aplikasi: transpile appContent → _compile (dummy filename)
  6. cari export main / Main / default
  7. restrictHostAPI(appName)     ← KUNCI PINTU (sebelum app jalan)
  8. new AppClass() → await execute(lib, args)
  9. hasil string → lib.std.print(); lalu lib.shell.exit(0)
        │
        ▼
KERNEL — tiap akses resource = syscall → PermissionManager
```

Detailed steps (matching the code)

1. **Kernel.spawnWorker** (`src/kernel/Scheduler.ts`) builds the `workerData: WorkerInitData`:

```
const workerData: WorkerInitData = {
    pid: pcb.pid,
    appName: options.appName || pcb.name,
    args: options.args || [],
    appPath: options.appPath,
```

```
    stackBkfsPath: options.stackBkfsPath,
    appContent: options.appContent,
    env: pcb.env,
    vfsCache: this.vfsCacheProvider ? this.vfsCacheProvider() : {}
};
```

`vfsCache` is the result of `rebuildVFSCache()`: the framework `/lib` is pre-compiled into memory at boot.

2. **execArgv** is chosen based on the target extension:
    - `*.js` → **JS-Direct (FAST)**: only `--enable-source-maps`.
    - otherwise (`.ts`) → **TS-Transpile**: adds `-r esbuild-register` and `-r tsconfig-paths/register`.
3. The **bootloader** (`WorkerEntry.ts`) immediately stores `realExit = process.exit.bind(process)` — used later to really exit even though `process.exit` has been sabotaged.
4. **Hijack `Module._load`**: all `require("@tsix/...")` / `require("@common/...")` calls (and relative aliases) are resolved from `vfsCache`. If found, the content is `_compile`d from memory (DME) with a **dummy filename**, then cached. If not found, it falls back to `originalLoad` (host require).
5. **UserLib is loaded from the memory cache**: `hijackRequire("@tsix/UserLib")` → `new UserLibClass(pid)` → `global._tsixLib`. This provides `lib.fs`, `lib.shell`, `lib.std`, `lib.net`, etc.
6. **App DME**: if there is no physical `appPath`, `appContent` (from `workerData`) is transpiled (TS → JS via `esbuild.transformSync`) then `_compile`d with **two filenames**:
    - `moduleFilename` (physical) → so `require` can find `node_modules`.
    - `stackFilename` (BKFS path, `.ts` → `.js`) → so the stack trace points to the correct BKFS path.
7. **Find AppClass**: `exports.main || exports.Main || exports.default || the first exported function`.
8. **`restrictHostAPI(appName)` is called right after AppClass is found and before `new AppClass()`** — "lock the doors before the app runs". From here on, `require` is restricted and `process.exit`/`process.kill` → throw.
9. **Run**: `new AppClass()` → `await app.execute(lib, args)`. If it returns a non-empty string, it is printed via `lib.std.print`, then `lib.shell.exit(0)`.
10. **If a runtime error occurs**: send `GUI_WINDOW_ERROR` to the parent (WM/Asteracea), print to `lib.std.error`, then `realExit(1)`.

Source Code

File	Role
`src/userland/WorkerEntry.ts`	Worker bootloader + sandbox (`restrictHostAPI`)
`src/kernel/Scheduler.ts`	`spawnWorker()` — builds `workerData`, `vfsCache`, `execArgv`
`src/common/IPCTypes.ts`	Types `WorkerInitData` & `SyscallResponse`

Snippets (code level)

> [!NOTE] All snippets below are copied from `src/userland/WorkerEntry.ts` (verify directly against the code).

1. Hijack `Module._load` (framework resolution from `vfsCache`)

```
const originalLoad = Module._load;
const vfsCache = (workerData as any).vfsCache || {};
const moduleCache: Record<string, any> = {};
```

```
Module._load = function (request: string, parent: any, isMain: boolean) {
    let normalizedRequest = request;

    // Resolve relative paths
    if (request.startsWith(".")) {
        if (request.includes("/common/")) {
            normalizedRequest = "@common/" + request.split("/common/")[1];
        } else if (request.includes("/lib/")) {
            normalizedRequest = "@tsix/" + request.split("/lib/")[1];
        } else if (parent && parent.filename) {
            const basename = path!.basename(parent.filename);
            if (basename.startsWith("@tsix_") && request.startsWith("./")) {
                normalizedRequest = "@tsix/" + request.substring(2);
            } else if (basename.startsWith("@common_") && request.startsWith("./")) {
                normalizedRequest = "@common/" + request.substring(2);
            }
        }
    }

    // Cached Module
    if (moduleCache[normalizedRequest]) return moduleCache[normalizedRequest];

    // Resolusi Memory Framework (VFS)
    let vfsPath = null;
    if (normalizedRequest.startsWith("@tsix/")) {
        vfsPath = "/lib/" + normalizedRequest.substring(6) + ".ts";
    } else if (normalizedRequest.startsWith("@common/")) {
        vfsPath = "/lib/common/" + normalizedRequest.substring(8) + ".ts";
    }

    if (vfsPath && vfsCache[vfsPath]) {
        const content = vfsCache[vfsPath];
        const dummyFilename = path!.join(process.cwd(), normalizedRequest.replace("/", "_") + ".js");

        const newMod = new Module(dummyFilename, parent);
        newMod.filename = dummyFilename;
        newMod.paths = Module._nodeModulePaths(process.cwd());

        // Framework modules are now pre-compiled in Kernel.
        // Direct execution for maximum performance.
        (newMod as any)._compile(content, dummyFilename);

        moduleCache[normalizedRequest] = newMod.exports;
        return newMod.exports;
    }

    return originalLoad.apply(this, arguments);
};
```

Alias → VFS path mapping: `@tsix/x` → `/lib/x.ts`; `@common/y` → `/lib/common/y.ts`. Relative imports from inside the framework are also normalized (`./z` in `@tsix_...` → `@tsix/z`).

2. `restrictHostAPI` — locking the doors (full)

```
const restrictHostAPI = (appName: string) => {
    const forbidden = (msg: string = "Security Violation: Direct Host API access is forbidden in TSIX Sandbox.") => {
        throw new Error(msg);
    };

    const isPrivileged = appName.toLowerCase().includes("server") ||
        appName.toLowerCase().includes("daemon") ||
        appName.toLowerCase().includes("dome") ||
        appName.toLowerCase().includes("tbuild") ||
        appName.toLowerCase().includes("vfs") ||
        appName.toLowerCase().includes("mysqld");
    const allowedModules = ["http", "ws", "path", "fs", "url", "esbuild", "crypto", "os", "bcryptjs", "mysql2", "mysql2/promise"];

    const privilegedRequire = (mod: string) => {
        // Framework aliases are ALWAYS allowed, even in sandbox
        if (mod.startsWith("@tsix/") || mod.startsWith("@common/") || mod.includes("/lib/") || mod.includes("/common/")) {
            return hijackRequire(mod);
```

```
        }

        if (allowedModules.includes(mod)) {
            return hostRequire!(mod);
        }
        forbidden(`Security Violation: Module '${mod}' is not in the privileged allow-list.`);
    };

    // Sembunyikan require jika ada (tergantung module loader)
    if (typeof require !== "undefined") {
        (global as any).require = isPrivileged ? privilegedRequire : (mod: string) => {
            // Even in sandbox, framework cores MUST be accessible
            if (mod.startsWith("@tsix/") || mod.startsWith("@common/") || mod.includes("/lib/") || mod.includes("/common/")) {
                return hijackRequire(mod);
            }
            forbidden();
        };
    }

    // Batasi akses process yang sensitif
    const p = (global as any).process;
    if (p) {
        p.exit = forbidden;
        p.kill = forbidden;
    }
};
```

Note: `allowedModules` has **11 entries** — including both `mysql2` **and** `mysql2/promise`. The privileged check uses substring matching: `server`, `daemon`, `dome`, `tbuild`, `vfs`, `mysqld` (checked with `toLowerCase()`).

3. Direct Memory Execution — `_compile` the app (DME path)

```
// Module filename HARUS physical path untuk require() nemu node_modules
const moduleFilename = path!.join(process.cwd(), appName + ".js");
// stackBkfsPath = BKFS path untuk stack trace (/opt/test/gui-test.js)
const stackBkfsPath = (data as any).stackBkfsPath;
const stackFilename = stackBkfsPath
    ? stackBkfsPath.replace(/\.ts$/, '.js')
    : moduleFilename;
// sourcefile untuk esbuild sourcemap — cukup nama file aja (tanpa path)
const sourceFileName = (stackBkfsPath || moduleFilename).split(/[\\/]/).pop()!.replace(/\.js$/, '.ts');

// Jika content adalah TypeScript, transpile dulu ke JavaScript
if (isTypeScript) {
    const esbuild = hostRequire!("esbuild");
    const result = esbuild.transformSync(content, {
        loader: "ts",
        format: "cjs",
        target: "node18",
        sourcemap: "inline",
        sourcefile: sourceFileName,
    });
    content = result.code;
}

const appModule = new Module(moduleFilename, module.parent);
appModule.filename = stackFilename;                // __filename = BKFS path
appModule.paths = Module._nodeModulePaths(path!.dirname(moduleFilename));

// _compile dengan stackFilename agar stack trace nunjuk BKFS path
(appModule as any)._compile(content, stackFilename);
```

4. Fatal error handling

```
process.on("unhandledRejection", (reason) => {
    const msg = reason instanceof Error ? reason.message : String(reason);
    console.error("[Worker Fatal] Unhandled Rejection:", msg);
    trySendErrorToParent(msg);
    realExit(1);
```

```
});

process.on("uncaughtException", (err) => {
    const msg = err instanceof Error ? err.message : String(err);
    console.error("[Worker Fatal] Uncaught Exception:", msg);
    trySendErrorToParent(msg);
    realExit(1);
});
```

`trySendErrorToParent` uses `global._tsixLib`: `lib.getParentPid()` then `lib.shell.send(parentPid, { type: "GUI_WINDOW_ERROR", ... })` — so the error shows up in an Asteracea window.

Exercises / Practice

1. Read `src/userland/WorkerEntry.ts` — find the line where `restrictHostAPI(appName)` is called. Note: it is called after AppClass is found, before `new AppClass()`.
2. Write an app that calls `process.exit(0)` — observe the "Security Violation" error. Compare with `lib.shell.exit(0)` (the correct way).
3. Write an app that does `require("fs")` — compare the error between a normal app vs an app named `my-daemon` (privileged).
4. Write an app named `evil-daemon` that does `require("mysql2")` — prove the substring heuristic can be bypassed. Then explain why this is not actually dangerous (the kernel still guards syscalls).
5. Explain why the `@tsix/*` framework must always be accessible, even in the sandbox.

References

- `wiki/Keamanan-dan-Sandboxing.md` — full sandbox model
- `wiki/course/00-overview.en.md` §6 — Worker Thread & Sandboxing (summary)
- `src/userland/WorkerEntry.ts` — bootloader + sandbox (main source code)
- `src/kernel/Scheduler.ts` — `spawnWorker()` (workerData + vfsCache)
- `src/common/IPCTypes.ts` — `WorkerInitData`

Module 11 — done. Continue to Module 12 — Module Resolution & DME.

Module 12 **Module Resolution & DME** Process Isolation

RFC-TSIX-EDU-002 | The twelfth module of the TSIX curriculum. It uncovers one of the most "magical" parts of TSIX: the kernel pre-compiles `/lib` into memory, workers execute from memory with no disk hits, and the filename-alias trick keeps relative imports working.

> DME (Direct Memory Execution) is how TSIX loads the `@tsix/*` and `@common/*` frameworks **without touching the filesystem** at runtime. The kernel pre-compiles `/lib` once at boot, the cache is copied to each worker via `workerData`, then `WorkerEntry` compiles modules from an in-memory string with a **fake filename** (dummy filename) so relative imports keep working.

Learning Objectives

- [ ] Explain the DME (Direct Memory Execution) concept
- [ ] Explain the `Module._load` hijack and the `@tsix/*` / `@common/*` alias rewrite
- [ ] Explain the dummy-filename trick for relative `./x` imports
- [ ] Explain the dual identity filename (physical vs BKFS)
- [ ] Explain `moduleCache` and why the framework feels instant
- [ ] Explain the difference between the TS-transpile and JS-Direct paths

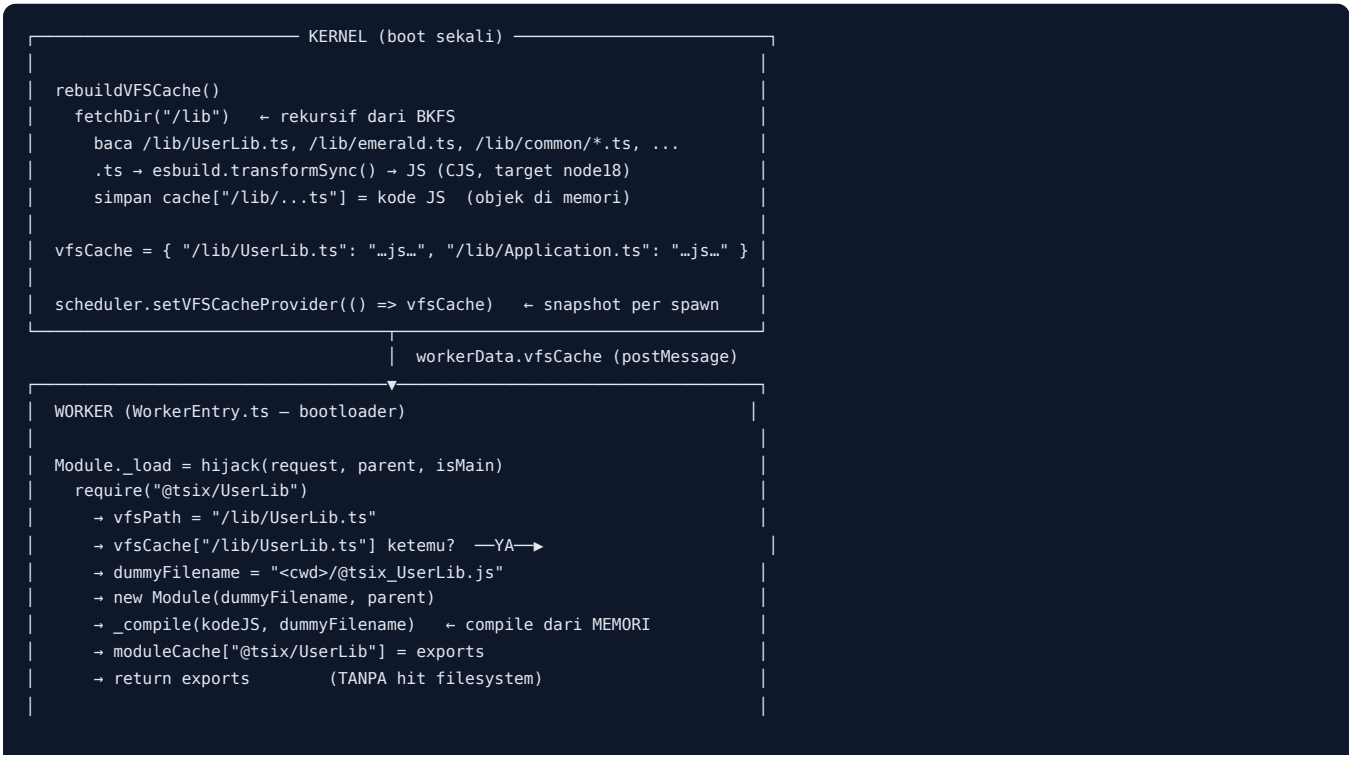
---

Core Concepts

What is Direct Memory Execution

DME answers two problems:

1. **Performance** — a normal `require()` resolves and reads files from disk every time a module is loaded. For a framework used by every app, this is expensive. DME replaces disk reads with a **lookup in a JavaScript object** (`vfsCache`).
2. **Alias consistency** — every app must see the **exact same** framework (`@tsix/*` / `@common/*`) with the same behavior, regardless of the VFS backend.



```
│  require("@tsix/UserLib") berikutnya → moduleCache (cached)              │
└──────────────────────────────────────────────────────────────────────────┘
```

Two cache layers: `vfsCache` vs `moduleCache`

Cache	Where	Key	Content	Lifecycle
`vfsCache`	Kernel (`Kernel.ts`)	BKFS path (`/lib/*.ts`)	pre-compiled JS code	once per boot
`moduleCache`	per worker (`WorkerEntry.ts`)	alias (`@tsix/UserLib`)	module `exports` object	once per worker

`vfsCache` only contains **code strings**. `moduleCache` contains **ready-to-use exports objects** — created by the worker the first time it loads a module. A framework file is pre-compiled once (by the kernel), but it is `_compile`d and cached once **per worker**.

Alias and path mapping

Alias	Rewritten to	vfsPath (vfsCache key)
`@tsix/UserLib`	—	`/lib/UserLib.ts`
`@tsix/emerald`	—	`/lib/emerald.ts`
`@common/GUITypes`	—	`/lib/common/GUITypes.ts`
`./UserLib` (parent `@tsix_*`)	`@tsix/UserLib`	`/lib/UserLib.ts`
`./GUITypes` (parent `@common_*`)	`@common/GUITypes`	`/lib/common/GUITypes.ts`

> [!IMPORTANT] The `@tsix/X` → `/lib/X.ts` mapping always appends the `.ts` extension. That is why framework files in BKFS **must** be `.ts` — a convention enforced by `vfs-bootstrap`. The kernel also caches `.js` / `.json` files under `/lib`, but the DME `@tsix/*` path only targets `.ts`.

Why this matters

1. **Performance: zero FS hits per require.** The framework is loaded once at boot, then executed from memory in every worker. Without DME, every app that does `import "@tsix/UserLib"` would have to read and re-compile the file from VFS — expensive for GUIs/daemons that must start fast.
2. **Alias consistency.** The relative rewrite `./x` → `@tsix/x` / `@common/x` ensures that internal framework code (e.g. `Application` using `UserLib`) always loads the **same instance** from the cache — no duplication of physical vs VFS paths.
3. **Single source of truth.** The kernel is the only one that reads `/lib` from BKFS. Workers have no direct access to the framework disk — this reinforces the sandbox boundary (see Module 11).

> [!TIP] DME is one of the **three core principles** of TSIX (syscall = the only door, everything-is-a-file, direct memory execution). See `wiki/course/00-overview.md` §1 and §6.2.

Two app execution paths

Path	Condition	Method
DME (VFS)	`appPath` empty, `appContent` present	`_compile` from a string; `.ts` files transpiled by esbuild in the worker
Physical (host)	`appPath` filled in	`hostRequire(finalAppPath)` — normal Node.js path, `-r esbuild-register`

Type detection: `isTypeScript = !(appPath || appName || "").toLowerCase().endsWith(".js")`.

Flow / How it works

1. **Kernel boot** (`Kernel.initializeSubsystems`) calls `rebuildVFSCache()`.
2. `rebuildVFSCache()` runs recursively (`fetchDir`) from `/lib` in BKFS; `.ts` files are transpiled by esbuild → `vfsCache`.
3. The kernel connects the scheduler: `scheduler.setVFSCacheProvider(() => vfsCache)`.
4. On `spawnWorker()`, the scheduler copies the cache: `vfsCache: this.vfsCacheProvider()` goes into `workerData`.
5. `new Worker(workerPath, { workerData, execArgv })` — a new thread with a cache snapshot.
6. `WorkerEntry.ts` (module scope) saves `originalLoad = Module._load`, then replaces `Module._load` with the hijacked version.
7. `main()` loads UserLib: `hijackRequire("@tsix/UserLib")` → through `Module._load` → vfsCache → `_compile` from memory.
8. The app is loaded via DME (if the content comes from VFS) or physically (if `appPath`).
9. The sandbox is enabled (`restrictHostAPI`) — see Module 11 — before the app runs.

Source Code

File	Role
`src/kernel/Kernel.ts`	`rebuildVFSCache()` — pre-compile `/lib` → `vfsCache`
`src/kernel/Scheduler.ts`	`setVFSCacheProvider()` + `spawnWorker()` — send the cache via `workerData`
`src/userland/WorkerEntry.ts`	Hijack `Module._load`, DME, dummy filename, dual identity

Snippets (code level)

1. Kernel: pre-compile `/lib` into memory

`rebuildVFSCache()` in `src/kernel/Kernel.ts` — copied exactly from the source:

```
private rebuildVFSCache() {
    this.bootLogStart(
      "VFS: Pre-compiling framework libraries (Memory Cache)... ",
    );
    try {
      const esbuild = require("esbuild");
      const cache: Record<string, string> = {};
      const fetchDir = (dir: string) => {
        if (!this.bkfs!.exists(dir)) return;
        const items = this.bkfs!.ls(dir);
        for (const item of items) {
          const p = `${dir}/${item.name}`;
          if (item.type === "DIRECTORY") {
            fetchDir(p);
          } else if (
            item.type === "FILE" &&
            (item.name.endsWith(".ts") ||
              item.name.endsWith(".js") ||
              item.name.endsWith(".json"))
          ) {
            let content = this.bkfs!.read(p);
            if (!content) continue;

            let code = content;

            // Transpile TS to JS for framework files
            if (item.name.endsWith(".ts")) {
              try {
                const result = esbuild.transformSync(code, {
                  loader: "ts",
                  format: "cjs",
                  target: "node18",
                  sourcemap: "inline",
                });
                code = result.code;
```

```
              } catch (err: any) {
                this.logger.error(`Failed to pre-compile ${p}: ${err.message}`);
              }
            }

            cache[p] = code;
          }
        }
      };
      fetchDir("/lib");
      this.vfsCache = cache;
      this.bootLogEnd(true, "OK");
    } catch (e: any) {
      this.bootLogEnd(false, `Error: ${e.message}`);
    }
  }
```

Things to note:

- `fetchDir` is **recursive** — it includes subfolders like `/lib/common/`.
- `.ts` files are transpiled via `esbuild.transformSync` (loader `ts`, format `cjs`, target `node18`). `.js` / `.json` files are copied as-is.
- The cache key = **BKFS path** (`/lib/UserLib.ts`), the value = **a JS code string**.

2. Kernel → Scheduler: cache channel

```
// Kernel.ts — saat boot
this.rebuildVFSCache();
this.scheduler.setVFSCacheProvider(() => {
  return this.vfsCache;
});

// Scheduler.ts — spawnWorker()
const workerData: WorkerInitData = {
    pid: pcb.pid,
    appName: options.appName || pcb.name,
    args: options.args || [],
    appPath: options.appPath,
    stackBkfsPath: options.stackBkfsPath,
    appContent: options.appContent,
    env: pcb.env,
    vfsCache: this.vfsCacheProvider ? this.vfsCacheProvider() : {}
};
```

`setVFSCacheProvider()` stores a *callback*, not a snapshot. Each `spawnWorker()` calls that callback to fetch the **current** cache and copy it into `workerData`.

3. Worker: hijack `Module._load` + dummy filename + `_compile`

In `src/userland/WorkerEntry.ts` — copied exactly from the source:

```
const originalLoad = Module._load;
const vfsCache = (workerData as any).vfsCache || {};
const moduleCache: Record<string, any> = {};

Module._load = function (request: string, parent: any, isMain: boolean) {
    let normalizedRequest = request;

    // Resolve relative paths
    if (request.startsWith(".")) {
        if (request.includes("/common/")) {
            normalizedRequest = "@common/" + request.split("/common/")[1];
        } else if (request.includes("/lib/")) {
            normalizedRequest = "@tsix/" + request.split("/lib/")[1];
        } else if (parent && parent.filename) {
            const basename = path!.basename(parent.filename);
            if (basename.startsWith("@tsix_") && request.startsWith("./")) {
                normalizedRequest = "@tsix/" + request.substring(2);
```

```
            } else if (basename.startsWith("@common_") && request.startsWith("./")) {
                normalizedRequest = "@common/" + request.substring(2);
            }
        }
    }

    // Cached Module
    if (moduleCache[normalizedRequest]) return moduleCache[normalizedRequest];

    // Resolusi Memory Framework (VFS)
    let vfsPath = null;
    if (normalizedRequest.startsWith("@tsix/")) {
        vfsPath = "/lib/" + normalizedRequest.substring(6) + ".ts";
    } else if (normalizedRequest.startsWith("@common/")) {
        vfsPath = "/lib/common/" + normalizedRequest.substring(8) + ".ts";
    }

    if (vfsPath && vfsCache[vfsPath]) {
        const content = vfsCache[vfsPath];
        const dummyFilename = path!.join(process.cwd(), normalizedRequest.replace("/", "_") + ".js");

        const newMod = new Module(dummyFilename, parent);
        newMod.filename = dummyFilename;
        newMod.paths = Module._nodeModulePaths(process.cwd());

        // Framework modules are now pre-compiled in Kernel.
        // Direct execution for maximum performance.
        (newMod as any)._compile(content, dummyFilename);

        moduleCache[normalizedRequest] = newMod.exports;
        return newMod.exports;
    }

    return originalLoad.apply(this, arguments);
};
```

The hijack steps, one by one:

1. **Normalize the request** — relative `./x` / `.../lib/x` paths are rewritten to aliases.
2. **Check `moduleCache`** — if already loaded, return exports directly (no re-compilation).
3. **Map alias → `vfsPath`** (`@tsix/X` → `/lib/X.ts`, `@common/X` → `/lib/common/X.ts`).
4. **Check `vfsCache`** — if found, take the JS code from memory.
5. **Dummy filename** — `<cwd>/@tsix_X.js` (see below).
6. **`_compile(content, dummyFilename)`** — Node.js compiles the module from a string, **without reading a file**.
7. **Save to `moduleCache`** and return `exports`.
8. If not in cache → **fallback** to `originalLoad.apply(...)` (normal Node.js path).

4. Dummy filename trick — framework module identity

The kernel only stores code in `vfsCache` keyed by the BKFS path (`/lib/UserLib.ts`). The `@tsix_*.js` names are **created when loaded in the worker**, not during pre-compilation:

```
const dummyFilename = path!.join(process.cwd(), normalizedRequest.replace("/", "_") + ".js");
```

Alias	`replace("/", "_")`	dummyFilename (basename)
`@tsix/UserLib`	`@tsix_UserLib`	`<cwd>/@tsix_UserLib.js`
`@tsix/Application`	`@tsix_Application`	`<cwd>/@tsix_Application.js`
`@common/GUITypes`	`@common_GUITypes`	`<cwd>/@common_GUITypes.js`

Why are the `@tsix_` / `@common_` prefixes important? Because relative rewrites inside the framework happen **based on the parent's `basename`**:

```
const basename = path!.basename(parent.filename);
if (basename.startsWith("@tsix_") && request.startsWith("./")) {
```

```
    normalizedRequest = "@tsix/" + request.substring(2);
}
```

This means that if `Application.ts` (loaded with filename `<cwd>/@tsix_Application.js`) does `import x from "./UserLib"`, then `./UserLib` → `@tsix/UserLib` → `/lib/UserLib.ts`. The alias cycle stays consistent across the whole framework. This is what makes `@tsix/*` look like an instant package.

> [!NOTE] So the "dual identity" of a framework module is: **content identity** = the BKFS path (`/lib/UserLib.ts`, the `vfsCache` key), while the **filename identity** seen by Node = the dummy path (`<cwd>/@tsix_UserLib.js`).

5. Dual identity filename for applications

When the worker loads an **app** via DME (the block in `main()`), two paths are created:

```
let content = (data as any).appContent;
const isTypeScript = !(appPath || appName || "").toLowerCase().endsWith(".js");
// Module filename HARUS physical path untuk require() nemu node_modules
const moduleFilename = path!.join(process.cwd(), appName + ".js");
// Stack filename = BKFS path biar stack trace bener (/opt/test/gui-test.js)
// stackBkfsPath = BKFS path untuk stack trace (/opt/test/gui-test.js)
const stackBkfsPath = (data as any).stackBkfsPath;
const stackFilename = stackBkfsPath
    ? stackBkfsPath.replace(/\.ts$/, '.js')
    : moduleFilename;
// sourcefile untuk esbuild sourcemap — cukup nama file aja (tanpa path)
const sourceFileName = (stackBkfsPath || moduleFilename).split(/[\\/]/).pop()!.replace(/\.js$/, '.ts');

// Jika content adalah TypeScript, transpile dulu ke JavaScript
if (isTypeScript) {
    try {
        const esbuild = hostRequire!("esbuild");
        const result = esbuild.transformSync(content, {
            loader: "ts",
            format: "cjs",
            target: "node18",
            sourcemap: "inline",
            sourcefile: sourceFileName,
        });
        content = result.code;
    } catch (transpileErr: any) {
        console.error(`[Worker ${pid}] TS Transpile Error: ${transpileErr.message}`);
        throw transpileErr;
    }
}

// Create a new module instance with physical path (for node_modules resolution)
const appModule = new Module(moduleFilename, module.parent);
appModule.filename = stackFilename;  // __filename shows BKFS path
appModule.paths = Module._nodeModulePaths(path!.dirname(moduleFilename));

// _compile dengan stackFilename agar stack trace nunjuk BKFS path
(appModule as any)._compile(content, stackFilename);

AppClass = appModule.exports.main || appModule.exports.Main || appModule.exports.default || appModule.exports;
```

So:

- **`moduleFilename`** (physical: `<cwd>/<appName>.js`) is used for `new Module(...)` and `Module._nodeModulePaths(...)` — so internal `require("node_modules/...")` calls in the app can resolve.
- **`stackFilename`** (BKFS: `/opt/test/gui-test.js`) is set as `appModule.filename` and passed to `_compile` — so `__filename` and the **stack trace** point to the correct location in VFS.

This is the answer to exercise #4: a single module has **two identities** because of two different needs (module resolution vs error reporting).

Exercises / Practice

1. Read `rebuildVFSCache()` in `src/kernel/Kernel.ts`. Trace how `/lib` is read recursively, how `.ts` files are transpiled, and how the results are stored in `vfsCache`. What are the cache's keys and values?
2. Add a log in `Module._load` (WorkerEntry.ts) when `vfsPath` is found. Run an app that does `import ... from "@tsix/UserLib"`. Observe that there is no filesystem hit — and that the second log comes from `moduleCache` (not a re-`_compile`).
3. Create an app that throws an error. Inspect its stack trace — does it point to the BKFS path (`/opt/...`) or the physical path (`<cwd>/...`)? Match it against `stackBkfsPath`.
4. Add a log for `basename(parent.filename)` in the relative-rewrite branch. Look at the `@tsix_*` value when internal framework code uses `./x`. Explain its relationship to the dummy filename.
5. Explain why the two filename identities are needed (require vs stack trace), and why the `@tsix_` / `@common_` prefixes determine the relative rewrite.
6. Compare the TS-transpile vs JS-Direct paths: when is `-r esbuild-register` used in `execArgv`, and when is esbuild called manually in the worker? (See Module 11.)

References

- `wiki/course/00-overview.md` §1, §6.2 — DME as a core principle & bootstrap summary
- `wiki/course/11-worker-thread-sandbox.md` — WorkerEntry as bootloader + `restrictHostAPI` (Module 11)
- `wiki/course/05-syscall-ipc.md` — syscall & IPC: why workers communicate through syscall, not direct `require` (Module 05)
- `src/kernel/Kernel.ts` — `rebuildVFSCache()`, `setVFSCacheProvider()`
- `src/kernel/Scheduler.ts` — `spawnWorker()`, `workerData.vfsCache`
- `src/userland/WorkerEntry.ts` — hijack `Module._load`, DME, dummy filename
- `src/common/IPCTypes.ts` — the `WorkerInitData` type (the `workerData` contract)

Module 12 — done. Part IV is complete. Continue to Module 13 — TTY & Virtual Console.

Module 13 **TTY & Virtual Console** Human Interaction

RFC-TSIX-EDU-002 | Thirteenth module of the TSIX curriculum. Understand the TTY as a full PTY emulation: screen buffer, ANSI subset, raw/cooked mode, and master-side ioctl.

> The TSIX TTY is not just a "text console" — it is a **full PTY emulation**. A remote application (e.g. `pixelterm`) can control the TTY from the master side through ioctl `0x2001` / `0x2002` (inject input / read output).

Learning Objectives

- [ ] Explain the role of TTYManager and the allocation of consoles 1–32
- [ ] Explain the TTY components (buffer, cursor, ANSI parser)
- [ ] Explain the important TTYDevice ioctls (clear, raw, 0x2001, 0x2002, winsize)
- [ ] Explain TTY allocation to processes via `ttyId` in EXEC
- [ ] Explain keyboard behavior (Ctrl+C, Alt+F1-6)

Core Concepts

Components

Component	File	Role
TTYManager	`src/kernel/tty/TTYManager.ts`	Manages 32 virtual consoles (`Map<number, TTY>`), `activeId`, TTY switching, onSwitch/onInterrupt callbacks
TTY	`src/kernel/tty/TTY.ts`	Buffer `char[y][x]`, cursor, ANSI subset parser, raw/cooked mode, re-rendering
TTYDevice	`src/kernel/devices/TTYDevice.ts`	`/dev/ttyN` — wraps a TTY into an `IDevice` (read/write/ioctl)

A TTY is used as three devices at once for a process "attached" to a console: FD 0 (stdin), FD 1 (stdout), and FD 2 (stderr) all point to the same `TTYDevice`. `init` (PID 1) is created with `fds: [tty1, tty1, tty1]` and `ttyId: 1`.

Buffer & Modes

- **Screen buffer**: `charBuffer: string[][]` holds the characters, `attrBuffer: string[][]` holds the ANSI styles (e.g. `"31;1"`). Both are indexed `[y][x]` (row first, then column). Default size is 80×24, but at boot `TTYManager` uses the host terminal size (`process.stdout.rows` / `columns`).
- **Cooked mode** (default): input is line-buffered. The TTY performs echo, `Enter` moves the line into `inputLines[]`, `Backspace` deletes the last character, and Ctrl+C produces a signal (it does not enter the buffer).
- **Raw mode** (`setRawMode(true)`): each character goes straight into `inputBuffer[]` without echo and without line editing. Suitable for line editors / shells.
- **Output buffer**: every `write()` also records data into `outputBuffer` — this is what the master reads via ioctl `0x2002` (`getOutput()` also empties it).

TTYDevice ioctl

cmd	Name	Meaning	Implementation
`1`	`CLEAR_SCREEN`	clear the TTY screen	`tty.clear()`; if active, reset the host stdout (`\x1bc`)
`2`	`SWITCH_TTY`	switch console	return `-1` — handled in `Syscalls.ts` via `Kernel.ttyManager.switch()`
`10`	`SET_RAW_MODE`	set raw/cooked mode	`tty.setRawMode(!!arg)`
`0x2001`	`INJECT_INPUT`	master types into the slave stdin	`tty.pushInput(arg)`
`0x2002`	`READ_OUTPUT`	master reads the slave stdout	`tty.getOutput()` (and empty the buffer)

cmd	Name	Meaning	Implementation
`4`	`TIOCGWINSZ`	window size	`{ lines: tty.height, columns: tty.width }`

> [!NOTE] The codes `0x2001` / `0x2002` are also used by `MySQLDevice` (`MYSQL_IOCTL_CONNECT` / `DISCONNECT`). The meaning of an ioctl depends on the driver — in `TTYDevice` both are PTY input injection / output reading.

ANSI Subset Handled

The TSIX ANSI parser is simple: a sequence `ESC [ ...` is parsed until a command letter (`cmd`) is found, then the arguments are split by `;`. The following subset is implemented in `TTY.handleANSI()`:

Sequence	Name	Behavior
`ESC[nA`	Cursor Up	move cursor up `n` rows (clamp 0)
`ESC[nB`	Cursor Down	move cursor down `n` rows (clamp `height-1`)
`ESC[nC`	Cursor Forward	move cursor forward `n` columns (clamp `width-1`)
`ESC[nD`	Cursor Backward	move cursor backward `n` columns (clamp 0)
`ESC[r;cH` / `ESC[r;cf`	Cursor Position	move cursor to absolute position (1-based)
`ESC[...m`	SGR (color/attribute)	store into `currentAttr` (default `"0"`)
`ESC[2J`	Erase Display	`clear()` the whole screen
`ESC[0J`	Erase Display (partial)	erase from cursor to bottom
`ESC[K`	Erase Line	erase from cursor to end of line
`ESC[nS`	Scroll Up (SU)	shift rows up by `n` rows
`ESC[nT`	Scroll Down (SD)	shift rows down by `n` rows
`ESC[nL`	Insert Line (IL)	insert `n` blank lines at the cursor row
`ESC[nM`	Delete Line (DL)	delete `n` lines at the cursor row
`ESC[s`	Save Cursor (DECSC)	save cursor position
`ESC[u`	Restore Cursor (DECRC)	restore cursor position

In addition, `write()` handles basic character controls: `\n` (new line), `\r` (`cursorX = 0`), `\b` (one column back). `render()` redraws the whole screen using absolute cursor positioning per row (`ESC[r;cH` + `ESC[2K`) so that it is not corrupted by host terminal wrapping.

TTY Allocation to Processes

Unlike PortManager (networking), TTY allocation to processes is done **via `ttyId` in the `EXEC` syscall** — the kernel overrides FD 0/1/2 to `tty{n}`:

- The `ttyId` argument is valid only for the range **1–12**.
- If valid, `stdin` / `stdout` / `stderr` are set to the device `tty{ttyId}`.
- If `ttyId` is not given, the process **inherits** `ttyId` from its parent (`pcb.ttyId`).
- `createProcess()` stores `ttyId` in the PCB; `runInit()` creates PID 1 with `ttyId: 1`, then calls `setForegroundProcess(pid, 1)`.

The scheduler maintains the mapping `ttyForegroundPids: Map<ttyId, pid>` — one foreground process per TTY. When a process **detaches** (daemonizes) or **exits**, it is removed from the foreground mapping and its `ttyId` is cleared. Example usage in userland: `init` (PID 1) on `tty1`, then `login` is spawned for other TTYs with a target `ttyId`.

Keyboard

- **Hotkey** Alt+F1–F6 → `TTYManager.switch` (also Alt+1–6 for macOS).

- **Ctrl+C** → `SIGINT` to the foreground process of the active TTY.
- **Console switch** → `SIGWINCH` to the foreground process of the newly active TTY.
- **Host window resize** → update `LINES` / `COLUMNS` in the env of all processes + `SIGWINCH` broadcast + `TTYManager.handleResize()`.

Flow / How It Works

Input: keystroke → foreground process

```
Host keyboard (process.stdin)
        │  data mentah (utf8)
        ▼
KeyboardDevice (data handler kernel)
        │  hotkey? (Alt+F1..6) ──► TTYManager.switch(...) → STOP
        ▼
Kernel: ttyManager.getActiveTTY().pushInput(data)
        │
        ▼
TTY.pushInput(data)
   ├─ cooked : echo + lineBuffer → Enter → inputLines[]
   └─ raw    : tiap char → inputBuffer[]
        │
        ▼
Proses foreground → syscall READ(fd 0) → TTYDevice.read()
        │  raw   → tty.read() → inputBuffer.shift()
        └  cooked → tty.read() → inputLines.shift()
```

Output: application → TTY → render

```
Aplikasi → syscall WRITE/PRINT(fd 1) → TTYDevice.write(data)
        │
        ▼
TTY.write(data)
   1. outputBuffer += data          (untuk master ioctl 0x2002)
   2. onWrite(data)                 (master remote ikut menerima)
   3. parse char: \n \r \b, ESC[... (subset ANSI), karakter biasa
        │  karakter biasa → putChar() → charBuffer[y][x] + attrBuffer[y][x]
        ▼
TTY aktif? → tty.onWrite → process.stdout.write(data)   (layar host)
Master?    → syscall READ(pid) → ioctl 0x2002 → getOutput() (pixelterm)
```

Boot wiring (Kernel.boot)

```
new TTYManager(32)                    → 32× TTY (ukuran host)
  └─ 32× TTYDevice "tty1..tty32"      → /dev/ttyN (isActive = aktifId === N)
devices: stdin=KeyboardDevice, fb0/stdout/stderr → tty1, null, tty1..32
kbd.setDataHandler       → getActiveTTY().pushInput(data)
kbd.setHotkeyHandler     → handleKeyboardHotkey (Alt+F1..6)
kbd.setInterruptHandler  → handleHostInterrupt   (Ctrl+C)
ttyManager.setOnSwitchCallback      → SIGWINCH ke foreground TTY baru
ttyManager.setOnInterruptCallback   → SIGINT ke foreground TTY
runInit() → createProcess("init", fds:[tty1×3], ttyId:1)
          → setForegroundProcess(pid, 1)
```

Console switch (Alt+F2)

```
Host kirim seq "\x1bOQ" (atau "\x1b[1;3Q")
  → KeyboardDevice.onHotkey → handleKeyboardHotkey(seq) → true (stop)
  → ttyManager.switch(2)
      1. activeId = 2
      2. reset host terminal: "\x1bc\x1b[3J"
      3. banner "TSIX VIRTUAL CONSOLE [ TTY 2 ]" 500ms (jika visualIdentity ada)
```

```
    4. render() buffer TTY2 → process.stdout
    5. onSwitchCallback(2) → SIGWINCH ke foreground PID TTY2
```

Ctrl+C → SIGINT

```
Host/kbd deteksi "\u0003"
  → pushInput: cooked → onInterrupt() (tidak masuk buffer); raw → onInterrupt() + tetap masuk buffer
  → TTYManager.onInterruptCallback(activeId)
  → Kernel → SIGINT ke foreground process TTY aktif
  → Scheduler.sendInterruptSignal(ttyId) → kill(fgPid, 2)
      → event "signal" SIGINT → grace 100ms → worker.terminate() jika tak ditangani
```

Source Code

File	Role
`src/kernel/tty/TTY.ts`	TTY emulation: buffer, cursor, ANSI subset, raw/cooked, render
`src/kernel/tty/TTYManager.ts`	32 console allocation, `activeId`, switch, callbacks
`src/kernel/devices/TTYDevice.ts`	Device wrapper `/dev/ttyN` + ioctl
`src/kernel/devices/KeyboardDevice.ts`	Keyboard driver + hotkey/interrupt detection
`src/kernel/Scheduler.ts`	Foreground per TTY (`ttyForegroundPids`), SIGINT/SIGWINCH
`src/kernel/Syscalls.ts`	EXEC `ttyId`, ioctl 0x2001/0x2002, switch/resize
`src/kernel/Kernel.ts`	Boot wiring, hotkey handler, resize listener, `runInit`

Snippet (code level)

TTY input buffer — push/pop (`src/kernel/tty/TTY.ts`)

```
public pushInput(data: string) {
    if (this.rawMode) {
        // RAW MODE: langsung ke inputBuffer (tiap karakter)
        for (const char of data) {
            if (char === "\u0003") {          // Ctrl+C
                if (this.onInterrupt) {
                    this.onInterrupt();
                }
                // Tetap push untuk app yang mau menangani sendiri
                this.inputBuffer.push(char);
            } else {
                this.inputBuffer.push(char);
            }
        }
    } else {
        // COOKED MODE: echo + backspace + line buffering
        for (const char of data) {
            const code = char.charCodeAt(0);
            if (char === "\u0003") {          // Ctrl+C → interrupt, jangan masuk buffer
                if (this.onInterrupt) this.onInterrupt();
                continue;
            }
            if (char === "\r" || char === "\n") {
                this.inputLines.push(this.lineBuffer + "\n");
                this.lineBuffer = "";
                this.write("\n");             // echo newline
                continue;
            }
            if (code === 127 || code === 8) { // Backspace
                if (this.lineBuffer.length > 0) {
                    this.lineBuffer = this.lineBuffer.slice(0, -1);
```

```
                    this.write("\b \b");      // echo hapus visual
                }
                continue;
            }
            this.lineBuffer += char;
            // ... filter escape sequence untuk echo (NORMAL/ESC/CSI),
            //     lalu echo hanya char printable (0x20-0x7E) + \n \r \t
        }
    }
}

public read(): string | null {
    if (this.rawMode) {
        if (this.inputBuffer.length === 0) return null;
        return this.inputBuffer.shift() || null;
    } else {
        if (this.inputLines.length === 0) return null;
        return this.inputLines.shift() || null;
    }
}

public setRawMode(enabled: boolean) {
    this.logger.debug(`setRawMode(${enabled}) - previously ${this.rawMode}`);
    this.rawMode = enabled;
    // Apps yang pindah ke raw mode biasanya tidak ingin input cooked tertunda
    if (enabled) {
        this.lineBuffer = "";
    }
}
```

ioctl TTYDevice (`src/kernel/devices/TTYDevice.ts`)

```
public ioctl(cmd: number, arg: any): any {
    if (cmd === 1) { // 1 = CLEAR_SCREEN
        this.tty.clear();
        if (this.isActive()) {
            process.stdout.write("\x1bc");
        }
        return 0;
    }
    if (cmd === 2) { // 2 = SWITCH_TTY
        return -1; // Handled in Syscalls.ts via Kernel.ttyManager
    }
    if (cmd === 10) { // 10 = SET_RAW_MODE
        this.tty.setRawMode(!!arg);
        return 0;
    }
    if (cmd === 0x2001) { // INJECT_INPUT (Master typing into slave stdin)
        this.tty.pushInput(arg as string);
        return true;
    }
    if (cmd === 0x2002) { // READ_OUTPUT (Master reading slave stdout)
        return this.tty.getOutput();
    }
    if (cmd === 4) { // 4 = TIOCGWINSZ (Get Window Size)
        return { lines: this.tty.height, columns: this.tty.width };
    }
    return -1;
}
```

TTYManager.switch (`src/kernel/tty/TTYManager.ts`)

```
public async switch(id: number, forceRedraw: boolean = false) {
    if (!this.ttys.has(id)) return;
    if (id === this.activeId && !forceRedraw) return;

    this.logger.info(`Switching from TTY${this.activeId} to TTY${id}`);
    this.activeId = id;

    // Reset total terminal host agar tidak ada sisa dari TTY lama
```

```
    process.stdout.write("\x1bc\x1b[3J");

    // Banner visual terpusat (500ms) jika ada visual identity
    if (this.visualIdentity && !forceRedraw) {
        const rows = process.stdout.rows || 24;
        const cols = process.stdout.columns || 80;
        const text = `TSIX VIRTUAL CONSOLE [ TTY ${id} ]`;
        const barLines = this.visualIdentity.split("\n");
        const bannerWidth = 32;
        const bannerHeight = 1 + barLines.length;
        const startRow = Math.max(1, Math.floor((rows - bannerHeight) / 2));
        const startCol = Math.max(1, Math.floor((cols - bannerWidth) / 2));
        process.stdout.write("\x1b[?25l");
        process.stdout.write(`\x1b[${startRow};${startCol}H\x1b[97m  ${text}\x1b[0m`);
        barLines.forEach((line, index) => {
            process.stdout.write(`\x1b[${startRow + 1 + index};${startCol}H${line}`);
        });
        await new Promise(resolve => setTimeout(resolve, 500));
        process.stdout.write("\x1bc\x1b[3J\x1b[?25h");
    }

    const content = this.getActiveTTY().render();
    process.stdout.write(content);

    // Notify foreground process di TTY yang baru aktif
    if (this.onSwitchCallback) {
        this.onSwitchCallback(id);
    }
}
```

`ttyId` allocation in EXEC (`src/kernel/Syscalls.ts`)

```
// --- TTY REDIRECTION SUPPORT ---
// Jika TTY tertentu diminta, override I/O default ke TTY itu
if (ttyId !== undefined && ttyId >= 1 && ttyId <= 12) {
    const targetTtyDevice = this.kernel.devices[`tty${ttyId}`];
    if (targetTtyDevice) {
        stdinDevice = targetTtyDevice;
        stdoutDevice = targetTtyDevice;
        stderrDevice = targetTtyDevice;
    }
} else {
    // Warisan normal jika tidak ada TTY khusus
    if (stdoutFd !== undefined && pcb.fdTable[stdoutFd]) {
        stdoutDevice = pcb.fdTable[stdoutFd]!.device;
    }
    if (stdinFd !== undefined && pcb.fdTable[stdinFd]) {
        stdinDevice = pcb.fdTable[stdinFd]!.device;
    }
}

const newPcb = this.scheduler.createProcess(binaryName, {
    fds: [stdinDevice, stdoutDevice, stderrDevice],
    // ... appName, args, env, uid/gid, dst
    ttyId: ttyId !== undefined ? ttyId : pcb.ttyId, // target TTY atau warisi dari parent
    ppid: pid,
});
```

Hotkey Alt+F1..6 (`src/kernel/Kernel.ts`)

```
private handleKeyboardHotkey(seq: string): boolean {
    const hotkeys: Record<string, number> = {
        // Alt+F1..F6 (Xterm, iTerm2, Terminal.app)
        "\x1b\x1bOP": 1,   "\x1b[1;3P": 1,   "\x1b[11;3~": 1,
        "\x1b\x1bOQ": 2,   "\x1b[1;3Q": 2,   "\x1b[12;3~": 2,
        "\x1b\x1bOR": 3,   "\x1b[1;3R": 3,   "\x1b[13;3~": 3,
        "\x1b\x1bOS": 4,   "\x1b[1;3S": 4,   "\x1b[14;3~": 4,
        "\x1b\x1b[15~": 5, "\x1b[15;3~": 5,  "\x1b[1;3;15~": 5,
        "\x1b\x1b[17~": 6, "\x1b[17;3~": 6,  "\x1b[1;3;17~": 6,
        // Alt+1..6 (macOS saat Option bertindak sebagai Meta)
```

```
        "\x1b1": 1, "\x1b2": 2, "\x1b3": 3,
        "\x1b4": 4, "\x1b5": 5, "\x1b6": 6,
    };
    if (hotkeys[seq]) {
        // FIRE AND FORGET: jangan await agar tidak memblokir input keyboard
        this.ttyManager?.switch(hotkeys[seq]);
        return true; // handled
    }
    return false;
}
```

Callback & foreground wiring (`src/kernel/Kernel.ts` + `src/kernel/Scheduler.ts`)

```
// Kernel: daftarkan callback TTY
this.ttyManager.setOnSwitchCallback((ttyId: number) => {
    const fgPid = this.scheduler?.getForegroundProcess(ttyId);
    if (fgPid) {
        this.logger.debug(`Sending SIGWINCH to PID ${fgPid} (TTY${ttyId} activated)`);
        this.scheduler?.sendEvent(fgPid, "signal", "SIGWINCH");
    }
});
this.ttyManager.setOnInterruptCallback((ttyId: number) => {
    const fgPid = this.scheduler?.getForegroundProcess(ttyId);
    if (fgPid) {
        this.logger.info(`Sending SIGINT to PID ${fgPid} (TTY${ttyId} Ctrl+C)`);
        this.scheduler?.sendEvent(fgPid, "signal", "SIGINT");
    }
});

// Scheduler: satu foreground process per TTY
public setForegroundProcess(pid: number | null, ttyId?: number) {
    if (pid !== null && !ttyId) {
        const pcb = this.getProcess(pid);
        ttyId = pcb?.ttyId || 1;
    }
    const targetTty = ttyId || 1;
    if (pid === null) {
        this.ttyForegroundPids.delete(targetTty);
    } else {
        this.ttyForegroundPids.set(targetTty, pid);
    }
}
```

Exercises / Practice

1. Read `src/kernel/tty/TTY.ts` — explain the buffer structure `char[y][x]` + `attr[y][x]` and the ANSI subset in `handleANSI()`.
2. Run `pixelterm` (if available) — observe how it uses ioctl `0x2001` (inject input) and `0x2002` (read output) from the master side.
3. From another TTY, press Alt+F2 — observe the console switch and the visual banner.
4. Read `src/kernel/devices/TTYDevice.ts` — explain each ioctl (`1`, `2`, `10`, `0x2001`, `0x2002`, `4`).
5. Trace the `EXEC` syscall in `src/kernel/Syscalls.ts` — what is different when `ttyId` is set vs not? When does a process inherit the parent's `ttyId`?
6. Press Ctrl+C on a foreground process — explain the signal path from `pushInput` → `onInterrupt` → `SIGINT`.

References

- `wiki/Kernel-dan-Scheduler.md` §TTY
- `wiki/course/00-overview.en.md` §8
- `src/kernel/tty/TTY.ts`, `src/kernel/tty/TTYManager.ts`, `src/kernel/devices/TTYDevice.ts`
- `src/kernel/Scheduler.ts` (§ `ttyForegroundPids`, `setForegroundProcess`, `sendInterruptSignal`)
- `src/kernel/Syscalls.ts` (syscall `EXEC`, `SEND`/`READ` via ioctl `0x2001`/`0x2002`, `SWITCH_TTY`, `WINSIZE`)

Module 13 — complete. Continue to Module 14 — Userland.

Module 14 **Userland: init / login / shell / apps** Human Interaction

RFC-TSIX-EDU-002 | Fourteenth module of the TSIX curriculum. Understand the application layer: PID 1 (init), login, shell, and the two styles of writing TSIX applications.

> All userland runs in a Worker Thread (Ring 4) — including PID 1. There is no inittab: the init logic is **hardcoded** in `init.ts`.

Learning Objectives

- [ ] Explain the role of init (PID 1) without inittab
- [ ] Explain the login flow: passwd+shadow(bcrypt) → setgroups→setgid→setuid
- [ ] Explain the two application styles: `IProgram` vs `Program()` wrapper
- [ ] Explain the "error = window" concept (`std.error`)
- [ ] Explain the role of `monitorProcess` (respawn login)

Core Concepts

Init (PID 1)

There is no `/etc/inittab` — all init decisions are hardcoded in `src/mirror/bin/init.ts` (class `Init`). Execution order:

1. **Enforce setuid** — `chmod("/bin/passwd.js", 2541)` and `chmod("/bin/sudo.js", 2541)`. The value `2541` (decimal) equals `0o4755` (setuid root + rwxr-xr-x). The target is `.js`, not `.ts`, because the runtime executes the `.js` sidecar.
2. **RSA identity** — check `/etc/keys/rsa/id_rsa.pub`. If it is missing, `SecurityAgent.generateKeyPair()` creates a new key (RSA 2048-bit) and saves it to `/etc/keys/rsa/`. If it exists, the key is verified. The SHA256 fingerprint is taken via `shell.getFingerprint()`, and the identity color bar (`SecurityAgent.generateVisualIdentity`) is sent to the TTY via `std.ioctl(1, 33, colorBar)` — ioctl 33 = `SET_VISUAL_IDENTITY`.
3. **Exec `rc.local`** — run `/etc/rc.local.js` then `waitpid`. Exit code `0` = successful startup; anything else is logged as a warning.
4. **Spawn login TTY2–6** — `for (let i = 2; i <= 6; i++) spawnLogin(i)`. Each login process is *monitored* by `monitorProcess` (respawn + 1s delay anti-crash-loop).
5. **Spawn login TTY1** (foreground) — done after the banner and fingerprint.
6. **Loop forever** — `while (true) { await new Promise(r => setTimeout(r, 10000)); }`. Init must not exit; if PID 1 dies, the kernel calls `process.exit(exitCode)` (keepAlive, `1` = reboot).

Login

`src/mirror/bin/login.ts` — one login process per TTY, spawned by init. Core flow:

1. Read `/etc/passwd` → find the user entry → take `uid` (field 3), `gid` (field 4), `home` (field 6), `shell` (field 7).
2. Read `/etc/shadow` → take the bcrypt hash (field 2) → `bcrypt.compareSync(password, hash)`.
3. Read `/etc/group` → collect *supplementary groups* (groups that contain the username, besides the primary gid).
4. `setgroups` → `setgid` → `setuid` — **mandatory POSIX order**. The GID must be changed before the UID; after the UID becomes non-root, the process loses the privilege to change the GID.
5. Set env `USER`/`HOME`, `chdir(home)`, then `exec` the shell — the shell is taken from `/etc/passwd` field 7 (usually `tsh`).

WM Login (Asteracea) — GUI mode

TTY login (`login.ts`) is re-spawned as a fresh root process per TTY, so it can always read `/etc/shadow` & `setuid`. In contrast, **the WM (Asteracea) is a SINGLE process that is both login manager and desktop session**:

- The WM starts as root (spawned by init), then after the first login it **permanently drops privileges** to the logged-in user.

- On logout → login again (e.g. as root), the WM is already non-root → **two problems**:
  1. It can no longer read `/etc/shadow` (0640 root) → password verification fails.
  2. The kernel rejects `setgroups` / `setgid` / `setuid` for non-root → cannot switch users.
- Solution (see Snippet):
  1. **Verify via a SetUID-root helper** — `login.js --verify <user> <pass> <file>` (login.js is already SetUID root) → result is written to a temp file (`OK` / `FAIL:...`) and read back by the WM. A file channel is used because the child's *exit code* is unreliable (WorkerEntry always finishes with `exit(0)`).
  2. **Saved UID (kernel)** — `pcb.suid`. When the WM drops from root, the kernel keeps `suid=0`; on re-login the WM may `setuid(0)` to **restore** root, then drop to the target user. This is the Unix Saved UID mechanism (`seteuid` / `setresuid`).
  3. **Credential order in the WM** — if the WM is not root yet → `setuid(0)` first (restore), then `setgroups` → `setgid` → `setuid(target user)` (POSIX order for dropping).

> [!NOTE] **Saved UID safety** — `suid` defaults to the process's own UID in `createProcess`, so **regular apps do NOT inherit saved root** and cannot escalate. Only a process that dropped from root (i.e. the WM) keeps the "ticket back to root".

Shell (tsh)

`src/mirror/bin/tsh.ts` — interactive shell, written in the `IProgram` style (`export class main implements IProgram`). Main features:

- Prompt `user@hostname:cwd#/$` customizable via the `PROMPT_FORMAT` env (`&username`, `&hostname`, `&cwd`, `&usertype`).
- Line editor: raw mode, history (up/down arrows), Tab completion, Ctrl+U, Home/End.
- Before running a command, the shell switches to cooked mode so foreground commands can receive SIGINT.
- `tsh <username>` — delegates to `/bin/login.js` (login is SetUID root, so it can authenticate & switch users).
- Resize (SIGWINCH / `RESIZE` event) updates the `LINES` / `COLUMNS` env without overwriting the foreground application's screen.

Two application styles

There are two styles for writing TSIX applications:

1. Class `IProgram` (mostly legacy) — export class `main` that implements `IProgram`, with the method `execute(lib, args)`. All APIs are accessed through the `lib` parameter:

```
export class main implements IProgram {
    async execute(lib: OSContext, args: string[]) {
        // ...
    }
}
```

Examples: `ls`, `cat`, `ps`, `kill`, `tsh`.

2. `Program()` wrapper + proxy singletons (new) — import the `std`, `fs`, `shell`, `net`, `db` singletons from `@tsix/Application`, then wrap the main function with `Program(fn)`:

```
import { Program, std, fs, shell, net } from "@tsix/Application";

export default Program(async (args) => {
    // ...
});
```

Examples: `esp-send`, `dome`, `iot-dashboard`.

> [!NOTE] **How `Program()` works** — `Program(fn)` returns an anonymous class that `implements IProgram`. The `execute` method stores the `OSContext` into `global._tsixOsc`, calls `fn(args)`, and if `fn` throws an error, calls

> `std.error(error.stack, appName)` then re-throws. The `std` / `fs` / `shell` / `net` / `db` singletons are `Proxy` objects that forward access to `_tsixLib` on the current thread. `os.pid` and `os.rand` are also available.

Comparison of the two styles:

Aspect	Class `IProgram` (legacy)	`Program()` wrapper (new)
Shape	Class `main` + method `execute()`	Anonymous function wrapped in `Program(fn)`
Signature	`execute(lib: OSContext, args: string[])`	`fn(args: string[])`
API access	Through the `lib` parameter (`lib.std`, `lib.fs`, ...)	Import proxy singletons (`std`, `fs`, `shell`, `net`, `db`)
Arguments	Second parameter	Single function parameter
Error handling	Manual (own try/catch)	Automatic → `std.error()` then re-throw
Implementation	`implements IProgram` directly	`Program()` returns a class `implements IProgram`
Examples	`ls`, `cat`, `ps`, `kill`, `tsh`	`esp-send`, `dome`, `iot-dashboard`

Error = "Window"

`std.error()` does 4 things:

1. Write to `/var/log/syslog`.
2. Find `fileHint` from the stack trace (skipping `UserLib`, `Application`, `emerald`).
3. Broadcast to the parent via IPC — message `{ type: "GUI_WINDOW_ERROR", wid, pid, file, error, context, timestamp }`.
4. Print to the TTY in red `[ERROR]`.

So when an application crashes, **an error window appears on the desktop** — not just text in the terminal.

Flow / How It Works

Boot: Kernel → init (PID 1)

```
Kernel (main.ts) — kernel.runInit()
  resolve /bin/init.js
  createProcess("init", fds:[tty1×3])   → PID 1
  setForegroundProcess(1, 1)

[init.ts — Worker Thread, Ring 4]
  1. Enforce setuid
       chmod /bin/passwd.js  2541   (0o4755)
       chmod /bin/sudo.js    2541
  2. RSA identity
       cek /etc/keys/rsa/id_rsa.pub
         ada       → verify + fingerprint
         tidak ada → generateKeyPair() → simpan id_rsa + id_rsa.pub
       fingerprint + color bar → ioctl(1, 33, colorBar)  (SET_VISUAL_IDENTITY)
  3. Exec /etc/rc.local.js + waitpid
  4. for (i = 2; i <= 6; i++) spawnLogin(i)
       exec /bin/login.js (ttyId=i)
       monitorProcess → respawn saat login mati (delay 1s anti-crash-loop)
  5. Banner ASCII + fingerprint color bar
  6. spawnLogin(1)   ← foreground TTY1
  7. while (true) { sleep 10s }   ← init tidak boleh exit
```

Login: TTY → shell

```
login.ts (satu proses per TTY)
  1. username = args[0] || readLine("Username: ")
  2. password = readPassword("Password:🔑")   ← masked, raw mode
  3. /etc/passwd → uid(3), gid(4), home(6), shell(7)
  4. /etc/shadow → hash bcrypt (field 2)
```

```
  5. bcrypt.compareSync(password, hash)
       gagal → "Login: Invalid username or password" → ulangi dari langkah 1
  6. MOTD: /etc/motd + frasa acak /etc/motd.json
  7. /etc/group → supplementary gids (grup yang memuat username)
  8. setgroups(gids) → setgid(gid) → setuid(uid)   ← urutan POSIX
  9. setenv USER / HOME → chdir(home)
  10. exec(shell dari /etc/passwd) → waitpid → break (satu sesi)
```

WM Login: GUI → desktop

```
asteracea.ts (single process = login manager + desktop)
  1. WM starts as root (spawned by init)
  2. Show GUI login screen
  3. Verify: spawn /bin/login.js --verify <user> <pass> <file>  ← SetUID root
       read result file: "OK" / "FAIL:..."
  4. /etc/passwd → uid(3), gid(4), home(6)   ← world-readable, read directly
  5. /etc/group → supplementary gids
  6. If the WM is not root yet (already logged in non-root) → setuid(0)  ← restore via Saved UID
  7. setgroups(gids) → setgid(gid) → setuid(uid)   ← POSIX order (drop)
  8. setenv USER / HOME → chdir(home)
  9. Show desktop — the WM BECOMES that user (taskbar, launcher, wallpaper)
  10. Logout → back to step 2 (WM stays alive; identity is reset via Saved UID)
```

Source Code

File	Role
`src/mirror/bin/init.ts`	PID 1 — hardcoded init (setuid, RSA, rc.local, spawn login)
`src/mirror/bin/login.ts`	passwd+shadow (bcrypt) authentication → setgroups/setgid/setuid → exec shell
`src/mirror/bin/tsh.ts`	Interactive shell (`IProgram` style)
`src/mirror/opt/asteracea/asteracea.ts`	WM + GUI login manager — verify via `login.js --verify`, re-elevate via Saved UID
`src/mirror/lib/UserLib.ts`	Framework: `std`, `fs`, `shell`, `net`, `db` (sub-library)
`src/mirror/lib/Application.ts`	`Program()` wrapper + proxy singleton
`src/mirror/lib/IProgram.ts`	`IProgram` & `OSContext` interfaces

Snippet (code level)

All snippets below are copied from the source — *code is the truth*.

Init — enforce setuid

```
// Runtime mengeksekusi sidecar .js (bukan source .ts),
// jadi chmod harus di .js. Nilai 2541 = 0o4755 (setuid root).
await lib.fs.chmod("/bin/passwd.js", 2541);
await lib.std.print(`${ok} [INIT] SetUID bit applied to: /bin/passwd.js\n`);
await lib.fs.chmod("/bin/sudo.js", 2541);
await lib.std.print(`${ok} [INIT] SetUID bit applied to: /bin/sudo.js\n`);
```

> `ok` is the green `[ OK ]` prefix defined at the start of `execute()` in `init.ts`.

Init — spawnLogin (TTY2-6)

```
const spawnLogin = async (ttyId: number) => {
    try {
        if (ttyId > 1)
            await lib.std.print(`${ok} Initializing session on TTY${ttyId}...\n`);
```

```
        const result = await lib.shell.exec("/bin/login.js", [], undefined, undefined, ttyId);
        if (result && result.pid) {
            terminals.set(ttyId, result.pid);
            await lib.std.log(`Login service spawned on TTY${ttyId} (PID ${result.pid})`, "init");
            // Kita nungguin di background (thread terpisah di Worker)
            this.monitorProcess(lib, ttyId, result.pid, spawnLogin);
        }
    } catch (e: any) {
        await lib.std.print(`Init: Error spawning login on TTY${ttyId}: ${e.message}\n`);
    }
};

// Start login on all TTYs (TTY2-6)
for (let i = 2; i <= 6; i++) {
    await spawnLogin(i);
}
```

Init — monitorProcess (respawn anti-crash-loop)

```
private async monitorProcess(
    lib: UserLib, ttyId: number, pid: number,
    respawn: (tty: number) => Promise<void>,
) {
    const exitCode = await lib.shell.waitpid(pid);
    await lib.std.print(
        `Init: Process on TTY${ttyId} (PID ${pid}) exited with code ${exitCode}. Respawning...\n`,
    );
    // Delay sedikit biar nggak spam kalau crash loop
    await new Promise(r => setTimeout(r, 1000));
    await respawn(ttyId);
}
```

Login — bcrypt verification

```
// 1. Cari user di /etc/passwd → uid, gid, home, shell
const parts = userEntry.split(":");
const uid = parseInt(parts[2]);
const gid = parseInt(parts[3]);
const home = parts[5];
const userShell = parts[6];   // shell dari /etc/passwd field 7

// 2. Ambil hash dari /etc/shadow → bcrypt.compareSync
const hash = shadowEntry.split(":")[1];
const match = bcrypt.compareSync(password, hash);
if (!match) {
    await lib.std.print("Login: Invalid username or password\n");
    continue;   // kembali ke loop → prompt ulang
}
```

Login — setgroups → setgid → setuid (POSIX order)

```
// 3.5. Resolve Supplementary Groups dari /etc/group
const supplementaryGids: number[] = [gid]; // dimulai dari primary gid
try {
    const groupContent = await lib.fs.readFile("/etc/group") || "";
    const groupLines = groupContent.split("\n")
        .map(l => l.trim()).filter(l => l.length > 0);
    for (const gLine of groupLines) {
        const gParts = gLine.split(":");
        const groupGid = parseInt(gParts[2]);
        const groupUsers = gParts[3] ? gParts[3].split(",") : [];
        if (groupUsers.includes(username.trim()) && groupGid !== gid) {
            supplementaryGids.push(groupGid);
        }
    }
} catch (e) {
    // Ignore group errors
}
```

```
// Set Identity — GID harus SEBELUM UID.
// Setelah UID menjadi non-root, proses kehilangan privilege untuk mengubah GID.
await lib.shell.setgroups(supplementaryGids);
await lib.shell.setgid(gid);
await lib.shell.setuid(uid);

// Set Env + exec shell
await lib.shell.setenv("USER", username.trim());
await lib.shell.setenv("HOME", home);
await lib.shell.chdir(home);
const result = await lib.shell.exec(userShell);   // shell dari /etc/passwd
if (result && result.pid) {
    await lib.shell.waitpid(result.pid);
}
```

WM Login — verify via login.js --verify (file channel)

```
// asteracea.ts — verify password via a SetUID-root helper.
// A non-root WM cannot read /etc/shadow (0640 root) → delegate to login.js.
const verifyOut = `/tmp/verify-${Date.now()}-${Math.random().toString(36).slice(2, 8)}.txt`;
let authOk = false;
const vp = await shell.exec("/bin/login.js", ["--verify", u, password, verifyOut]);
if (vp && vp.pid) {
    await shell.waitpid(vp.pid);                    // wait until done
    const res = (await fs.readFile(verifyOut)) || "";
    authOk = String(res).trim() === "OK";           // "OK" / "FAIL:..."
}
await fs.unlink(verifyOut);                         // cleanup temp file
```

```
// login.ts — helper side (runs as SetUID root, so it can read shadow)
if (args[0] === "--verify") {
    const vUser = (args[1] || "").trim();
    const vPass = args[2] || "";
    const vOut  = args[3] || "/tmp/verify-result.txt";
    let ok = false, errMsg = "";
    try {
        const shadow = (await lib.fs.readFile("/etc/shadow")) || "";
        const entry = shadow.split("\n").map(l => l.trim()).filter(l => l)
            .find(l => l.split(":")[0] === vUser);
        const hash = entry ? entry.split(":")[1] : "";
        ok = !!hash && bcrypt.compareSync(vPass, hash);
        if (!hash) errMsg = "account not found or no password";
    } catch (e) { errMsg = e.message || "verify error"; }
    await lib.fs.writeFile(vOut, ok ? "OK" : "FAIL:" + errMsg);
    return;   // done — WorkerEntry finishes the process
}
```

Saved UID (kernel) — setuid may restore to suid

```
// Syscalls.ts — SETUID with Saved UID (mirrors Unix setuid/seteuid)
if (this.isRoot(pcb)) {
    pcb.suid = pcb.uid;   // root may change UID freely; keep the way back (0 for root)
    pcb.uid = newUid;
    pcb.ruid = newUid;
} else if (newUid === pcb.suid) {
    pcb.uid = newUid;     // non-root may only RESTORE to its Saved UID (e.g. back to root)
    pcb.ruid = newUid;
} else {
    throw new Error("Permission Denied: Only root or root group members can change UID");
}

// Scheduler.ts — createProcess: suid defaults to the process's own UID
suid: options.suid ?? options.uid ?? 0,   // regular apps CANNOT escalate
```

App — `IProgram` style (legacy)

```
// greet.ts — gaya klasik: class main implements IProgram
import { IProgram, OSContext } from "@tsix/IProgram";

export class main implements IProgram {
    async execute(lib: OSContext, args: string[]): Promise<string | void> {
        const name = args[0] || "dunia";
        await lib.std.print(`Halo, ${name}!\n`);
        await lib.std.print(`CWD: ${await lib.shell.getcwd()}\n`);
    }
}
```

App — `Program()` wrapper style (new)

```
// halo.ts — gaya baru: Program() wrapper + proxy singleton
import { Program, std, os } from "@tsix/Application";

export default Program(async (args) => {
    const name = args[0] || "dunia";
    await std.print(`Halo, ${name}!\n`);
    await std.print(`PID: ${os.pid}\n`);
    return "Selesai.";
});
```

Exercises / Practice

1. Log in as `root`, then run `ps` — identify the login process per TTY (`/bin/login.js`).
2. From the root shell, run `tsh tamu` — observe the login delegation (setuid root) and the new shell with a different user.
3. Read `src/mirror/bin/login.ts` — trace the `setgroups` / `setgid` / `setuid` order and explain why this order is mandatory.
4. Read `src/mirror/bin/init.ts` — find `monitorProcess`; kill the login process on a TTY (`kill <pid>` from root) then observe init respawning it within about 1 second.
5. Write an app in the `IProgram` style and an app in the `Program()` style — compare their structures (see Snippet).
6. Create an app that calls `std.error()` — observe the error window appearing on the desktop (`GUI_WINDOW_ERROR` message).

References

- `wiki/Memulai.md`, `wiki/Perintah-Sistem.md`, `wiki/Panduan-Developer.md`
- `wiki/course/00-overview.en.md` §9 (Userland: init, shell, applications)
- `src/mirror/bin/init.ts`, `src/mirror/bin/login.ts`, `src/mirror/bin/tsh.ts`
- `src/mirror/lib/UserLib.ts`, `src/mirror/lib/Application.ts`, `src/mirror/lib/IProgram.ts`

Module 14 — done. Part V complete. Continue to Module 15 — Networking MQTNL.

Module 15 **Networking MQTNL** Networking

RFC-TSIX-EDU-002 | Fifteenth module of the TSIX curriculum. Understand MQTNL (MQTT Network Layer) — the TSIX networking protocol that uses MQTT pub/sub as the wire, instead of IP/routing.

> Instead of TCP/IP, MQTNL uses **MQTT pub/sub as the wire**. Address = **node name** (string), port = **application endpoint** (PortManager). Connections are connectionless/UDP-like — there is no real listen/accept, only polling emulation.

Learning Objectives

- [ ] Explain the concept of address = node name, port = application endpoint
- [ ] Explain the socket flow: bind → sendto → recvfrom
- [ ] Explain the role of PortManager and releasePortsByPid
- [ ] Explain why there is no listen/accept (polling emulation)
- [ ] List the networking syscalls (30–34)

Core Concepts

Model: MQTT pub/sub is the "wire"

MQTNL does not use TCP/IP. There is no IP address, no routing, no stateful connection. Instead, TSIX uses **MQTT pub/sub as the transport medium (wire)**:

- **Every node** connects to the **same MQTT broker**.
- **Address = node name** (string): `"tsix"`, `"tsix-node-2"`, `"esp32S3"`. This name is unique per device.
- **Topic =** `mqtnl@1.0/<dstAddress>` (JSON protocol) / `mqtnl@1.1/<dstAddress>` (Binary protocol). The topic content is an **MQTNL packet** (header + payload).
- **All nodes subscribe to** `mqtnl@1.x/#`, then **filter** packets by `dstAddress` (or `"*"` for broadcast). Packets for other nodes are dropped.

Sending is fire-and-forget without a session — that is why this model is **connectionless / UDP-like**.

Address & Port

Concept	Value	TCP/IP analog
Address	Node name (string)	IP address
Port	Application endpoint (0–65535)	TCP/UDP port
Wire	MQTT pub/sub topic	IP packet
Connection	Connectionless (fire-and-forget)	UDP

Each `sendto()` creates a self-contained packet. The packet header carries the destination address & port, so no connection needs to be kept.

Three Main Components

1. **SimpleMQTNLDriver** — *network interface* (`/dev/smqtnl0`, `/dev/smqtnl1`). Keeps the connection to the MQTT broker, publishes outbound packets, receives & filters inbound packets, 32KB fragmentation, reassembly, and decryption. Each instance has its own `localAddress` (node name).
2. **SocketDevice** — the abstraction of one socket: inbound data buffer + waiter list. Created per `SOCKET` syscall and stored in the FD table. It is passive — inbound data is `push`ed by the driver, and the application reads it via `recvfrom`.

3. **PortManager** — keeper of virtual ports 0–65535. `allocatePort()` for explicit bind, `allocateRandomPort(10000-20000)` for `bind(0)`, `releasePortsByPid()` when a process exits.

Per-port handler

When an application `bind(port)`s, the kernel calls `driver.registerHandler(port, cb)`. This handler routes inbound data to the socket owned by the application:

```
bind(port)
  → PortManager.allocatePort(port, pid)
  → socket.setPort(port); socket.driver = targetDriver
  → driver.registerHandler(port, (data) => socket.push(data))
```

When a packet arrives, the driver calls the handler on `dstPort`. This is **per-port dispatch**: one driver serves many ports at once.

Socket flow (summary)

```
App → socket() → bind(port) → driver.registerHandler(port)
  → sendto(addr, port, data) → driver.send → publish topic
  → MQTT broker → semua node subscribe 'mqtnl@1.x/#'
  → filter dstAddress → reassembly → decrypt → socket.push()
  → recvfrom() → buffer.shift()
```

Syscall

```
SOCKET=30, BIND=31, SENDTO=32, RECVFROM=33, NETSTAT=34
```

There is no `listen/accept` at the syscall level. UserLib emulates it: `listen()` = `socket()` + `bind()`; `accept()` = polling `recv()` until the first packet arrives.

Flow / How It Works

Sending flow from **App A** on node `"tsix"` (port N) to **App B** on node `"esp32S3"` (port M) via the MQTT broker:

Detailed steps:

1. **SOCKET** — App A and App B call `socket()`. The kernel creates a new `SocketDevice` and inserts it into the FD table (`context: "socket"`).
2. **BIND** — App A calls `bind(fd, N)`. The kernel determines the target driver (default `smqtnl0`), reserves port N via `PortManager.allocatePort(N, pid)`, then `registerHandler(N, cb)` routes data to socket A.

3. **SENDTO** — App A calls `sendto(fd, "esp32S3", M, data)`. The kernel calls `driver.send("esp32S3", M, data, FLAG_DATA, N)`.
4. **Fragmentation** — `send()` splits the payload into 32KB chunks when needed, then wraps each chunk in a packet with a 9-field header.
5. **Publish** — the driver publishes each chunk to the topic `mqtnl@1.0/esp32S3` (`qos: 0`, `retain: false`).
6. **Broker & filter** — all nodes subscribed to `mqtnl@1.x/#` receive the packet. Each driver filters: a packet is processed only when `dstAddress` equals its `localAddress` (or `"*"`). Other packets are dropped. Packets sent by the node itself (src = local) are also ignored.
7. **Reassembly & decryption** — the destination driver reassembles the chunks (session per `srcAddr:srcPort → dstAddr:dstPort`, TTL 30s), then decrypts the payload via `SecurityAgent` (the port can be upgraded to ChaCha20-Poly1305 via ioctl `0x1001`).
8. **Dispatch** — the driver calls the handler on `dstPort` → `socket.push({ src, port, localPort, data, isBinary, ts })` → data enters the socket buffer.
9. **RECVFROM** — App B calls `recv(fd)`. The kernel reads the buffer (`read()` = `buffer.shift()`). If empty, the kernel waits event-driven (`waitForData`) until data is pushed.

> [!NOTE] **Local loopback.** If `dstAddress` equals the sender node's own address (or `"localhost"`), the packet does not go out to the broker — the driver forwards it directly to the local driver instance (`SimpleMQTNLDriver.findLocal()`), similar to `localhost` on a real OS.

Source Code

File	Role
`src/kernel/devices/SimpleMQTNLDriver.ts`	MQTNL network interface
`src/kernel/devices/SocketDevice.ts`	Socket = device (everything is a file)
`src/kernel/PortManager.ts`	Virtual port allocation
`src/mirror/lib/NetworkLib.ts`	`lib.net` API (legacy, OSContext-based)
`src/mirror/lib/UserLib.ts`	Inline `lib.net` (new, `this.dispatch`)
`src/kernel/Syscalls.ts`	SOCKET–NETSTAT syscall implementation (30–34)
`src/sysconfig.json`	Default network interface configuration

> [!WARNING] **Dual NetworkLib.** There are two `lib.net` implementations: inline in `UserLib.ts` (new) and `NetworkLib.ts` (legacy, OSContext-based). Both reach the same syscalls.

Default network interfaces (`sysconfig.json`)

At boot, the kernel reads `cfg.network.interfaces` and creates one `SimpleMQTNLDriver` per entry (`Kernel.ts` → "Initialize Network Interfaces"). Each interface has a **deviceName** (`/dev` name), an **address** (node name), and a **broker**:

deviceName	address (node name)	broker	defaultPort
`smqtnl0`	`tsix`	`mqtt://192.168.1.204`	1883
`smqtnl1`	`tsix-node-2`	`mqtt://192.168.1.204`	1883

`cfg.network.defaultDevice` = `smqtnl0`. If `bind()` is called without an `address` argument, the kernel uses this default interface.

Snippet (code level)

All snippets below are **copied from the source** — "code is truth". Trimmed parts are marked `// ...`.

NetworkLib — usage from the app side (`UserLib.ts`)

`lib.net` is a `NetworkLib` instance on `UserLib` (new apps) and on `NetworkLib.ts` (legacy apps based on `OSContext`). A socket is returned as an **fd** (number), not an object:

```
export class NetworkLib {
  constructor(
    private dispatch: (code: SyscallCode, args: any) => Promise<any>,
  ) { }

  public async socket(): Promise<number> {
    return await this.dispatch(SyscallCode.SOCKET, null);
  }

  public async bind(
    fd: number,
    port: number,
    address?: string,
  ): Promise<boolean> {
    return await this.dispatch(SyscallCode.BIND, { fd, port, address });
  }

  // listen() = socket + bind — emulasi, tidak ada syscall LISTEN
  public async listen(port: number): Promise<number> {
    const fd = await this.socket();
    const ok = await this.bind(fd, port);
    return ok ? fd : -1;
  }

  // accept() = polling recv() sampai ada paket pertama
  public async accept(serverFd: number): Promise<any> {
    while (true) {
      const pkt = await this.recv(serverFd);
      if (pkt) {
        return {
          fd: serverFd,
          src: pkt.src,
          port: pkt.port,
          localPort: pkt.localPort || 0,
          firstPkt: pkt,
        };
      }
      await new Promise((r) => setTimeout(r, 100));
    }
  }

  public async sendto(
    fd: number,
    address: string,
    port: number,
    data: any,
    flag: number = 0,
    srcPort: number = 0,
  ): Promise<boolean> {
    return await this.dispatch(SyscallCode.SENDTO, {
      fd,
      address,
      port,
      data,
      flag,
      srcPort,
    });
  }

  public async recv(fd: number): Promise<any> {
    return await this.dispatch(SyscallCode.RECVFROM, fd);
  }
}
```

Usage example (server–client pattern):

```
// Server di node "tsix"
const fd = await lib.net.listen(8080);              // socket + bind(8080)
const client = await lib.net.accept(fd);            // polling recv → { src, port, firstPkt }
const data = await lib.net.recv(fd);                // baca data

// Client mengirim ke node "esp32S3", port 5000
await lib.net.sendto(fd, "esp32S3", 5000, JSON.stringify({ cmd: "ping" }));
```

PortManager — bind & release (`PortManager.ts`)

```
public allocatePort(port: number, pid?: number): boolean {
    if (port < 0 || port > 65535) return false;
    if (this.usedPorts.has(port)) return false;

    this.usedPorts.add(port);
    if (pid !== undefined) this.portOwner.set(port, pid);
    return true;
}

public allocateRandomPort(min: number = 10000, max: number = 20000): number | null {
    for (let i = 0; i < 100; i++) {
        const port = Math.floor(Math.random() * (max - min + 1)) + min;
        if (!this.usedPorts.has(port)) {
            this.usedPorts.add(port);
            return port;
        }
    }
    return null;
}

public releasePortsByPid(pid: number): void {
    const toRelease: number[] = [];
    this.portOwner.forEach((owner, port) => {
        if (owner === pid) toRelease.push(port);
    });
    for (const port of toRelease) {
        this.usedPorts.delete(port);
        this.portOwner.delete(port);
    }
}
```

> [!TIP] `releasePortsByPid()` is called when a process exits — a **safety net** if an application forgets to release its ports.

SocketDevice — buffer & handler dispatch (`SocketDevice.ts`)

```
public push(data: any) {
    this.buffer.push(data);
    // Bangunkan semua reader yang sedang nunggu data (event-driven)
    while (this.waiters.length > 0) {
        const w = this.waiters.shift()!;
        w();
    }
}

public read(): any {
    return this.buffer.shift() || null;
}

public async waitForData(timeoutMs: number): Promise<boolean> {
    if (this.buffer.length > 0) return true;
    return new Promise<boolean>((resolve) => {
        let timer: ReturnType<typeof setTimeout> | undefined;
        const onData = () => {
            if (timer) clearTimeout(timer);
```

```
            const idx = this.waiters.indexOf(onData);
            if (idx >= 0) this.waiters.splice(idx, 1);
            resolve(true);
        };
        timer = setTimeout(() => {
            const idx = this.waiters.indexOf(onData);
            if (idx >= 0) this.waiters.splice(idx, 1);
            resolve(false);
        }, timeoutMs);
        this.waiters.push(onData);
    });
}
```

SimpleMQTNLDriver — init & send (`SimpleMQTNLDriver.ts`)

```
public init(ctx: KContext): void {
    this.client = mqtt.connect(this.brokerUrl);

    this.client.on("connect", () => {
        for (const proto of this.protocols) {
            const prefix = proto.getTopicPrefix();
            this.client?.subscribe(`${prefix}#`);   // subscribe "mqtnl@1.0/#" & "mqtnl@1.1/#"
        }
    });

    this.client.on("message", (topic, message) => {
        this.handleIncomingMessage(topic, message);
    });
}
```

```
public async send(address: string, port: number, data: any, flag: PacketFlags = PacketFlags.FLAG_DATA, srcPort: number = 0) {
    // [LOOPBACK] Reserved alias "localhost" selalu menunjuk ke interface
    // pengirim sendiri — mirip localhost di OS sungguhan.
    if (address === "localhost") {
        address = this.localAddress;
    }

    // [LOOPBACK] Kalau tujuan adalah alamat lokal (milik node ini), kirim
    // langsung ke receiver tanpa keluar ke broker MQTT.
    const localTarget = SimpleMQTNLDriver.findLocal(address);

    // Cek koneksi hanya untuk paket yang benar-benar keluar ke broker.
    if (!localTarget && (!this.client || !this.client.connected)) return false;

    // ... (normalisasi payload, pemilihan protokol JSON/Binary, enkripsi,
    //      fragmentasi 32KB, header 9 field) — lihat file sumber

    const topic = `${prefix}${address}`;   // mis. "mqtnl@1.0/esp32S3"

    for (let i = 0; i < packetCount; i++) {
        // ... (bungkus packet, protocol.pack)
        if (localTarget) {
            localTarget.handleIncomingMessage(topic, payloadBuffer);   // LOOPBACK lokal
        } else {
            await new Promise((resolve) => {
                this.client!.publish(topic, payloadBuffer, { qos: 0, retain: false }, (err) => {
                    if (err) this.logger.error(`MQTT Publish error: ${err.message}`);
                    resolve(true);
                });
            });
        }
    }
    return true;
}
```

BIND — connecting a socket to the driver & port (`Syscalls.ts`)

```
case SyscallCode.BIND: {
    const { fd, port, address } = args as {
```

```
        fd: number;
        port: number;
        address?: string;
    };
    const entry = pcb.fdTable[fd];
    if (!entry || !(entry.device instanceof SocketDevice))
        throw new Error("Invalid Socket FD");

    const socket = entry.device as SocketDevice;
    if (socket.bound) throw new Error("Socket already bound");

    // ... pilih targetDriver (dari `address`, atau cfg.network.defaultDevice)

    const portMgr = this.kernel.getPortManager();
    let actualPort = port;

    if (port === 0) {
        const randomPort = portMgr.allocateRandomPort();
        if (randomPort === null) throw new Error("No random ports available");
        actualPort = randomPort;
    } else {
        if (!portMgr.allocatePort(port, pcb.pid))
            throw new Error(`Port ${port} already in use`);
    }

    socket.setPort(actualPort);
    socket.driver = targetDriver;

    // Register handler
    targetDriver.registerHandler(actualPort, (data) => {
        socket.push(data);
    });

    return true;
}
```

Exercises / Practice

1. Read `wiki/Networking-MQTNL.md` — the complete flow and topics.
2. Read `src/kernel/PortManager.ts` — understand `allocateRandomPort` and `releasePortsByPid`.
3. Run two nodes (e.g. `tsix` and `esp32S3` on an MQTT broker) — send a message between them.
4. Read `src/kernel/devices/SocketDevice.ts` — explain how a socket becomes an `IDevice`.
5. From an app, run `lib.net.netstat()` — compare the result with the interface table in `sysconfig.json`.

References

- `wiki/Networking-MQTNL.md` — full MQTNL documentation
- `wiki/course/00-overview.md` §7 — Networking MQTNL
- `src/kernel/devices/SimpleMQTNLDriver.ts` — network interface
- `src/kernel/devices/SocketDevice.ts` — socket = device
- `src/kernel/PortManager.ts` — virtual port allocation
- `src/kernel/Syscalls.ts` — SOCKET–NETSTAT syscall implementation (30–34)
- `src/sysconfig.json` — default network interfaces

Module 15 — done. Continue to Module 16 — Wire Protocol MQTNL.

Module 16 **Wire Protocol MQTNL** Networking

RFC-TSIX-EDU-002 | Sixteenth module of the TSIX curriculum. Understand the data format on top of MQTT: the JSON vs Binary dual protocol, magic byte detection, and fragmentation.

> MQTNL has **two wire protocols**: JSON v1.0 (readable) and Binary v1.1 (compact, without encryption — for byte-exact OTA). The driver detects the protocol from the **first magic byte** per srcAddress.

Learning Objectives

- [ ] Explain the `IMQTNLProtocol` contract
- [ ] Distinguish the JSON v1.0 vs Binary v1.1 protocol
- [ ] Explain magic byte detection
- [ ] Explain 32KB fragmentation and 30s TTL reassembly
- [ ] Explain why userland never touches the protocol directly

---

Core Concepts

The IMQTNLProtocol Contract

All MQTNL wire protocols implement a single contract. This interface is the "only door" between the driver and the wire format — adding a new protocol (e.g. v2) only requires implementing a new class, without changing the driver.

```
export interface IMQTNLProtocol {
    /** Nama protokol (misal: "JSON", "Binary") */
    getName(): string;
    /** Byte pertama penanda protokol dalam byte stream.
     *  JSON: '[' (0x5B). Binary: 'B' (0x42). */
    getMagicChars(): number[];
    /** MQTT Topic Prefix (misal: "mqtnl@1.0/", "mqtnl@1.1/") */
    getTopicPrefix(): string;
    /** Objek packet internal → Buffer/String untuk dikirim ke broker MQTT */
    pack(packet: any): Buffer | string;
    /** Raw data yang diterima → objek packet internal */
    unpack(data: Buffer | string): any;
}
```

Packet header (9 fields + payload)

The internal packet structure (the object that is `pack`ed/`unpack`ed) is identical for both protocols. The only difference is how fields are written to the wire.

Field	JSON (index array)	Binary (offset)	Type
`srcAddress`	0	S_ADDR_LEN + S_ADDR	string
`srcPort`	1	S_PORT	u16 LE
`dstAddress`	2	D_ADDR_LEN + D_ADDR	string
`dstPort`	3	D_PORT	u16 LE
`packetCount`	4	PACKET_COUNT	u16 LE
`packetIndex`	5	PACKET_INDEX	u16 LE
`dataSize`	6	DATA_SIZE	u32 LE
`packetHeaderFlag`	7	FLAG	u8

Field	JSON (index array)	Binary (offset)	Type
`forwarded`	8	FORWARDED	u8
`payload`	9	PAYLOAD (remaining buffer)	string / Buffer

> [!NOTE] `packetCount` and `packetIndex` are part of the header (not a separate feature). `packetCount` = the total number of chunks, `packetIndex` = the chunk number (0-based). `dataSize` = the **total** payload size before it is split (not the size per chunk).

Dual protocol JSON vs Binary

Aspect	JSON v1.0	Binary v1.1
Class	`MQTNLProtocolJSON`	`MQTNLProtocolBinary`
Topic prefix	`mqtnl@1.0/<addr>`	`mqtnl@1.1/<addr>`
Magic byte	`[` (0x5B)	`B` (0x42) + ver `0x01`
Wire format	JSON array (text, readable)	Compact binary buffer, byte-exact
Readability	easy for humans to read	hard to read (binary)
Size	larger (text overhead)	smaller & precise
Encryption	RSA handshake + ChaCha20-Poly1305	no encryption (plain, for OTA)
Used for	general communication (default legacy)	byte-exact OTA firmware transfer

 RSA handshake: SYN → SYN-ACK → ACK → encrypted DATA *Source:* *wiki/diagram/Networking-MQTNL-2.mmd*

Binary v1.1 frame format

Byte-by-byte diagram of the binary frame (N = length of `srcAddress`, M = length of `dstAddress`, L = payload length):

```
| 0x42 | 0x01 |  len  | <src addr>      | srcPort |  len  | <dst addr>      | dstPort |
| MAGIC| VER  | sAL=N | srcAddr (N b)   |  u16LE  | dAL=M | dstAddr (M b)   |  u16LE  |

| pktCount | pktIndex | dataSize | flag | fwd | <payload>        |
|  u16LE   |  u16LE   |  u32LE   |  u8  | u8  | payload (L b)    |
```

- Fixed header size = 18 bytes + N + M (`2 + 1 + N + 2 + 1 + M + 2 + 2 + 2 + 4 + 1 + 1`).
- Example: `srcAddress="tsix"` (N=4), `dstAddress="esp32S3"` (M=7) → header = 29 bytes.
- All fields are read in a deterministic order (no delimiter), so the address lengths must be sent (`S_ADDR_LEN` / `D_ADDR_LEN`).
- **There is no CRC/checksum** inside the frame. OTA integrity is protected by `dataSize` (u32) and the byte-exact raw path.

Magic byte detection

The driver does not guess the protocol from the topic. It reads the **first byte** of the MQTT payload and matches it against `getMagicChars()` of each registered protocol:

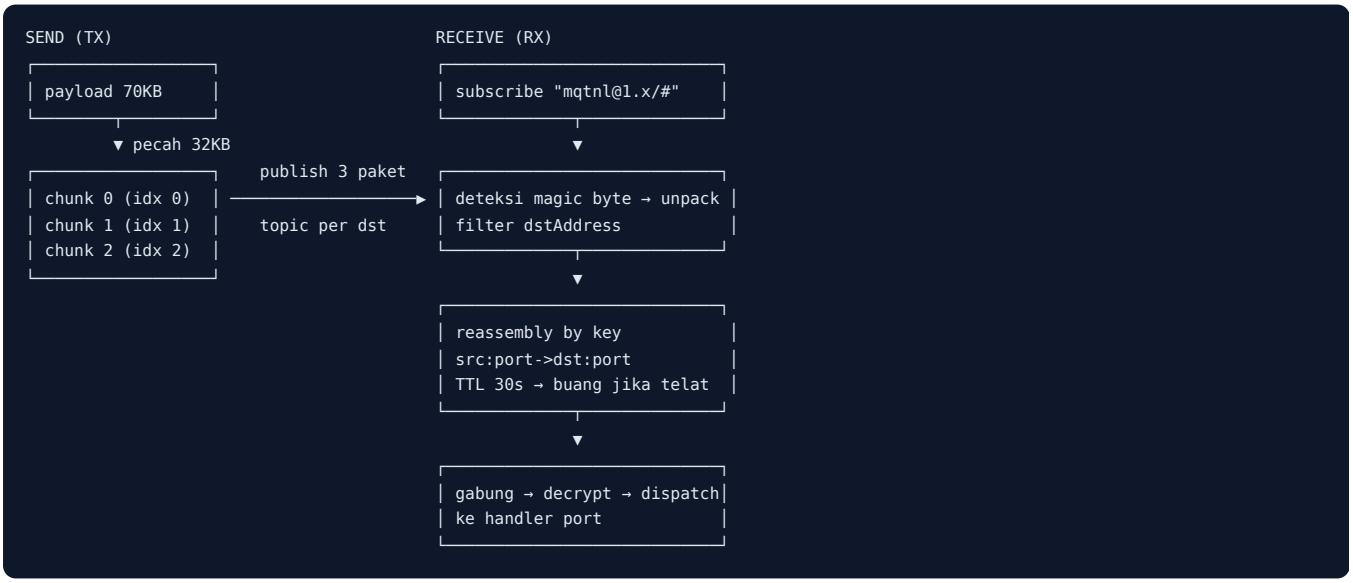
```
// Detect protocol by Magic Byte (di SimpleMQTNLDriver.handleIncomingMessage)
const magicByte = Buffer.isBuffer(message) ? message[0] : (message as string).charCodeAt(0);
const proto = this.protocols.find(p => p.getMagicChars().includes(magicByte));
if (!proto) {
    this.logger.error(`Protocol mismatch on ${topic}. Magic: 0x${message[0].toString(16)}`);
    return;
}
packet = proto.unpack(message);
```

```
// [MULTI-PROTO] Register protocol untuk pengirim ini (per srcAddress)
this.protocolRegistry.set(packet.header.srcAddress, proto);
```

The result is stored in `protocolRegistry` **per** `srcAddress`. When sending back to that address, the driver uses the same protocol (default: JSON).

Fragmentation & Reassembly

- **Fragmentation**: the payload is split into 32KB chunks (`packetSize = 32768`). Each chunk is sent as a separate MQTT packet with `packetCount` / `packetIndex` filled in.
- **Reassembly**: the receiver collects chunks in a session per `src:port->dst:port`. The session is complete when the number of unique chunks equals `packetCount`, then they are merged in order (`Buffer.concat` for Buffer, `join("")` for strings).
- **30 second TTL**: sessions that are incomplete within 30 seconds are dropped (`cleanupExpiredPackets`, run every 60 seconds).



OTA firmware streaming via MQTNL Binary v1.1 *Source:* *wiki/diagram/mqtnl-binary-ota-1.mmd*

Send path (TX) — `SimpleMQTNLDriver.send()`

1. Determine the protocol: `protocolRegistry.get(address) || activeProtocol` (default JSON).
2. `useRaw = (protocol.getName() === "Binary")` — if Binary, **skip encryption** (`securedPayload = payload`), so the byte length is exact for OTA.
3. If JSON, encrypt the payload via `SecurityAgent.securePacketOut()` (RSA handshake → ChaCha20-Poly1305).
4. **Fragmentation**: split `securedPayload` into 32KB chunks (`this.packetSize`).
5. For each chunk `i`: create a packet object (`packetCount = chunks.length`, `packetIndex = i`, `dataSize = securedPayload.length`), `protocol.pack(packet)` → Buffer, then `publish` to the topic `${prefix}${address}` (or local loopback if the address is this node).

Receive path (RX) — `SimpleMQTNLDriver.handleIncomingMessage()`

1. Detect the protocol from the **first magic byte** of the MQTT payload.
2. `proto.unpack(message)` → packet object. Store the protocol in `protocolRegistry` per `srcAddress`.
3. **Packet filter**: drop packets whose `dstAddress` is not a local address (except `"*"`).
4. **Reassembly**: store `packet.payload` in the session `src:port->dst:port`. If not complete yet → wait for the next chunk.
5. If complete: merge the chunks in order → **decrypt** (Binary: `securePacketInRaw` + plain fallback; JSON: `securePacketIn`).
6. **Dispatch**: send `{src, port, localPort, data, isBinary, ts}` to the destination port handler (or serve PING/broadcast).

Snippet (code level)

> All snippets below are **VERIFIED** — copied directly from the source.

JSON v1.0 — pack/unpack (`MQTNLProtocolJSON.ts`)

```
export class MQTNLProtocolJSON implements IMQTNLProtocol {
    getName(): string { return "JSON"; }
    getMagicChars(): number[] { return [0x5B]; } // '['
    getTopicPrefix(): string { return "mqtnl@1.0/"; }

    pack(packet: any): string {
        return JSON.stringify([
            packet.header.srcAddress, packet.header.srcPort,
            packet.header.dstAddress, packet.header.dstPort,
            packet.header.packetCount, packet.header.packetIndex,
            packet.header.dataSize, packet.header.packetHeaderFlag,
            packet.header.forwarded, packet.payload,
        ]);
    }

    unpack(data: Buffer | string): any {
        const str = Buffer.isBuffer(data) ? data.toString("utf8") : data;
        const packed = JSON.parse(str);
        return {
            header: {
                srcAddress: packed[0], srcPort: packed[1],
                dstAddress: packed[2], dstPort: packed[3],
                packetCount: packed[4], packetIndex: packed[5],
                dataSize: packed[6], packetHeaderFlag: packed[7],
                forwarded: packed[8],
            },
            payload: packed[9],
        };
    }
}
```

Binary v1.1 — pack/unpack (`MQTNLProtocolBinary.ts`)

```
export class MQTNLProtocolBinary implements IMQTNLProtocol {
    getName(): string { return "Binary"; }
    getMagicChars(): number[] { return [0x42]; } // 'B'
    getTopicPrefix(): string { return "mqtnl@1.1/"; }

    pack(packet: any): Buffer {
        const srcAddr = Buffer.from(packet.header.srcAddress || "", "utf8");
        const dstAddr = Buffer.from(packet.header.dstAddress || "", "utf8");
        const payload = Buffer.isBuffer(packet.payload)
            ? packet.payload
            : Buffer.from(packet.payload || "", "utf8");

        const headerSize = 2 + 1 + srcAddr.length + 2 + 1 + dstAddr.length
                         + 2 + 2 + 2 + 4 + 1 + 1;
        const buf = Buffer.alloc(headerSize + payload.length);
        let offset = 0;
        buf.writeUInt8(0x42, offset++);             // Magic 'B'
        buf.writeUInt8(0x01, offset++);             // Proto Ver 1
        buf.writeUInt8(srcAddr.length, offset++);
        srcAddr.copy(buf, offset); offset += srcAddr.length;
        buf.writeUInt16LE(packet.header.srcPort, offset); offset += 2;
        buf.writeUInt8(dstAddr.length, offset++);
        dstAddr.copy(buf, offset); offset += dstAddr.length;
        buf.writeUInt16LE(packet.header.dstPort, offset); offset += 2;
        buf.writeUInt16LE(packet.header.packetCount, offset); offset += 2;
        buf.writeUInt16LE(packet.header.packetIndex, offset); offset += 2;
        buf.writeUInt32LE(packet.header.dataSize, offset); offset += 4;
        buf.writeUInt8(packet.header.packetHeaderFlag, offset++);
```

```
        buf.writeUInt8(packet.header.forwarded, offset++);
        payload.copy(buf, offset);
        return buf;
    }

    unpack(data: Buffer | string): any {
        const buffer = Buffer.isBuffer(data) ? data : Buffer.from(data, "utf8");
        let offset = 0;
        if (buffer.readUInt8(offset++) !== 0x42) throw new Error("Invalid Magic Byte");
        if (buffer.readUInt8(offset++) !== 0x01) throw new Error("Invalid Protocol Version");

        const srcAddrLen = buffer.readUInt8(offset++);
        const srcAddress = buffer.subarray(offset, offset + srcAddrLen).toString("utf8");
        offset += srcAddrLen;
        const srcPort = buffer.readUInt16LE(offset);
        offset += 2;

        const dstAddrLen = buffer.readUInt8(offset++);
        const dstAddress = buffer.subarray(offset, offset + dstAddrLen).toString("utf8");
        offset += dstAddrLen;
        const dstPort = buffer.readUInt16LE(offset);
        offset += 2;

        const packetCount = buffer.readUInt16LE(offset);
        offset += 2;
        const packetIndex = buffer.readUInt16LE(offset);
        offset += 2;
        const dataSize = buffer.readUInt32LE(offset);
        offset += 4;
        const packetHeaderFlag = buffer.readUInt8(offset++);
        const forwarded = buffer.readUInt8(offset++);
        const payload = buffer.subarray(offset);

        return {
            header: { srcAddress, srcPort, dstAddress, dstPort,
                      packetCount, packetIndex, dataSize,
                      packetHeaderFlag, forwarded },
            payload: payload,
        };
    }
}
```

Fragmentation — 32KB loop (`SimpleMQTNLDriver.send()`)

```
// --- FRAGMENTATION LAYER ---
const chunks: (string | Buffer)[] = [];
if (securedPayload.length === 0) {
    chunks.push(useRaw ? Buffer.alloc(0) : "");
} else {
    for (let i = 0; i < securedPayload.length; i += this.packetSize) {
        if (typeof securedPayload === "string") {
            chunks.push(securedPayload.substring(i, i + this.packetSize));
        } else {
            chunks.push(securedPayload.subarray(i, i + this.packetSize));
        }
    }
}

const packetCount = chunks.length;
const topic = `${prefix}${address}`;

for (let i = 0; i < packetCount; i++) {
    const packet = {
        header: {
            srcAddress: this.localAddress,
            srcPort: srcPort,
            dstAddress: address,
            dstPort: port,
            packetCount: packetCount,        // jumlah total chunk
            packetIndex: i,                  // chunk ke-i (0-based)
            dataSize: securedPayload.length, // ukuran TOTAL payload
            packetHeaderFlag: flag,
```

```
            forwarded: 0,
        },
        payload: chunks[i],
    };
    const payloadBuffer = protocol.pack(packet);
    this.txBytes += payloadBuffer.length;
    // publish ke topic (atau loopback lokal jika alamat = node ini)
}
```

Reassembly — merging chunks (`SimpleMQTNLDriver.handleIncomingMessage()`)

```
// --- REASSEMBLY LAYER ---
const key = `${packet.header.srcAddress}:${packet.header.srcPort}->${packet.header.dstAddress}:${packet.header.dstPort}`;

if (!this.receivedPackets.has(key)) {
    this.receivedPackets.set(key, {
        total: packet.header.packetCount,
        received: new Map(),
        timestamp: Date.now(),
    });
}

const session = this.receivedPackets.get(key)!;
session.received.set(packet.header.packetIndex, packet.payload);
session.timestamp = Date.now();

if (session.received.size < session.total) {
    return; // belum lengkap — tunggu chunk berikutnya
}

// Semua chunk tiba!
this.receivedPackets.delete(key);

const entries = Array.from(session.received.entries()).sort((a, b) => a[0] - b[0]);
const assembledPayload = Buffer.isBuffer(entries[0][1])
    ? Buffer.concat(entries.map(e => e[1] as Buffer))
    : entries.map(e => e[1] as string).join("");
```

30 second TTL — expired session cleanup

```
private cleanupExpiredPackets() {
    const now = Date.now();
    const ttl = 30000; // 30 detik untuk chunk yang ditinggalkan
    for (const [key, session] of this.receivedPackets.entries()) {
        if (now - session.timestamp > ttl) {
            this.logger.warn(`[REASSEMBLY] Dropped incomplete packet from ${key} (TTL Expired)`);
            this.receivedPackets.delete(key);
        }
    }
}
```

Exercises / Practice

1. Read `src/common/protocols/MQTNLProtocolJSON.ts` — understand the JSON packet structure (10-element array).
2. Read `src/common/protocols/MQTNLProtocolBinary.ts` — compute the binary header size for `srcAddress="tsix"` and `dstAddress="esp32S3"` (answer: 29 bytes).
3. Read `src/kernel/devices/SimpleMQTNLDriver.ts` — trace the TX path (`send`) and RX path (`handleIncomingMessage`), then find `packetSize` and `ttl`.
4. Read `wiki/mqtnl_binary_ota.md` — how Binary is used for byte-exact OTA.
5. Explain why Binary v1.1 deliberately has no encryption.
6. Experiment: send a payload larger than 32KB from an application, observe the `[FRAGMENTATION]` and `[REASSEMBLY]` logs.

References

- `wiki/mqtnl-ota.md`, `wiki/mqtnl_binary_ota.md` — OTA via MQTNL
- `wiki/course/00-overview.md` §7
- `src/common/protocols/IMQTNLProtocol.ts`, `MQTNLProtocolJSON.ts`, `MQTNLProtocolBinary.ts`

Module 16 — done. Part VI complete. Continue to Module 17 — PixelSpace Protocol.

Module 17 **PixelSpace Protocol** GUI & Desktop

RFC-TSIX-EDU-002 | Seventeenth module of the TSIX curriculum. Understand the GUI data contract: the Worker→Kernel→DOME→Browser data flow, GUIRegistry as the window ownership authority, and the security sanctions.

> PixelSpace is the **constitution of TSIX's GUI**. Its contract (`GUITypes.ts`) must not be changed carelessly — every layer above it (DOME, Emerald, Cashew, Asteracea) depends on the shape of this data.

Learning Objectives

- [ ] Explain the UI mounting and event data flow
- [ ] Explain the role of GUIRegistry (auth wid↔pid)
- [ ] Mention the main GUIActions
- [ ] Explain the security sanctions (SIGKILL / SIGSEGV)
- [ ] Explain why the kernel overrides `payload.pid`

Core Concepts

Data flow

```
Worker (app) → GUI_REQ (61) → Kernel (GUIRegistry auth pid↔wid)
  → DOME daemon (WS broadcast) → Browser (DOM)
Browser → event → DOME → Kernel (SEND_MSG) → Worker (callback)
```

UI mounting flow

```
Worker App   → GUI_REQ (MOUNT_NODE) → Kernel → gui_request event → DOME → WS → Browser (createElement)
```

Event flow (click/input)

```
Browser → WS → DOME → SEND_MSG ke pid → Kernel → ipc_message event → Worker → callback → updateProps
```

GUIAction (GUI operations)

Action	Purpose
`CREATE_WINDOW` / `DESTROY_WINDOW`	Create / destroy a window
`MOUNT_NODE` / `UNMOUNT_NODE`	Mount / unmount an element
`UPDATE_PROPS`	Change element properties
`MINIMIZE_WINDOW` / `RESTORE_WINDOW`	Hide / restore
`MAXIMIZE_WINDOW` / `UNMAXIMIZE_WINDOW`	Fullscreen / normal

GUIRegistry (kernel)

The **single** authority for window ownership:

- `wid → pid` (primary map) + `pid → Set<wid>` (reverse map, fast lookup on exit)
- Z-index auto-increment (starts at 100)
- `registerDaemon(pid)` — only "gued" may receive forwarded GUI_REQ
- Automatic cleanup when a process dies (`destroyAllForPid`)

Security

Violation	Sanction
Malformed payload format	`SIGKILL` — process is killed
Accessing a window owned by another PID	`SIGSEGV` — segmentation fault

> [!IMPORTANT] **The kernel always overrides `payload.pid`.** Never trust the `pid` sent by the application — the kernel sets the real identity from the process context.

Source Code

File	Role
`src/common/GUITypes.ts`	Data contract (IDOMNode, IGUIPayload, GUIAction, IBrowserEvent, IGUIEventIPC)
`src/kernel/GUIRegistry.ts`	Window authority + auth
`src/kernel/Syscalls.ts`	`GUI_REQ` handler + security
`src/mirror/root/ps-sample1.ts`	Raw protocol practice

Snippet (code level)

GUIRegistry.createWindow

```
public createWindow(wid: string, pid: number, title: string = "Untitled"): IWindowEntry {
    if (this.windows.has(wid)) {
        throw new Error(`GUIRegistry: Window '${wid}' already exists.`);
    }
    const entry: IWindowEntry = { wid, pid, title,
        zIndex: this.nextZIndex++, focused: true, createdAt: Date.now() };
    this.windows.set(wid, entry);
    // Update reverse map pid → Set<wid>
    if (!this.pidToWids.has(pid)) this.pidToWids.set(pid, new Set());
    this.pidToWids.get(pid)!.add(wid);
    // Defocus window lain
    return entry;
}
```

Exercise / Practice

1. Read `src/common/GUITypes.ts` — understand the entire "constitution" interface.
2. Read `src/mirror/root/ps-sample1.ts` — practice the raw protocol without a toolkit.
3. Run a GUI app then `ps` — find the gued/DOME PID. Read `wiki/identity_guid_ipc_walkthrough.md` for the identity flow.
4. Modify a window owned by another PID via GUI_REQ — observe SIGSEGV.

References

- `wiki/PIXELSPACE_DEVELOPER_GUIDE.md` §1-2, 10
- `wiki/course/00-overview.en.md` §10
- `src/common/GUITypes.ts`, `src/kernel/GUIRegistry.ts`, `src/kernel/Syscalls.ts`

Module 17 — complete. Continue to Module 18 — DOME Engine.

Module 18 **DOME Engine (Display Server)** GUI & Desktop

RFC-TSIX-EDU-002 | Eighteenth module of the TSIX curriculum. Understand DOME: WebSocket relay + primitive DOM producer + compositor (titlebar, drag, resize, focus, replay).

> DOME is the TSIX display server — analogous to X11/Wayland, but rendered as **DOM in the host browser** via WebSocket :8080. The server is monolithic (single daemon, relay + compositor) because of drag/resize latency considerations (a separate compositor = ~2–4ms overhead). The browser client is instead modular: `dome.ts` serves static `dome-client-*.js` modules via `/dome/*.js`.

Learning Objectives

- [ ] Explain DOME's role as a relay + compositor
- [ ] Explain the DOME specification & features
- [ ] Explain the monolithic vs separate-compositor design tradeoff
- [ ] Explain the role of `windowStates` and replay
- [ ] Explain the enforcement of the `maximizable` flag on all maximize paths
- [ ] Explain navigation protection, per-app traffic, DDC modules, and boot readiness
- [ ] Explain the DOME ↔ Kernel ↔ Browser relationship

Core Concepts

Specification

Aspect	Description
Location	`src/mirror/opt/dome/dome.ts` (server) + thin `dome-client.html` + `dome-client-*.js` modules (browser)
Port	WebSocket :8080
Role	Display server (Ring 4 daemon)
Model	WS relay + primitive DOM producer + compositor

Features

- Window registry, Z-index, event relay
- **State Replay** (`windowStates`): all MOUNT_NODE & UPDATE_PROPS are stored, then re-sent when the browser reconnects (F5)
- **State Pruning** (`pruneWindowState`): when a node is unmounted, DOME cleans up the state — prevents orphan state from leaking into replay
- **Orphan Discard** (browser-side): MOUNT_NODE whose parent is not found is discarded — prevents dialog/modal residue after F5
- **Overlay Layer Search**: `findElementById` searches 3 levels — `win.el` → `__global_start_menu__` → `__tsix_overlay_layer__`
- **Modular client**: browser logic is split into static modules `dome-client-core.js`, `-term.js`, `-codemirror.js`, `-chart.js`, `-dom.js`, `-windows.js`, `-ui.js`, `-ddc.js`, `-res.js`; `dome.ts` reads them from VFS `/opt/dome/*.js` at startup and serves them via `/dome/*.js`
- **`maximizable` flag**: honored on all maximize paths — double-click titlebar, "Maximize" context menu, server guard `maximize_window`, and syscall guard `MAXIMIZE_WINDOW`
- **Maximize while minimized**: the window is first shown at its last position before the rect is measured; restore uses `_origRect` if `_savedRect` is zero
- **Navigation protection**: refresh (F5/Ctrl+R/Cmd+R) and back/forward require confirmation (`protectNavigation()` + `beforeunload`)

- **Per-app traffic accounting**: `TRAFFIC_QUERY` excludes the asking app's own traffic and reports `selfTxBytes` / `selfTxPkts`
- **DDC module**: `dome-client-ddc.js` hosts Native-JS apps (Fabric.js / Three.js via CDN) in a Shadow DOM; DOME relays `DDC_MSG` / `DDC_RESIZE` / `DDC_STOP`; `destroyDDCByWid(wid)` is called when a window is closed
- **Boot readiness**: DOME writes `/var/run/dome.ready`; `rc.local` polls the marker (10s timeout, 200ms interval) before starting Asteracea — replaces `sleep 1s`
- **`ensureListener()`**: attaches a listener once per element per event — replaces the `cloneNode` pattern that dropped old listeners

Design tradeoff: why monolithic?

	Separate compositor (X11-style)	**Monolithic DOME**
Overhead	~2–4ms per drag/resize operation	direct, no extra IPC
Advantage	modularity	low latency

> Because the main target is **latency-sensitive drag/resize/focus interaction**, DOME chose to be monolithic. There is a design note & refactor plan in the wiki.

Replay Flow

```
Browser F5 → WebSocket reconnect
  → DOME kirim CREATE_WINDOW (semua window aktif)
  → DOME kirim semua stored states (MOUNT_NODE + UPDATE_PROPS)
  → Jika window sedang maximized, DOME kirim MAXIMIZE_WINDOW ulang
  → Browser build ulang UI + terima event dari user
```

Snippets (code level)

State pruning (condensed)

```
const allIds = new Set<string>([targetId]);
for (const state of states) {
    if (state?.type === "MOUNT_NODE" && state.node) {
        const nodeIds = new Set<string>();
        collectNodeIds(state.node, nodeIds);
        if (nodeIds.has(targetId)) for (const id of nodeIds) allIds.add(id);
    }
}
// Hapus MOUNT_NODE (parent atau node di dalam tree) & UPDATE_PROPS (target exact match)
```

Browser orphan discard

```
if (parent) {
    parent.appendChild(domEl);
} else {
    console.log('[DEBUG] Orphan discarded:', node.id, targetId); // DISCARD
}
```

Maximize guard — all paths honor the `maximizable` flag

```
// dome-client-windows.js — double-click titlebar & handleMaximizeWindow
if (!w._isMaximized && w.maximizable === false) return; // titlebar dbl-click
if (win.maximizable === false) return;                  // handleMaximizeWindow
```

```
// dome.ts — guard otoritatif sisi server (event browser + syscall)
if (event.eventType === "maximize_window") {
    if (entry.maximizable === false) return;
}
// syscall GUIAction.MAXIMIZE_WINDOW
if (windows.get(wid)?.maximizable === false) break;
```

Safe restore — fallback when the saved rect is zero

```
// dome-client-windows.js — handleRestoreWindow
let saved = win._savedRect;
if (saved && (saved.width <= 0 || saved.height <= 0)) {
    saved = win.el._origRect || null; // jangan restore ke (0,0,0,0)
}
```

ensureListener — listeners are not duplicated and not lost

```
// dome-client-dom.js — dipakai buildDOM() & handleUpdateProps()
function ensureListener(el, event, listener) {
    if (!el.__tsixL) el.__tsixL = Object.create(null);
    if (!el.__tsixL[event]) {
        el.addEventListener(event, listener);
        el.__tsixL[event] = true;
    }
}
```

Per-app traffic — the observer does not count itself

```
// dome.ts — TRAFFIC_QUERY meng-exclude traffic app penanya
const self = appTraffic.get(payload.pid) || { txBytes: 0, txPkts: 0 };
const stats = {
    ...wsTraffic,
    txBytes: Math.max(0, wsTraffic.txBytes - self.txBytes),
    txPkts: Math.max(0, wsTraffic.txPkts - self.txPkts),
    selfTxBytes: self.txBytes, // traffic milik app penanya
    selfTxPkts: self.txPkts,
};
```

DDC cleanup — stop the RAF loop when the window is closed

```
// dome-client-windows.js — handleDestroyWindow (safety net)
if (typeof TSIX.destroyDDCByWid === "function") {
    TSIX.destroyDDCByWid(wid); // stop semua runtime DDC window ini
}
```

Practice / Hands-on

1. Open `http://localhost:8080` — observe the DOME client in the browser.
2. Press F5 while several windows are open — observe the navigation confirmation prompt and the state replay.
3. Read `src/mirror/opt/dome/dome.ts` — find `windowStates`, `pruneWindowState`, the `maximizable` guard, the `DDC_*` relay, and `TRAFFIC_QUERY`.
4. Read `src/mirror/opt/dome/dome-client-windows.js` — find `handleMaximizeWindow` (maximize-while-minimized) and `handleRestoreWindow`.
5. Read `src/mirror/opt/dome/dome-client-core.js` — find `protectNavigation()`.
6. Read `src/mirror/opt/dome/dome-client-ddc.js` — find `initDDC` and `destroyDDCByWid`.
7. Read `src/mirror/opt/dome/dome-client-dom.js` — find `ensureListener`.
8. Read `src/mirror/etc/rc.local.ts` — find the polling of `/var/run/dome.ready` before starting Asteracea.

References

- `wiki/PIXELSPACE_DEVELOPER_GUIDE.md` §2, §11
- `wiki/course/00-overview.en.md` §10
- `wiki/changelogs/dome.md` — DOME change history
- `src/mirror/opt/dome/dome.ts` + `src/mirror/opt/dome/dome-client-*.js`
- `src/mirror/etc/rc.local.ts` — polling of `/var/run/dome.ready`

Module 18 — done. Continue to Module 19 — Emerald Widget Toolkit.

Module 19 **Emerald Widget Toolkit** GUI & Desktop

RFC-TSIX-EDU-002 | Module nineteen of the TSIX curriculum. Understand the GUI toolkit layer on top of the stable protocol: the Screen wrapper, factory functions, and self-rendering connected widgets.

> Emerald (`@tsix/emerald`) is the GTK/Qt of TSIX. All UI is built as a **Virtual DOM Tree** (`IDOMNode`) and sent via syscall — applications have **no access** to `document`, `window`, or any browser API.

Learning Objectives

- [ ] Explain the position of Emerald in the GUI stack
- [ ] Explain factory functions (`div`, `button`, `input`, ...)
- [ ] Explain the Screen wrapper & the mount → bind → loop pattern
- [ ] Explain connected widgets (self-rendering), including `ConnectedDataGrid`
- [ ] Explain `ensureListener()` — listener props persistent across `UPDATE_PROPS` (`cloneNode` bug fixed)

Core Concepts

Position in the architecture

```
BROWSER → DOME (display server) → KERNEL (auth) → EMERALD (kamu di sini)
```

Basic app structure

```
import { Program } from "@tsix/Application";
import { Screen, div, h1, paragraph } from "@tsix/emerald";

export const main = Program(async (args: string[]) => {
  // 1. Buat Screen (jendela)
  // 2. Bangun Virtual DOM tree dengan factory functions
  // 3. Mount, bind event, loop
});
```

Factory functions

Factories produce `IDOMNode`. The full list is in `src/mirror/lib/emerald.ts`:

- Basic: `text()`, `div()`, `span()`, `h1()`/`h2()`/`h3()`, `paragraph()`, `button()`, `input()`, `textarea()`, `selectBox()`, `image()`
- IoT: `lineChart()`, `radialGauge()`, `verticalGauge()`, `sevenSegment()`, `indicatorLamp()`, `toggleSwitch()`, `sensorCard()`, `relayCard()`, `badge()`, `taskbarButton()`
- Table: `dataGrid()` (static), `slider()`

`alert()`, `confirm()`, `question()`, `openFileDialog()`, `saveFileDialog()` are **not factories** — they are methods on `Window`/`Screen` that show modal overlay dialogs, not functions that produce `IDOMNode`.

Basic pattern: Mount → Bind → Loop

```
const screen = new Screen({ title: "App", width: 400, height: 300 });
await screen.mount(
  div({ id: "root", style: { padding: "16px" } },
    h1({ text: "Hello" }),
    button({ id: "btn", text: "Klik" }),
  ),
);
// bind event → updateProps (di-batch & auto-flush) → loop
```

```
await screen.on("btn", "click", async () => {
  await screen.update("btn", { text: "✅ Diklik!" });
});
```

Connected widgets (self-rendering)

Connected widgets render themselves and update the screen on their own — a higher-level pattern than plain factories. The common pattern is `build()` → `mount(screen)` → `setData()` / `setValue()`, using **targeted updates** (not a full `setContent`).

The `Connected*` classes in `emerald.ts`:

- `ConnectedLineChart`, `ConnectedRadialGauge`, `ConnectedSevenSegment`, `ConnectedIndicatorLamp`
- `ConnectedVerticalGauge`, `ConnectedToggle`, `ConnectedSensorCard`, `ConnectedRelayCard`
- `ConnectedDataGrid` — interactive data table: asc/desc sort, column resize (native drag), row selection based on stable keys, incremental `appendData()` & `maxRows` option. One scroll container (sticky `th` header) — horizontal & vertical scroll automatically stay in sync.

> [!IMPORTANT] **Listener props & `ensureListener()` (`cloneNode` bug fixed).** Four listener props — `onClickId`, `onContextMenuId`, `onInputId`, `onKeydownId` — are installed by the DOME client through the `ensureListener()` helper in `src/mirror/opt/dome/dome-client-dom.js` (used in `buildDOM()` and `handleUpdateProps()`), **once per element per event type** (tracked via `el.__tsixL`). Previously `handleUpdateProps` cloned nodes to "clean up old listeners" — `cloneNode` does not copy listeners, so if one batch of `UPDATE_PROPS` carries several listener props at once (e.g. `onInputId` + `onKeydownId` on a password field), a newly installed listener could be lost. With `ensureListener`, listeners are **persistent across `UPDATE_PROPS`** — not lost and not duplicated. Best practice still applies: set listener props (`onClickId`, `onInputId`, etc.) **at mount time** (in `build()` / `mount()`), then register callbacks via `screen.on()` / `win.bindHandler()`.

Source Code

File	Role
`src/mirror/lib/emerald.ts`	Widget toolkit `@tsix/emerald`
`src/mirror/lib/theme.ts`	Theme
`src/mirror/lib/cashew.ts`	Layer on top of Emerald (Module 20)
`src/mirror/opt/dome/dome-client-dom.js`	DOME client DOM engine — `buildDOM()`, `handleUpdateProps()`, `ensureListener()`
`src/mirror/root/ps-sample2.ts`, `ps-sample3.ts`	Practice

Exercises / Practice

1. Read `wiki/emerald-in-a-nutshell.md` — work through Hello World up to the complete case study.
2. Build a form with an input + button; bind a click event that changes the text.
3. Use `alert()` / `confirm()` — observe how it appears as a window overlay.
4. Read `src/mirror/lib/emerald.ts` — find the `Screen` and `setContent` implementations.
5. Build a `ConnectedDataGrid` with column sort & resize; use `appendData()` for real-time data and `maxRows` to limit rows.

References

- `wiki/emerald-in-a-nutshell.md`, `wiki/cashew-in-a-nutshell.md`
- `wiki/PIXELSPACE_DEVELOPER_GUIDE.md` §5-7
- `src/mirror/lib/emerald.ts`, `src/mirror/lib/theme.ts`

Module 19 — done. Continue to Module 20 — Cashew Component Framework.

Module 20 **Cashew Component Framework** GUI & Desktop

RFC-TSIX-EDU-002 | Twentieth module of the TSIX curriculum. Understand the OOP/Delphi-style layer above Emerald: `TForm`, `TButton`, `TEdit`, auto-bind lifecycle, and complex widgets.

> Cashew (`@tsix/cashew`) adopts the component pattern of **Delphi / Turbo Pascal** — components are **stateful classes** with properties & events, not nested functions. It is the VCL analogue on top of GTK.

Learning Objectives

- [ ] Explain the OOP/Delphi-style philosophy of Cashew
- [ ] Explain the `TForm.run()` lifecycle (auto-bind)
- [ ] Explain the basic components: TForm, TPanel, TLabel, TButton, TEdit
- [ ] Explain why `onClickId`/`onInputId` are set at build time (mount-time)
- [ ] Name the complex widgets (TChart, TSevenSegment, TSensorCard, etc.)

Core Concepts

Philosophy

In Emerald you write UI with nested functions (`div([button(...)])`). In Cashew, you create **class objects**:

```
const form = new TForm("My App", 400, 300);
const btn = new TButton("btn-click");
btn.caption = "Klik";
btn.onClick = () => { count++; lblCounter.caption = "Count: " + count; };
form.add(btn);
await form.run();
```

TForm.run() lifecycle

`run()` has an **auto-bind lifecycle**: bindEventHandler + refresh per component. After `run()`, property changes (`caption`, `text`, etc.) automatically sync to the screen.

The sequence in `TForm.run()` (`src/mirror/lib/cashew.ts`):

1. **Theme** — load the active theme before building so the CSS color variables are correct.
2. **Build & mount** — build the `IDOMNode` tree via `this.build()` then `_screen.mount(...)`.
3. **Auto-bind** — DFS traversal over all children, call `comp.bindEventHandler(screen)` (event registration).
4. **Auto-refresh** — DFS traversal over all children, call `await comp.refresh(screen)` (render dynamic data, e.g. `TListBox`, `TDataGrid`).
5. **Setup** — call `await onSetup(screen)` when set (extra binding).
6. **Event loop** — `_screen.loopUntilClose()` waits until the window is closed.

```
sequenceDiagram
    participant App as Aplikasi (main)
    participant Form as TForm.run()
    participant Comp as Komponen (TButton, TEdit, TDataGrid, ...)
    participant Screen as Screen (Emerald)

    App->>Form: await run()
    Form->>Form: loadCurrent() — theme
    Form->>Screen: mount(build())
    loop auto-bind (DFS)
        Form->>Comp: bindEventHandler(screen)
        Comp->>Screen: screen.on(id, "click"/"input", cb)
    end
```

```
    loop auto-refresh (DFS)
        Form->>Comp: await refresh(screen)
        Comp->>Screen: setContent / setData
    end
    Form->>App: await onSetup(screen)
    Form->>Screen: loopUntilClose()
    loop event loop
        Screen->>Comp: event (click / input / timer)
        Comp->>Screen: screen.update(id, props)
    end
```

Component Hierarchy

All components derive from the base class `TComponent` in `src/mirror/lib/cashew.ts`:

```
graph TD
    TC[TComponent] --> TForm
    TC --> TPanel
    TC --> TLabel
    TC --> TButton
    TC --> TEdit
    TC --> TMemo
    TC --> TCheckBox
    TC --> TListBox
    TC --> TRadioButton
    TC --> TComboBox
    TC --> TStatusBar
    TC --> TLineChart
    TC --> TRadialGauge
    TC --> TSevenSegment
    TC --> TIndicatorLamp
    TC --> TToggleSwitch
    TC --> TVerticalGauge
    TC --> TSensorCard
    TC --> TRelayCard
    TC --> TSlider
    TC --> TTimer
    TC --> TChart
    TC --> TDataGrid
```

> [!NOTE] **Not classes.** `TDialogs` is a **static** utility (not a `TComponent`). `HStack`, `VStack`, `Spacer`, `TScrollBox`, `TFlowPanel`, `TGridPanel`, `TSplitHorizontal`, `TSplitVertical`, `TGroupBox` are **functions** that return `TComponent` — quick layout without new subclasses. Align constants (`alTop`, `alClient`, etc.) are available for `style.position`.

Why events are set at build time (mount-time)

`onClickId` / `onInputId` are written directly into `props` in the constructor or `build()` (example: `TButton` sets `this.props.onClickId = id`). When DOME mounts the node, the listener is attached once. Adding listeners via `app.on()` after mount risks colliding with DOME's `cloneNode` pattern, which **does not copy listeners** — see Module 19.

Basic components

Component	Function
`TForm`	Main window (`add()`, `alert()`, `confirm()`, `screen`, `onSetup`; options `maximizable` / `resizable` / `fullscreen` / `frameless`)
`TPanel`	Container (with/without style)
`TLabel`	Text (`caption`)
`TButton`	Button (`onClick`, `caption`, `enabled`)
`TEdit`	Text input (`onInput`, `text`, `placeholder`)

Component	Function
`TMemo`, `TCheckBox`, `TListBox`, `TStatusBar`, `TRadioButton`, `TComboBox`	Others
`TScrollBox`, `TFlowPanel`, `TGridPanel`, `TSplitHorizontal/Vertical`, `TGroupBox`	Layout
`HStack`, `VStack`, `Spacer`	Quick layout

Complex & IoT widgets

Component	Function
`TDialogs`	Static dialog utility: `alert`, `confirm`, `input`, `openFile`, `saveFile`
`TTimer`	Delphi-style interval timer; auto-cleanup when the form closes (`onTimer`)
`TChart`	Real-time multi-series chart (Lightweight Charts via IPC DOME)
`TLineChart`	SVG line chart (spline, fill, scroll animation)
`TRadialGauge`	Circular speedometer-style gauge
`TSevenSegment`	LED 7-segment display (options `scale`, `height`)
`TIndicatorLamp`	ON/OFF indicator lamp with glow
`TToggleSwitch`	Clickable ON/OFF switch
`TVerticalGauge`	Vertical glass tube; value text dual-layer masking
`TSensorCard`	IoT sensor card with progress bar
`TRelayCard`	ON/OFF relay card
`TSlider`	Horizontal range slider

TDataGrid — one scroll container

`TDataGrid` wraps Emerald's `ConnectedDataGrid` (`src/mirror/lib/emerald.ts`). A 2026-08-03 change makes the grid use a **single scroll container**:

- **One scroll container** — header table + `<style>` + body table live inside one `body-scroll` (`overflow: auto`). The header is pinned via **`th` sticky** (`position: sticky; top: 0; z-index: 2`), the same pattern as the static `dataGrid()`.
- **No manual compensation** — the `thead-scroll` wrapper, relayed `scrollLeft`, `scrollbar-gutter`, and `calc(100% - scrollbarWidth)` were removed. Horizontal & vertical scrolling sync automatically; header width == body width == container width.
- **Column resize** — the resize scope is the grid wrapper `table.closest(".tsix-dgrid")` (not `table.parentElement`). The drag handle changes the width of `<col>` in **all colgroups** (header + body) at once.
- **`col_resized`** — when dragging finishes, the browser sends `col_resized` with `targetId` = **grid id** (wrapper `.tsix-dgrid`) → `ConnectedDataGrid` stores the width and re-applies it on every render (sort/refresh/setData).

> [!IMPORTANT] **Mount-time event.** Set `onClickId` / `onInputId` at build time (mount-time) — not in `app.on()` — to avoid the `cloneNode` bug (see Module 19).

Flow / How It Works

1. Create a `TForm` object — two forms: sequential `new TForm(title, w, h, ...)` or object literal `new TForm({ title, ... })`.
2. Add components via `form.add(...)` — this forms a parent-child tree.
3. Set properties (`caption`, `text`, `value`) and event handlers (`onClick`, `onInput`, `onChange`, `onTimer`).

4. Call `await form.run()` — build → mount → auto-bind → auto-refresh → `onSetup` → loop.
5. Change properties at any time; bound components automatically call `screen.update(...)` to the screen.
6. Window closes → `loopUntilClose()` finishes; managed timers are cleaned up automatically.

Source Code

- `src/mirror/lib/cashew.ts` — the whole framework (TComponent, TForm, TDialogs, basic components, layout, IoT widgets, TDataGrid)
- `src/mirror/lib/emerald.ts` — `ConnectedDataGrid` (grid rendering & column width state)
- `src/mirror/opt/dome/dome-client-dom.js` — DOM build, `data-col-resize`, `col_resized` event
- `wiki/cashew-in-a-nutshell.md`, `wiki/emerald-in-a-nutshell.md` — summary

Snippet (code level)

Cashew quick start

```
import { Program } from "@tsix/Application";
import { TForm, TLabel, TButton, TStatusBar } from "@tsix/cashew";

export const main = Program(async () => {
  const form = new TForm("My App", 400, 300);
  let count = 0;

  const lblCounter = new TLabel("counter");
  lblCounter.caption = "Count: 0";
  form.add(lblCounter);

  const btnClick = new TButton("btn-click");
  btnClick.caption = "Klik";
  btnClick.onClick = () => { count++; lblCounter.caption = "Count: " + count; };
  form.add(btnClick);

  const status = new TStatusBar("status");
  status.text = "✅ Siap";
  form.add(status);

  // Tidak perlu bind manual — TForm.run() memanggil bindEventHandler()
  // untuk setiap komponen secara otomatis (auto-bind lifecycle).
  await form.run();
});
```

TForm.run() — auto-bind lifecycle (from cashew.ts)

```
async run(): Promise<void> {
  // 1. Theme — load dulu biar komponen pake warna yang benar
  try {
    const t = await ensureTheme();
    await t.loadCurrent();
    t.watch();
  } catch (_) { /* theme opsional — skip jika gagal */ }

  // 2. Build & mount ke Screen
  this._screen = new Screen(opts); // title, width, height, maximizable, ...
  await this._screen.mount(this.build());
  // (opsional) kirim WINDOW_THEME ke DOME untuk CSS variables

  // 3. Auto-bind: DFS — panggil bindEventHandler() untuk semua komponen
  const autoBind = (comp: TComponent) => {
    comp.bindEventHandler(this._screen);
    for (const child of comp.children) autoBind(child);
  };
  for (const child of this.children) autoBind(child);

  // 4. Auto-refresh: DFS — panggil refresh() (misal TListBox, TDataGrid)
```

```
  const autoRefresh = async (comp: TComponent) => {
    await comp.refresh(this._screen);
    for (const child of comp.children) await autoRefresh(child);
  };
  for (const child of this.children) await autoRefresh(child);

  // 5. Setup custom — setelah bind & refresh
  if (this.onSetup) await this.onSetup(this._screen);

  // 6. Event loop sampai window ditutup
  await this._screen.loopUntilClose();
}
```

TComponent — bindEventHandler & refresh (base no-op)

```
export class TComponent {
  // Daftarkan event handler ke Screen. Otomatis dipanggil oleh TForm.run().
  bindEventHandler(screen: Screen): void {
    // Base: no-op — subclass override untuk register event
  }

  // Update tampilan setelah mount. Otomatis dipanggil oleh TForm.run().
  async refresh(screen: Screen): Promise<void> {
    // Base: no-op — subclass override untuk render dinamis
  }

  build(): IDOMNode {
    return {
      id: this.id,
      tag: this.tag,
      props: { ...this.props, style: { ...this.style } },
      children: this._children.map((c) => c.build()),
    };
  }
}
```

Widget example — TButton & TEdit (from cashew.ts)

```
// TButton — bind onClick saat auto-bind
// constructor: this.tag = "button"; this.props.onClickId = id;
export class TButton extends TComponent {
  public onClick: (() => void) | null = null;
  bindEventHandler(screen: Screen): void {
    this._screen = screen;
    if (this.onClick) screen.on(this.id, "click", this.onClick);
  }
}

// TEdit — bind onInput saat auto-bind
// constructor: this.tag = "input"; this.props.onInputId = id;
export class TEdit extends TComponent {
  public onInput: ((value: string) => void) | null = null;
  bindEventHandler(screen: Screen): void {
    this._screen = screen;
    if (this.onInput) {
      screen.on(this.id, "input", (ev: any) => {
        this.onInput!(ev?.value || "");
      });
    }
  }
}
```

Widget example — TDataGrid (from cashew.ts)

```
// Penggunaan di app
const grid = new TDataGrid(
  "sensor",
  [
```

```
    { key: "node_id", label: "Node", width: 140 },
    { key: "value", label: "Nilai", width: 80, align: "right" },
    { key: "timestamp", label: "Waktu", width: "40%" },
  ],
  [],
  { height: 300 },
);
form.add(grid);
grid.onSort = (key, dir) => { reloadSorted(key, dir); };
grid.onRowClick = (index, record) => { openDetail(record); }; // index = row-key stabil

// Data inkremental — hanya baris baru yang dikirim ke browser (hemat WS)
await grid.appendData(newRows);
```

```
// (dari cashew.ts — TDataGrid) auto-bind oleh TForm.run(): mount
// ConnectedDataGrid + daftarkan handler sort & klik row.
bindEventHandler(screen: Screen): void {
  this._screen = screen;
  void this.grid
    .mount(
      screen,
      (key, dir) => { if (this.onSort) this.onSort(key, dir); },
      (index, record) => { if (this.onRowClick) this.onRowClick(index, record); },
    )
    .catch(() => {});
}

// (dari cashew.ts — TDataGrid) auto-refresh: render data awal
async refresh(screen: Screen): Promise<void> {
  await this.grid.setData(this._data);
}
```

TTimer — Delphi-style interval

```
const timer = new TTimer("tmr-update", 1000);
timer.onTimer = () => { chart.pushData(Date.now(), sensor.value); };
timer.enabled = true;
form.add(timer); // bindEventHandler auto-start saat form.run()
```

Layout helpers

```
form.add(
  HStack(
    new TButton("btn-a"),
    new TButton("btn-b"),
    Spacer(),            // flex: 1 — mendorong tombol berikutnya ke kanan
    new TButton("btn-c"),
  ),
);

// Dua panel bersebelahan dengan divider yang bisa di-drag
form.add(TSplitHorizontal(leftPanel, rightPanel, "1fr"));
```

Exercises / Practice

1. Read `wiki/cashew-in-a-nutshell.md` — follow the quick start and the layout components.
2. Build an app with `TForm` + `TPanel` + `TEdit` + `TListBox`: type in the TEdit, click the button, and the result goes into the TListBox.
3. Use `TDialogs.confirm()` for confirmation before deleting an item.
4. Read `src/mirror/lib/cashew.ts` — find the implementations of `TForm.run()`, `bindEventHandler()`, and `refresh()`.
5. Create a `TDataGrid` with `appendData()` for continuously growing data (log / telemetry).

References

- `wiki/cashew-in-a-nutshell.md`, `wiki/emerald-in-a-nutshell.md`
- `wiki/course/00-overview.en.md` §10
- `src/mirror/lib/cashew.ts`, `src/mirror/lib/emerald.ts`
- `src/mirror/opt/dome/dome-client-dom.js`
- Changelog: `wiki/changelogs/cashew.md`, `wiki/changelogs/dome.md`

Module 20 — done. Continue to Module 21 — Asteracea & TDE.

Module 21 **Asteracea & TDE** GUI & Desktop

RFC-TSIX-EDU-002 | Twenty-first module of the TSIX curriculum. Understand the TSIX Window Manager: an ordinary PixelSpace app (fullscreen frameless) that manages the lifecycle of other GUI apps.

> Key point: **Asteracea is not part of the kernel/DOME** — it is an ordinary PixelSpace app (Ring 4, sandboxed worker) that runs fullscreen. It launches, controls, and manages the lifecycle of other GUI apps.

Learning Objectives

- [ ] Explain Asteracea's position in the architecture
- [ ] Explain the queue-based IPC philosophy (MessageBus)
- [ ] Explain the taskbar (pinned/running/foreign)
- [ ] Explain the launcher & fuzzy search
- [ ] Explain lifecycle events via `/opt/asteracea/wm-pid`

Core Concepts

Specification

Aspect	Description
Location	`src/mirror/opt/asteracea/asteracea.ts`
Mode	Fullscreen, frameless
Model	Queue-based IPC (`MessageBus`)
Protocol	PixelSpace Protocol via DOME
Privilege	Ring 4 (sandboxed worker)
Auto-start	`/etc/rc.local.ts` (after DOME is ready — poll `/var/run/dome.ready`)

Queue-based philosophy

All events enter a FIFO queue (`MessageBus`) and are processed one by one — preventing race conditions and starvation:

```
App A ──(shell.send)──► Kernel ──(ipc_message)──► MessageBus Queue ──► handlers
App B ──(GUI_REQ)─────► DOME  ──(ipc_message)──► MessageBus Queue ──► handlers
Browser ──(click)─────► DOME  ──(shell.send)────► MessageBus Queue ──► handlers
```

 Desktop notification flow: app → kernel → Asteracea → browser *Source:* *wiki/diagram/ASTERACEA_WM-1.mmd*

Components

- **Taskbar**: pinned / running / foreign windows
- **Launcher**: start menu + **fuzzy search** for apps
- **Login** & **wallpaper**
- **Desktop Context Menu (DCM)**
- **App State Manager**

IPC lifecycle events

The WM listens for `GUI_WINDOW_*` lifecycle events: `GUI_WINDOW_CREATED`, `GUI_WINDOW_MINIMIZED`, `GUI_WINDOW_RESTORED`, `GUI_WINDOW_MAXIMIZED`, `GUI_WINDOW_UNMAXIMIZED`, `GUI_WINDOW_CLOSED`, and `GUI_WINDOW_ERROR`. Other GUI apps communicate with the WM through `/opt/asteracea/wm-pid` (PID file) — Emerald broadcasts events to Asteracea.

Fixes & Latest Features

To stay in sync with the real code, the following behaviors were added:

- **Daemonize** — `main` starts with `shell.daemonize("Asteracea Window Manager")`. The process detaches from the tty1 console; stdio is redirected to `/dev/null`. Logs still go to `/var/log/syslog` via VFS (`std.log`/`std.error`), and GUI rendering stays normal because it goes through DOME.
- **Taskbar icon & tooltip for foreign apps** — the `GUI_WINDOW_CREATED` handler forwards `payload.icon || "💻"` to `registerForeignApp()`. The icon is used for the taskbar button; `title` (the window title) becomes the tooltip via the `title` prop (translated to `data-tt` in DOME). Foreign apps can now show a custom icon.
- **`GUI_WINDOW_MAXIMIZED` / `GUI_WINDOW_UNMAXIMIZED` handlers** — both call `transitionTo(appId, "RUNNING")` and set the active taskbar style (same as `GUI_WINDOW_RESTORED`). WM state stays in sync after maximizing via the taskbar context menu; the next taskbar click becomes a minimize toggle.
- **Login without password prefill** — the password field is no longer filled with `value: "1"` (initialized as `loginPass = ""`). The old prefill would "stick" in front of the new password and make login always fail after `passwd` changes the password.
- **Wallpaper persistence** — before writing `/opt/asteracea/wallpaper/current-wp.b64`, `showWallpaperDialog` creates the `/opt/asteracea/wallpaper` folder (inside a try-catch). Previously the folder did not exist → `fs.writeFile` failed silently → blank wallpaper after reboot.

Source Code

File	Role
`src/mirror/opt/asteracea/asteracea.ts`	Window manager
`src/mirror/opt/asteracea/menu/*.menu`	Launcher menu configuration
`src/mirror/etc/rc.local.ts`	Auto-start DOME + Asteracea (poll `/var/run/dome.ready`)

Exercises / Practice

1. Log in to TSIX — observe the desktop: taskbar, launcher, wallpaper.
2. Open the launcher and type part of an app name — observe fuzzy search.
3. Read `wiki/ASTERACEA_WM.md` — learn about the MessageBus and the App State Manager.
4. Read `src/mirror/opt/asteracea/asteracea.ts` — find the `GUI_WINDOW_*` lifecycle handlers.

References

- `wiki/ASTERACEA_WM.md` — complete WM documentation
- `wiki/PIXELSPACE_DEVELOPER_GUIDE.md` §3, §9
- `wiki/course/00-overview.md` §10
- `src/mirror/opt/asteracea/asteracea.ts`, `src/mirror/opt/asteracea/menu/*.menu`

Module 21 — done. Continue to Module 22 — State Replay & Persistence.

Module 22 **State Replay & Persistence** GUI & Desktop

RFC-TSIX-EDU-002 | Module twenty-two of the TSIX curriculum. Understand how the UI is restored when the browser reconnects (F5): `windowStates`, `pruneWindowState`, orphan discard, findElementById across 3 layers, and navigation protection (`allowReload`).

> F5 in the browser should **not clear the desktop**. Since navigation protection (DOME 2026-08-06), refresh requires confirmation first. DOME stores all MOUNT_NODE & UPDATE_PROPS in `windowStates` and resends them to the new browser after reload is confirmed. Pruning ensures no orphan state leaks into the replay.

Learning Objectives

- [ ] Explain the State Replay mechanism
- [ ] Explain `pruneWindowState` and why it is needed
- [ ] Explain browser-side orphan discard
- [ ] Explain `findElementById` across 3 layers
- [ ] Explain navigation protection (`allowReload`) and replay after reload
- [ ] Explain the `_savedRect` → `_origRect` fallback during restore
- [ ] Explain the bug scenarios that are prevented (dialog residue after F5)

Core Concepts

State Replay

`windowStates` = `Map<wid, state[]>` — stores all MOUNT_NODE & UPDATE_PROPS as "replay payloads".

Lifecycle (in `dome.ts`, server side):

- `CREATE_WINDOW` → `windowStates.set(wid, [])` — initializes the state history.
- `MOUNT_NODE` → push `{ type, wid, node, targetId }` to the array.
- `UPDATE_PROPS` → push `{ type, wid, targetId, props }` to the array.
- `UNMOUNT_NODE` → `pruneWindowState(wid, targetId)` then broadcast.
- `DESTROY_WINDOW` → `windowStates.delete(wid)` — the history is cleared.

Replay on reconnect (F5):

```
Browser F5 → WebSocket reconnect
  → DOME kirim CREATE_WINDOW (semua window aktif)
  → DOME kirim semua stored states (MOUNT_NODE + UPDATE_PROPS)
  → jika entry.isMaximized → kirim MAXIMIZE_WINDOW (status maximize dipulihkan)
  → DOME kirim tema terakhir (lastThemeColors)
  → Browser build ulang UI + terima event dari user
```

On the browser side, when the socket `onclose` fires, all windows from the old session are discarded (`state.windows.clear()`). Then `onopen` is ready to receive the replay from the server.

State Pruning (`pruneWindowState`)

When `UNMOUNT_NODE` is called, DOME cleans up the related state:

1. **Collect**: for each MOUNT_NODE state, gather ALL nodeIds from its tree (including children) via `collectNodeIds`.
2. **Filter**: remove the MOUNT_NODE if `state.targetId === targetId`, or if any nodeId in its tree equals `targetId`. UPDATE_PROPS is only removed when `state.targetId === targetId` (exact match).

This prevents **child states** (from `setContent()`) from lingering after the parent is unmounted → prevents DOM residue on F5.

Browser-side orphan discard

If `handleMountNode` receives a MOUNT_NODE whose `targetId` is not found in the DOM, the node **is discarded** — not a fallback to `win.content`. This prevents:

- Wallpaper dialog child states from appearing on the desktop after F5
- Login overlay residue
- Orphan modal/dialog elements

Overlay layer search

`findElementById(wid, nodeId)` searches at 3 levels:

1. `win.el.querySelector([data-tsix-id=...])` — inside the window
2. `__global_start_menu__` — global start menu
3. `__tsix_overlay_layer__` — overlay layer (launcher, dialog)

This allows elements that were extracted to the overlay (e.g. `launcher-grid`) to still be found by mount/replay.

Navigation Protection (refresh confirmation)

Since DOME 2026-08-06, refresh no longer runs directly. The `protectNavigation()` IIFE in `dome-client-core.js` uses the `allowReload` flag:

- **F5 / Ctrl+R / Cmd+R** is intercepted in the capture phase → `preventDefault()` → `window.confirm("Yakin mau refresh TDE? ...")` dialog.
  - **OK** → sets `allowReload = true`, then `location.reload()`.
  - **Cancel** → the page stays intact, no reload.
- **back/forward, close tab, other navigation** → the `beforeunload` event shows the native browser dialog. It is skipped when `allowReload` is set (so an already-confirmed refresh does not prompt twice).
- `pointerdown` (once) marks interaction — prevents Chrome 91+ from disabling `beforeunload` on pages that have not been touched yet.

After a confirmed reload, **state replay still works**: DOME resends CREATE_WINDOW + stored states to the new WebSocket connection, so the desktop is fully restored.

Safe restore (`_savedRect` → `_origRect`)

When a window is restored (e.g. after minimize), `handleRestoreWindow` uses `_savedRect`. If `_savedRect` has zero dimensions (`width <= 0 || height <= 0`) — e.g. leftover from maximize while the window is still `display:none` — the code **falls back to `win.el._origRect`**. This prevents the window from being restored to `(0,0,0,0)` (top-left corner with odd dimensions).

Snippet (code level)

Pruning (`pruneWindowState`)

```
const collectNodeIds = (node: any, ids: Set<string>): void => {
  if (!node || typeof node !== "object") return;
  if (typeof node.id === "string" && node.id) ids.add(node.id);
  if (Array.isArray(node.children)) {
    for (const child of node.children) collectNodeIds(child, ids);
  }
};

const pruneWindowState = (wid: string, targetId: string): void => {
  const states = windowStates.get(wid) || [];
  const filtered = states.filter((state: any) => {
    if (state?.type === "MOUNT_NODE") {
      const nodeIds = new Set<string>();
      collectNodeIds(state.node, nodeIds);
      // hapus MOUNT_NODE jika targetId-nya = targetId, atau
```

```
      // nodeId di dalam tree-nya ada yang = targetId (termasuk child)
      if (state.targetId === targetId || nodeIds.has(targetId))
        return false;
    }
    if (state?.type === "UPDATE_PROPS") {
      if (state.targetId === targetId) return false; // exact match saja
    }
    return true;
  });
  windowStates.set(wid, filtered);
};
```

Browser orphan discard

```
if (targetId) {
  const parent = TSIX.findElementById(wid, targetId);
  if (parent) {
    parent.appendChild(domEl);
  } else {
    // Parent not found — discard (jangan fallback ke win.content)
  }
} else {
  win.content.appendChild(domEl);
}
```

Cross-layer search (`findElementById`)

```
function findElementById(wid, nodeId) {
  // 1) di dalam window milik app
  const win = state.windows.get(wid);
  if (win) {
    const el = win.el.querySelector(
      '[data-tsix-id="' + CSS.escape(nodeId) + '"]',
    );
    if (el) return el;
  }
  // 2) start menu global
  const gm = document.getElementById("__global_start_menu__");
  if (gm) {
    if (gm.getAttribute("data-tsix-id") === nodeId) return gm;
    const child = gm.querySelector(
      '[data-tsix-id="' + CSS.escape(nodeId) + '"]',
    );
    if (child) return child;
  }
  // 3) overlay layer (launcher, dialog)
  const overlay = document.getElementById("__tsix_overlay_layer__");
  if (overlay) {
    if (overlay.getAttribute("data-tsix-id") === nodeId) return overlay;
    const child = overlay.querySelector(
      '[data-tsix-id="' + CSS.escape(nodeId) + '"]',
    );
    if (child) return child;
  }
  return null;
}
```

Navigation protection (`protectNavigation`)

```
(function protectNavigation() {
  let allowReload = false; // true saat user sudah konfirmasi refresh

  // Tandai interaksi user — Chrome 91+ mematikan beforeunload
  // jika halaman belum pernah disentuh.
  document.addEventListener("pointerdown", function () { }, { once: true });

  const isRefreshKey = (e) =>
    e.key === "F5" ||
```

```
    ((e.key === "r" || e.key === "R") && (e.ctrlKey || e.metaKey));

  document.addEventListener(
    "keydown",
    function (e) {
      if (!isRefreshKey(e) || e.repeat) return;
      e.preventDefault();
      const ok = window.confirm(
        "Yakin mau refresh TDE?\n\n" +
        "Semua window yang berjalan akan ditutup lalu di-restore " +
        "ulang dari server.\n\n" +
        "OK = refresh, Cancel = batal",
      );
      if (ok) {
        allowReload = true;
        window.location.reload();
      }
    },
    true, // fase capture
  );

  window.addEventListener("beforeunload", function (e) {
    if (allowReload) return; // refresh yang sudah dikonfirmasi
    e.preventDefault();
    e.returnValue = "";
  });
})();
```

Restore fallback (`_savedRect` → `_origRect`)

```
let saved = win._savedRect;
// Safety: rect nol (mis. sisa maximize saat window masih hidden)
// → jangan restore ke (0,0,0,0), pakai posisi/ukuran asli.
if (saved && (saved.width <= 0 || saved.height <= 0)) {
  saved = win.el._origRect || null;
}
```

Exercises / Practice

1. Open several windows (including dialog/modal), then press F5 — a confirmation dialog appears. Choose **OK** — observe the UI restored via state replay. Choose **Cancel** — the desktop is unchanged.
2. Close a window that has a child dialog, then F5 (OK) — observe no dialog residue.
3. Read `src/mirror/opt/dome/dome.ts` — find `windowStates`, `pruneWindowState`, and the replay block on reconnect.
4. Read `src/mirror/opt/dome/dome-client-dom.js` — find the orphan discard in `handleMountNode`.
5. Read `src/mirror/opt/dome/dome-client-core.js` — find `findElementById` (3 layers) and `protectNavigation` (`allowReload`).
6. Read `src/mirror/opt/dome/dome-client-windows.js` — find the `_savedRect` → `_origRect` fallback in `handleRestoreWindow`.

References

- `wiki/PIXELSPACE_DEVELOPER_GUIDE.md` §11
- `wiki/course/00-overview.md` §10
- `src/mirror/opt/dome/dome.ts` (windowStates, pruneWindowState, replay)
- `src/mirror/opt/dome/dome-client-dom.js` (orphan discard)
- `src/mirror/opt/dome/dome-client-core.js` (findElementById, protectNavigation)
- `src/mirror/opt/dome/dome-client-windows.js` (_savedRect → _origRect)

Module 22 — complete. Part VII is done. Continue to Module 23 — Development Workflow.

Module 23 **Development Workflow** Development

RFC-TSIX-EDU-002 | Twenty-third module of the TSIX curriculum. Understand the host↔VFS cycle: install, vfs-bootstrap, sync-vfs, vfs-pull, SYNC_TO_HOST, userlib-update, and SDK mirroring.

> TSIX has two worlds: the **host** (`src/mirror` + `src/common` folders in the repo) and the **VFS** (file database, its path is read from `kernel.database` in `src/sysconfig.json`, default `system.db`). Developers write on the host, then sync to the VFS. There is also the reverse path — applications inside the VFS write back to the host (root-only).

Learning Objectives

- [ ] Explain the host ↔ VFS cycle
- [ ] Distinguish `install`, `vfs-bootstrap`, `sync-vfs`, `vfs-pull`
- [ ] Explain `SYNC_TO_HOST` (syscall)
- [ ] Explain `userlib-update` (SDK mirroring)
- [ ] Explain the role of tsconfig paths (`@tsix/*`)

Core Concepts

Two worlds: host vs VFS

- **Host** — the place to write code: `src/mirror/` (rootfs: `bin`, `lib`, `etc`, `sbin`, ...) and `src/common/` (framework: `SyscallCode`, `IPCTypes`, `GUITypes`).
- **VFS** — the filesystem booted by the kernel: the file database `system.db` (BKFS/SQLite). The kernel reads its path from `kernel.database`.
- **`src/root/`** — the result of `vfs-pull`: a copy of the VFS on the host for inspection / permanent changes.

All host scripts get the DB path via `scripts/lib/db-path.ts`, so its value is always in sync with the installed path.

> [!IMPORTANT] Since `install.ts` exists, a fresh install **does not need** a separate `vfs:bootstrap` run — the installer already performs the whole rootfs sync itself.

Script table & sync direction

Command	File	Direction	Purpose
`npm run install`	`scripts/install.ts`	host → DB	Interactive fresh install: create new DB + write `sysconfig.json` + full rootfs sync
`npm run vfs:bootstrap`	`scripts/vfs-bootstrap.ts`	host → DB	Bulk sync `src/mirror` → `/` and `src/common` → `/lib/common`
`npx ts-node scripts/sync-vfs.ts <path>`	`scripts/sync-vfs.ts`	host → DB	Sync one file (wired to VS Code "run on save")
`npm run vfs:pull`	`scripts/vfs-pull.ts`	DB → host	Pull the whole VFS → `src/root` (except runtime folders)
syscall `SYNC_TO_HOST` (53)	`src/kernel/Syscalls.ts`	DB → host	Root-only: write one VFS file to the host (limited to project root)
`sudo userlib-update`	`src/mirror/sbin/userlib-update.ts`	DB → host	Inside TSIX: sync `/lib` → `src/.tsix_sdk/lib`
`npm run bkfs:create`	`scripts/create-bkfs.ts`	—	Create an empty DB (optional standard directory skeleton)

Shared configuration

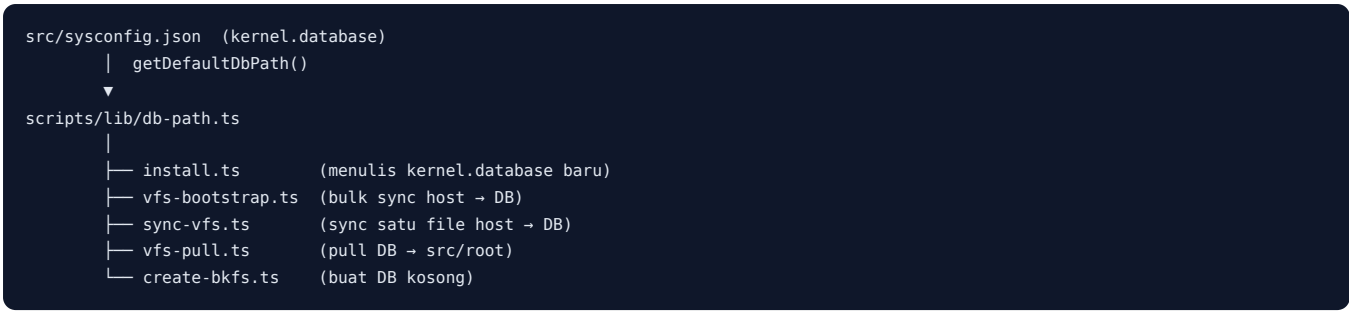
- `src/sysconfig.json` — `kernel.database` (DB path), `kernel.rootHostPath`, `scheduler.workerEntryPath`, `scheduler.bootEntry`, `network.interfaces`.
- `tsconfig.json` — `@tsix/*` → `src/.tsix_sdk/lib/*`, `src/root/lib/*`, `src/mirror/lib/*`; `@common/*` → `src/common/*`; `@bin/*` → `src/root/bin/*`, `src/.tsix_sdk/bin/*`.

> [!NOTE] There is no automatic host → VFS sync at boot. The host is the "writing surface"; the VFS is the runtime source of truth. Changes are moved via the scripts/syscalls above.

Flow / How It Works

Host ↔ VFS cycle diagram

sysconfig → db-path → scripts



1. Fresh install (`npm run install`)

1. Ask interactive configuration: hostname, default user login, MQTT broker, per-interface address, default MQTT port, kernel verbose, new DB path, (optional) root password.
2. Write the result to `src/sysconfig.json` (stores the new `kernel.database`).
3. Create a new `.db` file. If the file already exists and without `--force` → stop; with `--force` → the old file is backed up to `*.bak-<timestamp>`.
4. Sync rootfs: `src/mirror` → `/`, then `src/common` → `/lib/common`. Each `.ts` is transpiled to a sidecar `.js`; executable directories get execute mode (`/sbin` = `0744`, others `0755`); `login`, `passwd`, `sudo` get SetUID (`4755` root).
5. Sync explicit `/etc` files without extension: `passwd`, `shadow` (mode `0640`), `group`, `crontab`, `profile`, `motd`, `fstab.md`, `pkg-demo.conf`.

6. Write a fresh `fstab.json` — only `/tmp` as ramfs (mode `1777` sticky); device-specific mounts are not carried over.
7. Clear `/etc/crontab`.
8. If a root password is provided → it is bcrypt-hashed and written to `/etc/shadow`.
9. Verify: total nodes, number of `passwd` accounts, group list, `shadow` entries.

```
npm run install                            # interaktif, path dari sysconfig
npm run install -- --path data/tsix.db     # path database tertentu
npm run install -- --path data/tsix.db --force   # timpa (auto-backup)
npm run install -- --defaults              # non-interaktif, semua default
npm run install -- --no-config             # skip tulis sysconfig.json
```

2. Bulk sync (`npm run vfs:bootstrap`)

Mass sync of `src/mirror` → `/` and `src/common` → `/lib/common`. Useful for the initial device installation or after many edits at once.

```
npm run vfs:bootstrap                # DB default dari sysconfig
npm run vfs:bootstrap -- data/test.db  # path lain (posisi argumen)
```

3. Sync one file (`sync-vfs`)

For one quick file (e.g. on save in the editor). Path mapping: `src/mirror/*` → `/*`, `src/common/*` → `/common/*`. `.ts` files are also transpiled to a `.js` sidecar.

```
npx ts-node scripts/sync-vfs.ts src/mirror/bin/hello.ts
```

4. Pull back (`npm run vfs:pull`)

Pulls the entire VFS → `src/root`, with mapping: `/etc/*` → `src/root/etc/*`, `/root/*` → `src/root/home/root/*`, others → `src/root/<path>`. Runtime folders are excluded: `dev`, `tmp`, `proc`, `logs`, `var`.

```
npm run vfs:pull
```

5. Syscall `SYNC_TO_HOST` (inside TSIX)

Applications inside the VFS (root) can write files to the host via the `SYNC_TO_HOST` syscall. `userlib-update` and `vfs-pull` (the `sbin` version) use it.

```
# dari shell TSIX, sebagai root
sudo userlib-update   # sync /lib → src/.tsix_sdk/lib
vfs-pull              # tarik seluruh VFS → host root
```

Source Code

- `scripts/install.ts` — fresh install agent (host → DB).
- `scripts/vfs-bootstrap.ts` — bulk sync host → DB.
- `scripts/sync-vfs.ts` — sync one file host → DB.
- `scripts/vfs-pull.ts` — pull DB → `src/root`.
- `scripts/create-bkfs.ts` — create an empty DB.
- `scripts/lib/db-path.ts` — shared DB path from `sysconfig`.
- `src/kernel/Syscalls.ts` — `SYNC_TO_HOST` / `SYNC_FROM_HOST` implementation.
- `src/mirror/lib/UserLib.ts` — `syncToHost()` / `syncFromHost()`.
- `src/mirror/sbin/userlib-update.ts`, `src/mirror/sbin/vfs-pull.ts` — commands inside TSIX.

Snippet (code level)

Shared DB path (`scripts/lib/db-path.ts`)

```
export function getDefaultDbPath(): string {
  const configPath = path.resolve(__dirname, "../../src/sysconfig.json");
  try {
    const raw = fs.readFileSync(configPath, "utf8");
    const cfg = JSON.parse(raw);
    if (cfg && typeof cfg.kernel === "object" && cfg.kernel.database) {
      return String(cfg.kernel.database);
    }
  } catch (_) {
    /* abaikan — pakai fallback */
  }
  return "system.db";
}
```

Execute mode & SetUID (`isSetuidBinary` / `applyBinaryMode`)

```
const EXEC_DIRS = ["/bin", "/sbin", "/usr/bin", "/usr/local/bin"];

function isSetuidBinary(vfsPath: string): boolean {
  return /\/bin\/(login|passwd|sudo)\.(ts|js)$/.test(vfsPath);
}

function isExecutableBinary(vfsPath: string): boolean {
  return EXEC_DIRS.some((d) => vfsPath.startsWith(d + "/"));
}

function applyBinaryMode(bkfs: BKFS, vfsPath: string, label = "INSTALL"): void {
  if (isSetuidBinary(vfsPath)) {
    bkfs.chmod(vfsPath, 0o4755);   // setuid root
    bkfs.chown(vfsPath, 0, 0);
  } else if (isExecutableBinary(vfsPath)) {
    // /sbin = root-only (0o744), lainnya 0o755
    bkfs.chmod(vfsPath, vfsPath.startsWith("/sbin/") ? 0o744 : 0o755);
  }
}
```

Recursive `syncDir` (`vfs-bootstrap.ts`, identical in `install.ts`)

```
const syncDir = (hostDir: string, vfsDir: string) => {
  if (!bkfs.exists(vfsDir)) bkfs.mkdir(vfsDir, 0, 0, 0o755);
  const items = fs.readdirSync(hostDir);
  for (const item of items) {
    const fullHostPath = path.join(hostDir, item);
    const fullVfsPath = path.join(vfsDir, item).replace(/\\/g, "/");
    const stats = fs.statSync(fullHostPath);

    if (stats.isDirectory()) {
      syncDir(fullHostPath, fullVfsPath);
      continue;
    }

    const isTarget =
      item.endsWith(".ts") || item.endsWith(".js") ||
      item.endsWith(".json") || item.endsWith(".html") ||
      item.endsWith(".css") || item.endsWith(".menu") ||
      item.endsWith(".mp3") || item.endsWith(".wav");
    if (!isTarget) continue;

    // Binary (mp3/wav) disimpan sebagai latin1
    const isBinary = item.endsWith(".mp3") || item.endsWith(".wav");
    const content = isBinary
      ? fs.readFileSync(fullHostPath).toString("latin1")
      : fs.readFileSync(fullHostPath, "utf8");
    bkfs.touch(fullVfsPath, content);

    // Transpile TS → JS sidecar (yang dieksekusi runtime)
    if (fullVfsPath.endsWith(".ts")) {
```

```
      const result = esbuild.transformSync(content, {
        loader: "ts", format: "cjs", target: "node18", sourcemap: "inline",
      });
      if (result.code) {
        const jsPath = fullVfsPath.substring(0, fullVfsPath.length - 3) + ".js";
        bkfs.touch(jsPath, result.code);
        if (isSetuidBinary(jsPath) || isExecutableBinary(jsPath)) {
          applyBinaryMode(bkfs, jsPath);
        }
      }
    }

    if ((isSetuidBinary(fullVfsPath) || isExecutableBinary(fullVfsPath)) &&
        (fullVfsPath.endsWith(".ts") || fullVfsPath.endsWith(".js"))) {
      applyBinaryMode(bkfs, fullVfsPath);
    }
  }
};
```

`install.ts` flow (summarized from source)

```
async function main() {
  const opts = parseArgs(process.argv.slice(2));
  const cfg = loadConfig();
  const dbRel = opts.dbPath || cfg.kernel.database || "system.db";
  let rootPassword = "";

  // 1. Konfigurasi interaktif → cfg (hostname, user, broker, port, verbose, db path, root password)
  // 2. Tulis src/sysconfig.json
  if (opts.writeConfig) {
    cfg.kernel.database = path.relative(PROJECT_ROOT, dbPath).replace(/\\/g, "/");
    saveConfig(cfg);
  }
  // 3. Buat DB baru (file lama di-backup bila --force)
  const bkfs = new BKFS(dbPath);
  // 4. Sync rootfs
  syncDir(bkfs, MIRROR_ROOT, "/");            // src/mirror → /
  syncDir(bkfs, COMMON_ROOT, "/lib/common");  // src/common → /lib/common
  // 5. fstab fresh (hanya /tmp ramfs), crontab kosong, password root opsional
  // 6. Verifikasi & tutup DB
}
```

Syscall `SYNC_TO_HOST` (`src/kernel/Syscalls.ts`)

```
case SyscallCode.SYNC_TO_HOST: {
  if (!this.isRoot(pcb))
    throw new Error("Permission Denied: Only root or root group members can perform physical sync.");
  const { vfsPath, hostPath } = args;

  // Keamanan: hostPath tidak boleh keluar dari project root
  const projectRoot = process.cwd();
  const absoluteHostPath = path.resolve(projectRoot, hostPath);
  if (!absoluteHostPath.startsWith(projectRoot)) {
    throw new Error("Security Violation: Target path is outside project root.");
  }

  const { vfs, relativePath } = this.mountManager.resolve(vfsPath);
  const content = vfs.read(relativePath);
  if (content === null) throw new Error(`Source file not found in VFS: ${vfsPath}`);

  const parentDir = path.dirname(absoluteHostPath);
  if (!fs.existsSync(parentDir)) fs.mkdirSync(parentDir, { recursive: true });
  fs.writeFileSync(absoluteHostPath, content);
  this.logger.info(`[SYNC] Applied VFS:${vfsPath} -> HOST:${hostPath}`);
  return true;
}
```

`UserLib.syncToHost` (`src/mirror/lib/UserLib.ts`)

```
public async syncToHost(vfsPath: string, hostPath: string): Promise<boolean> {
  return await this.dispatch(SyscallCode.SYNC_TO_HOST, { vfsPath, hostPath });
}
```

Exercises / Practice

1. Run `npm run install -- --defaults` — check `src/sysconfig.json` is written and a new DB is created, then `npm start`.
2. Run `npm run install` (interactive) — fill in hostname & user login, set the root password, observe the per-file sync.
3. Write a new app in `src/mirror/bin/`, then `npm run vfs:bootstrap` — run the app in the TSIX shell.
4. Change one file then `npx ts-node scripts/sync-vfs.ts src/mirror/bin/<file>.ts` — compare its speed vs the full bootstrap.
5. Change a file in the VFS (from the TSIX shell), then `npm run vfs:pull` — check the change appears in `src/root`.
6. Read `wiki/Memulai.md` — follow the flow starting from scratch.

References

- `wiki/Memulai.md`, `wiki/Panduan-Developer.md`
- `wiki/course/00-overview.en.md` §11 (Development Flow / VFS Sync)
- `wiki/Virtual-File-System.md` — VFS ↔ Host Synchronization
- `scripts/install.ts`, `scripts/vfs-bootstrap.ts`, `scripts/sync-vfs.ts`, `scripts/vfs-pull.ts`, `scripts/create-bkfs.ts`, `scripts/lib/db-path.ts`
- `src/kernel/Syscalls.ts`, `src/mirror/lib/UserLib.ts`, `src/mirror/sbin/userlib-update.ts`, `src/mirror/sbin/vfs-pull.ts`

Module 23 — done. Continue to Module 24 — Best Practices & App Writing.

Module 24 **Best Practices & Writing Apps** Development

RFC-TSIX-EDU-002 | Module twenty-four of the TSIX curriculum. Closes the curriculum with best practices: two app styles, error = window, themes, UPDATE_PROPS batching, and the Piagam Antigonon.

> The curriculum is complete here. This module is the "practice book" — a collection of rules that keep TSIX applications consistent, secure, and easy to maintain.

Learning Objectives

- [ ] Distinguish the two app styles (`IProgram` vs `Program()`)
- [ ] Explain "error = window" (the 4-step `std.error`)
- [ ] Explain `UPDATE_PROPS` batching
- [ ] Explain the Piagam Antigonon (4 GUI rules)
- [ ] Apply best practices to new apps

Core Concepts

Two application styles

1. Class `main implements IProgram` (classic style, the majority in `/bin`):

The contract is in `src/mirror/lib/IProgram.ts`:

```
export interface OSContext {
    std: StdLib;
    fs: FsLib;
    shell: ShellLib;
    aux: AuxLib;
}

export interface IProgram {
    execute(os: OSContext, args: string[]): Promise<string | void>;
}
```

Minimal example (the `cat.ts` / `echo.ts` pattern):

```
import { IProgram, OSContext } from "../lib/IProgram";

export class main implements IProgram {
    async execute(os: OSContext, args: string[]): Promise<string | void> {
        const { std } = os;
        await std.print("Hello from TSIX!\n");
        // void → WorkerEntry memanggil shell.exit(0)
    }
}
```

2. `Program()` wrapper + proxy singletons (new, recommended for new apps & GUI):

```
import { Program, std } from "@tsix/Application";

export const main = Program(async (args: string[]) => {
    await std.print("Hello from TSIX!\n");
});
```

> Alias advantage: **path independence** — the import stays the same no matter where the file is placed (`/bin/` , `/root/test/` , etc.). It eliminates "Module not found" errors when moving folders.

Comparison of the two styles:

Aspect	`main implements IProgram`	`Program()` wrapper
Import	`import { IProgram, OSContext } from "../lib/IProgram"`	`import { Program, std, fs, shell, net, db } from "@tsix/Application"`
Entry point	`async execute(os: OSContext, args)`	callback function `async (args) => ...`
Library access	`os.std` , `os.fs` , `os.shell` , `os.aux`	proxy singletons `std` , `fs` , `shell` , `net` , `db`
Path	relative import to the file location	absolute import `@tsix/*` — path independent
Error handling	manual (WorkerEntry catches other errors → `realExit(1)`)	automatic: the wrapper catches errors → `std.error()` → rethrow
Suitable for	CLI utilities in `/bin`	new apps, GUI (Emerald), daemons

How the wrapper works (`src/mirror/lib/Application.ts`): `Program(fn)` returns a class that `implements IProgram` . When `execute` is called, it stores the `OSContext` into `global._tsixOsc` and then calls `fn(args)` . If `fn` throws an error, the wrapper sends that error to the parent via `std.error()` (it becomes a desktop error window), then rethrows so that WorkerEntry also handles it. The `std` / `fs` / `shell` / `net` / `db` proxies read `UserLib` from `global._tsixLib` lazily — that is why the imports are path-independent.

Error = "Window"

Don't let an app crash silently. `std.error(message, context?, wid?)` displays the error as a **desktop window popup** (processed by WM/Asteracea). The 4-step flow (see `StdLib.error()` in `src/mirror/lib/UserLib.ts`):

1. **Log to syslog** — write a `[ERROR]` line + timestamp to `/var/log/syslog` .
2. **Extract `fileHint`** — read the stack trace, find the first source file that is not an internal library (`UserLib` , `Application` , `emerald`).
3. **Broadcast to the parent (WM)** — send an IPC `GUI_WINDOW_ERROR` containing `{ wid, pid, file, error, context, timestamp }` to the parent process; Asteracea displays it as a popup.
4. **Print red TTY** — print `\x1b[31m[ERROR]\x1b[0m <message>` to the terminal (stderr style).

Each step is non-fatal — if one fails (e.g., syslog not yet present), the other steps still run.

Batching UPDATE_PROPS

Don't send one `UPDATE_PROPS` per property change. **Batch** several changes and send them together — this reduces the IPC count and render latency. In Emerald (`src/mirror/lib/emerald.ts` , class `Window`) the mechanism is already automatic:

- `updateProps(targetId, props)` does not send immediately — the changes are merged into `dirtyProps` (a Map), then `scheduleFlush()` is scheduled.
- `scheduleFlush()` uses `setTimeout(..., 0)` : all changes within one async tick are collected first, then flushed at the end of the tick. If a flush is already scheduled, there is no double scheduling.
- `flushNow()` sends all pending `UPDATE_PROPS` at once.
- `Screen.update()` / `setText()` / `setVisible()` / `setStyle()` all use this batch path.
- `setContent()` is deliberately **not** batched (sent directly via `sendImmediate`) — one atomic `innerHTML=""` operation followed by `MOUNT_NODE` per child.
- `Screen.on()` calls `flush()` automatically after binding — ensuring the listener is attached in the browser.

Best practice: don't write a loop `for (...) await app.update(id, props)` expecting each item to be sent individually — Emerald already batches it for you.

Piagam Antigonon (4 GUI rules)

The Piagam Antigonon is a strict rule set for AI agents & developers who write TSIX GUIs (source: `wiki/PIXELSPACE_DEVELOPER_GUIDE.md`):

1. **NO DOM in Userland** — `@tsix/emerald` MUST NOT touch `document.*` or `window.*`. All rendering goes through the PixelSpace protocol (Worker → Kernel → DOME → Browser).
2. **State-Sync** — Don't send `UPDATE_PROPS` in a tight loop; use batching.
3. **Memory Cleanup** — Every unmounted node must have its event listeners cleaned up (prevent memory leaks).
4. **Type Safety** — All payloads must conform to `IGUIPayload`; malformed payload → `SIGKILL`.

> [!TIP] Two related practices are equally important: attach **mount-time** listeners (`onClickId` / `onInputId` in props at build time, not `app.on()` after mount) to avoid the `cloneNode` bug (Modules 19 & 20), and keep the UI safe for **state replay** after F5 (Module 22).

Flow / How It Works

Steps to write a correct TSIX app (the `Program()` style):

1. **Choose the style** — short CLI utilities: `main implements IProgram`. New apps & GUI: `Program()`.
2. **Write the entry point** — `export const main = Program(async (args) => {...})` (or `export class main implements IProgram`).
3. **Declare GUI mode** (if the app shows a window) — `export const appMode = "gui";` so WorkerEntry handles the GUI.
4. **Access libraries via proxy** — `std`, `fs`, `shell`, `net`, `db` — import once from `@tsix/Application`.
5. **GUI: use Emerald** — `new Screen({...})`, `app.mount(...)`, `app.on(...)`, `app.loopUntilClose()`.
6. **Error = window** — call `await std.error(msg, context)` on failure; don't let it crash silently.
7. **Theme** — `await theme.loadCurrent()` + `theme.watch()` so colors follow Asteracea preferences.
8. **Batch updates** — leave batching to Emerald; don't send `UPDATE_PROPS` per item.
9. **Follow the Piagam Antigonon** — no direct DOM, no tight loops, clean up listeners, payloads conform to `IGUIPayload`.

Source Code

File	Contents
`src/mirror/lib/IProgram.ts`	`IProgram` & `OSContext` contract
`src/mirror/lib/Application.ts`	`Program()` wrapper + proxy singletons (`std`, `fs`, `shell`, `net`, `db`)
`src/mirror/lib/UserLib.ts`	`StdLib` (`print`, `log`, `error`), `FsLib`
`src/mirror/lib/emerald.ts`	`Window` / `Screen`, `UPDATE_PROPS` batching, `bindHandler`
`src/mirror/lib/theme.ts`	`ThemeProvider` — `loadCurrent`, `watch`, `switchTo`, `applyToDome`
`src/mirror/bin/cat.ts`, `echo.ts`	`IProgram` style examples
`src/mirror/root/ps-sample2.ts`, `ps-sample3.ts`	`Program()` + Emerald examples
`src/mirror/opt/set-theme/set-theme.ts`	Theme example (dark/light switch)
`wiki/PIXELSPACE_DEVELOPER_GUIDE.md`	Piagam Antigonon

Snippet (code level)

App CLI — `IProgram` style (readfile)

`execute(os, args)` uses `os.std` / `os.fs` (the `OSContext` contract). The returned string is printed by WorkerEntry; `void` → `shell.exit(0)`:

```
import { IProgram, OSContext } from "../lib/IProgram";

export class main implements IProgram {
    async execute(os: OSContext, args: string[]): Promise<string | void> {
        const { std, fs } = os;
        if (args.length === 0) {
            return "Usage: readfile <path>"; // string return → dicetak WorkerEntry
        }
        const fd = await fs.open(args[0], "r"); // fd < 0 = gagal buka
        if (fd < 0) {
            await std.print(`Error: Cannot open ${args[0]}\n`);
            return;
        }
        const content = await fs.read(fd);
        await std.print(content ?? "");
        await fs.close(fd);
    }
}
```

> [!TIP] To read a file in one shot, `FsLib` already provides `readFile(path)`, which wraps `open` → `read` → `close`.

App CLI — `Program()` style (hello)

```
import { Program, std } from "@tsix/Application";

export const main = Program(async (args: string[]) => {
    await std.print(`Hello, ${args[0] ?? "world"}!\n`);
});
```

`std.error` — error becomes a window

```
// Log syslog → ekstrak fileHint → broadcast GUI_WINDOW_ERROR → TTY merah
await std.error("Disk full", "myapp");
await std.error("Connection timeout", "net", app.wid); // wid opsional
```

When the app calls this, WM/Asteracea shows an error popup on the desktop — not a console crash.

Theme — import & apply

```
import { theme } from "@tsix/theme";
import { Screen, div } from "@tsix/emerald";

// Muat tema aktif (terang/gelap) sesuai prefs Asteracea
await theme.loadCurrent();
theme.watch(); // ikut perubahan tema saat app berjalan

// Pakai warna terpusat — tanpa hardcode hex
const app = new Screen({ title: "App", width: 400, height: 300 });
await app.mount(
  div({
    id: "root",
    style: { background: theme.colors.bg, color: theme.colors.text },
  }),
);

// Ganti tema global + broadcast THEME_CHANGED ke DOME
await theme.switchTo("theme-light.json");
// Terapkan ke window ini (titlebar, border, shadow)
await theme.applyToDome(domePid, app.wid);
```

Theme files are located in `/opt/asteracea/theme-*.json` (`theme-dark.json`, `theme-light.json`). Full example: `src/mirror/opt/set-theme/set-theme.ts`.

Exercises / Practice

1. Write a "Hello" app in both styles — compare the structure & imports.
2. Add `std.error()` to an app that deliberately crashes — observe the error window on the desktop.
3. Refactor a GUI app to batch UPDATE_PROPS — measure the difference in responsiveness.
4. Read `wiki/DEVELOPER_GUIDE_SCRIPTING-V2.md` — work through the v2.1 script examples.

References

- `wiki/PIXELSPACE_DEVELOPER_GUIDE.md` — Piagam Antigonon & PixelSpace guide
- `wiki/DEVELOPER_GUIDE_SCRIPTING-V2.md`, `wiki/Panduan-Developer.md`
- `wiki/course/00-overview.md` §9
- `src/mirror/lib/IProgram.ts`, `src/mirror/lib/Application.ts`, `src/mirror/lib/UserLib.ts`
- `src/mirror/lib/emerald.ts`, `src/mirror/lib/theme.ts`
- `src/mirror/root/ps-sample2.ts`, `src/mirror/root/ps-sample3.ts`
- `src/mirror/opt/set-theme/set-theme.ts`, `src/mirror/opt/test/gui-demo.ts`, `src/mirror/opt/file-cruiser/file-cruiser.ts`
- `src/mirror/bin/cat.ts`, `src/mirror/bin/echo.ts`

Module 24 — complete. The TSIX curriculum is complete! 🎉 Keep writing? Update `toc.md` and create the `.en.md` translation (FORMAT §5).